CORE-ET CPU Subsystem Micro-Architecture Reference

Table of Contents

Document Control . . . 1
Document Author . . . 1
Revision History . . . 1
Source Material and Acknowledgements . . . 1
1. Neighborhood . . . 3
1.1. Neighborhood Overview . . . 3
1.1.1. Partitioning . . . 3
1.1.2. Shared Resources and Core Connections . . . 3
1.1.3. Voltage-Domain Crossings . . . 3
1.1.4. External Interfaces . . . 4
1.2. Neighborhood Instruction Cache . . . 4
1.2.1. Structure and Connections . . . 5
1.3. Front End – I-Cache Interface . . . 6
1.3.1. Overview . . . 6
1.3.2. Interface Signals . . . 7
1.3.3. Request Arbitration . . . 8
1.3.4. Request and Response Timing . . . 10
1.3.5. Miss, Sleep and Fill-Done Protocol . . . 11
1.3.6. Fault and Error Reporting . . . 12
1.3.7. Complete Path for Four Cores . . . 13
1.3.8. Livelock and Starvation Avoidance . . . 15
1.3.9. Known Limitations . . . 21
1.4. L0 Micro-Cache . . . 21
1.4.1. Overview . . . 21
1.4.2. Interface . . . 23
1.4.3. Cache Organization and Address Decode . . . 28
1.4.4. Pipeline Stages . . . 30
1.4.5. Instruction TLB . . . 34
1.4.6. Micro-Tag and Micro-Data Arrays . . . 44
1.4.7. PMA Check . . . 47
1.4.8. Miss Handling . . . 48
1.4.9. Flush . . . 53
1.4.10. Bypass Mode . . . 54
1.4.11. Code Prefetcher . . . 59
1.4.12. Debug Access . . . 65
1.5. L1 Instruction Cache . . . 66
1.5.1. Overview . . . 66
1.5.2. Interface . . . 67

1.5.3. Cache Organization and Address Decode . . . 69
1.5.4. Request Arbitration Between L0 Micro-Caches . . . 71
1.5.5. Pipeline and Hit Timing . . . 71
1.5.6. Tag, LRU and Data Arrays . . . 73
1.5.7. External Data RAM . . . 74
1.5.8. Miss Path . . . 74
1.5.9. Errors and ECC . . . 76
1.5.10. Flush, Bypass Mode and Prefetch . . . 76
1.5.11. Debug Access . . . 76
1.6. Shared Page-Table Walker . . . 77
1.6.1. Clients and Arbitration . . . 77
1.6.2. Walk Sequence and Page Sizes . . . 77
1.6.3. PTE Cache and Invalidation . . . 77
1.7. Memory Request and Response Path . . . 77
1.7.1. Miss, Evict and Fill FIFOs . . . 77
1.7.2. Request Arbitration . . . 77
1.7.3. Response Routing and Backpressure . . . 77
1.8. Tensor Support . . . 77
1.8.1. Cooperative Tensor Load . . . 77
1.8.2. Tensor Store Buffer . . . 77
1.8.3. TBOX Router . . . 77
1.9. Synchronization, Messaging, Registers and PMU . . . 78
1.9.1. Fast Local Barrier . . . 78
1.9.2. Local Message Network . . . 78
1.9.3. IPI and FCC Paths . . . 78
1.9.4. Neighborhood Registers and APB Access . . . 78
1.9.5. Debug Channel . . . 78
1.9.6. Performance Monitoring Unit . . . 78

Document Control

Document Author

Ryan Naveed, 10xEngineers — ryan.naveed@10xengineers.ai

Revision History

Version	Date	Author	Notes
v0.1	2026-10-06	Ryan Naveed	• First draft of the document. • Chapter 1, ET-SoC-1 Overview: written, except the Memory Map, which contains a heading only. • Chapter 2, ET-Minion Core: Core Overview (block partitioning, hardware threads, instruction flow) and Front End written. Integer Pipeline, Data Cache and VPU contain headings only. • Chapter 3, Neighborhood: the Overview and the Front End — I-Cache Interface are written. The remaining sections contain headings only.

Source Material and Acknowledgements

The original micro-architecture documents are in `docs/uarch/` of this repository, one per module. Where a document and the RTL differ, the RTL is followed.

The engineers who worked on each module are listed below.

Engineer	Module documents
Albert Navarro	FE/Intpipe Description
Francisco Torres	Minion DCache Description
Guillem Solé	Cooperative TensorLoad Description
Ildefonso Gomariz	CORE-ET Neighborhood MAS; Neighborhood I-Cache Description; Front End — I-Cache Interface; CORE-ET Minion Shire Description; Cooperative TensorLoad Description; ET-Link Specification
Jordi Muñoz	Minion VPU Specification
Juanjo Galindo	CORE-ET Minion Shire Description; Minion Shire DLL Constraints
Miquel Izquierdo	ET-Link Specification
Sebastià Tortella	Minion Description; FE/Intpipe Description; ET-Link Specification
Sri Samudrala	Minion VPU Specification

Engineer	Module documents
Xavier Revés	CORE-ET Neighborhood MAS; Minion Description; Minion DCache Description; CORE-ET Minion Shire Description; Minion Shire DLL Constraints; ET-Link Specification

Chapter 1. Neighborhood

1.1. Neighborhood Overview

1.1.1. Partitioning

Figure: Neighborhood block diagram.

1.1.2. Shared Resources and Core Connections

1.1.3. Voltage-Domain Crossings

A Neighborhood is divided into two clock and voltage domains:

- **Low-voltage (LV) region**: the eight ET-Minion cores and most of the Neighborhood channel, running on the Neighborhood clock.
- **High-voltage (HV) region**: the logic that interfaces to the Shire Channel, running on the Shire clock.

Blocks were assigned to the two regions for performance and power: the cores and the channel logic run at the lower voltage to save power, while the logic and the memory cells that need speed run at the higher one. Two properties of an SRAM cell decide where its memory has to go, and both improve with voltage:

- **Access time.** A cell driven from a higher supply switches its bit lines faster, so the array is read and written in less time.
- **Noise margin.** Noise margin is the amount of disturbance, from coupling between neighbouring lines, supply droop or process variation, that a cell can absorb before the value it holds or the value being read out flips. It shrinks as the supply falls, so reads and writes become less reliable. A memory macro is therefore characterized with a minimum operating voltage, and carries trim settings, read assist, write assist and write-pulse width, that buy margin back at low voltage at the cost of speed.

Memory cells were kept at the higher voltage for both reasons, which is also why the cells of the crossing FIFOs themselves are always placed in the high-voltage region, whichever direction the interface goes. Margin is not the same as immunity, so the data path from the memory is protected as well: the line returned from the L1 instruction cache data RAM is covered by ECC, the error is reported to the requesting core with the response and it is logged in the Neighborhood ESRs (Section 1.3.6).

The two regions are connected by a semi-synchronous interface, and every path between them passes through voltage-crossing FIFOs, whose crossing elements contain the level shifters. The paths that cross are the memory request and response paths, the fast local barrier path, the debug and ESR accesses, and the SRAM interface of the L1 instruction cache, whose data RAM is in the high-voltage region of the Shire Channel. Each crossing costs latency, which is why these paths are longer than their logic alone suggests.

1.1.4. External Interfaces

Table: Neighborhood interface groups.

1.2. Neighborhood Instruction Cache

A Neighborhood groups eight ET-Minion cores. Each core has its own private L1 data cache, but none of them has a private instruction cache. Instead, the eight cores share one instruction cache, a 32 KB L1 I-Cache that the Neighborhood adds as a shared resource. The cores of a Neighborhood normally execute the same code, so a shared cache holds one copy of it instead of eight, and that saves both area and power.

The shared instruction cache has two levels of access because of the voltage region its memory has to be built in. SRAM cells are faster and hold their contents more reliably at a higher supply voltage, so the cache's data RAM is placed in the high-voltage region, while the Neighborhood itself operates at low voltage (Section 1.1.3).

The cache's data RAM is therefore placed outside the Neighborhood, in the Shire Channel, and every access to it passes through the voltage-domain crossing, which lengthens the access time. Serving every core from that cache alone would also leave all 16 threads of the Neighborhood, eight cores with two threads each, competing for its single request port. Access to that shared port is handled by LRU arbitration, so a thread would wait behind up to 15 others for its turn, even when the line it needs is already present.

To overcome both, the long read latency of its data RAM in another voltage region and the wait for a turn among 16 bidding threads, a micro level was added in front of that shared cache: two small first-level caches, called L0 micro-caches, inside the Neighborhood, next to the cores. Their data arrays are built in the Neighborhood channel, so a hit is served without crossing into the Shire Channel, and each micro-cache serves half the cores, so eight threads share a port instead of 16.

The shared instruction cache of the Neighborhood is thus split into the following hierarchy:

- **L0 micro-caches**: 16 fully associative lines of 64 bytes each, with their own instruction TLB, PMA check, LRU victim selection and page-table walker port (Section 1.6), and a five-stage lookup. There are two of them, called the north and the south micro-cache, serving Minions 0 to 3 and Minions 4 to 7 respectively, that is eight threads each. Both are identical except for one function, the code prefetcher, enabled in the north micro-cache only, for fetching cache lines into the L1 instruction cache.
- **L1 instruction cache**: 32 KB, 128 sets x 4 ways, 64-byte lines, shared by both micro-caches and accessed only when one of them misses. Its tags, LRU and control are inside the Neighborhood; its data RAM is the one in the Shire Channel.

Figure 1 shows the structure. Section 1.2.1 describes how the two cache levels are packaged in the RTL and what blocks they connect to. The micro-caches and the L1 instruction cache are described in detail in Section 1.4 and Section 1.5, and the path from the core's Front End to the cache, with its arbitration and priority updates, in Section 1.3.

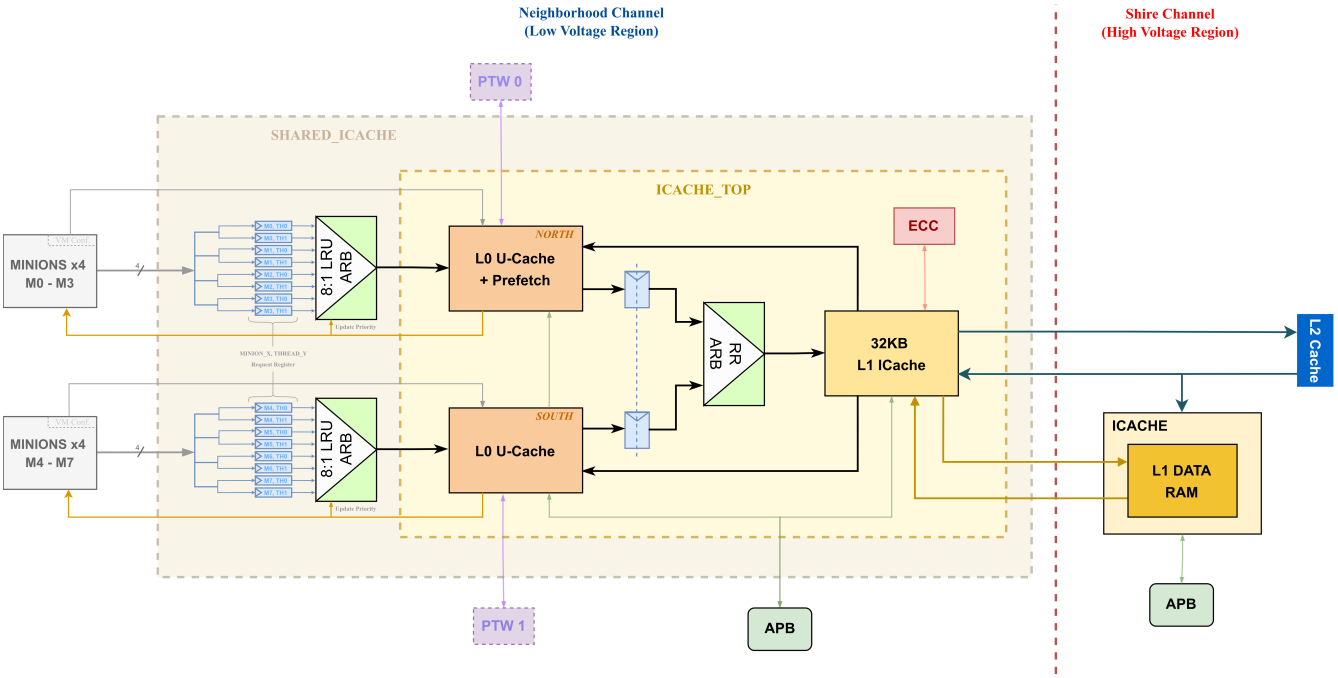


Figure 1. Neighborhood instruction cache

1.2.1. Structure and Connections

The cache is implemented in two modules:

- `icache_top` contains both L0 micro-caches and the L1 instruction cache, everything except the L1 data RAM, that is the micro-cache arrays and pipelines, the instruction TLBs, the PMA checks, the L1 tag, LRU and data-array control, and the round-robin arbiter through which the two micro-caches reach the L1.
- `neigh_shared_icache` wraps `icache_top` and adds the Neighborhood-side logic of the core interface, the logic that receives the fetch request of each of the eight cores, holds it per thread and presents one of them to a micro-cache: one request register per thread, the 8:1 LRU arbiter that selects among them, and the per-thread miss counters and bid mask that prevent livelock. These are described in Section 1.3.

The top-level `neigh_shared_icache` block connects to five things outside itself (Table 1):

Table 1. Shared instruction cache external interfaces

Interface	Purpose
Cores	Fetch requests and address-translation configuration, one port per core. Responses and the fill-done indication, one port per L0 micro-cache, each driven to the four cores that micro-cache serves.
Page-table walkers	Translation requests from the instruction TLB of each micro-cache, one walker per micro-cache.
Shire Cache	Line requests on an L1 miss, and their responses, over the Neighborhood request and response path.
L1 data RAM	SRAM read and write requests to the data RAM in the Shire Channel, and the read data returned from it.

Interface	Purpose
ESR and APB	Configuration (prefetch, bypass, protection and page size), error logging, and debug access to the arrays.

The signals of each interface are listed with the block that uses them: the core-side signals in Section 1.3.2, and the rest in the detailed sections of the two cache levels.

1.3. Front End – I-Cache Interface

This section describes the fetch protocol between the ET-Minion cores and the instruction cache, from the Neighborhood side. The core side is described in [sec-ch2-fe-l0].

1.3.1. Overview

The eight cores of a Neighborhood are served by two L0 micro-caches, each shared by four cores, that is by eight threads, two threads per core. Both L0 micro-caches are backed by the L1 instruction cache of the Neighborhood. Figure 2 shows the whole fetch path and names the signals of the interface: the two thread buffers and the 2:1 LRU arbiter inside each core, the per-thread request registers and the 8:1 LRU arbiter of each micro-cache, and the round-robin arbiter through which both micro-caches reach the L1 instruction cache.

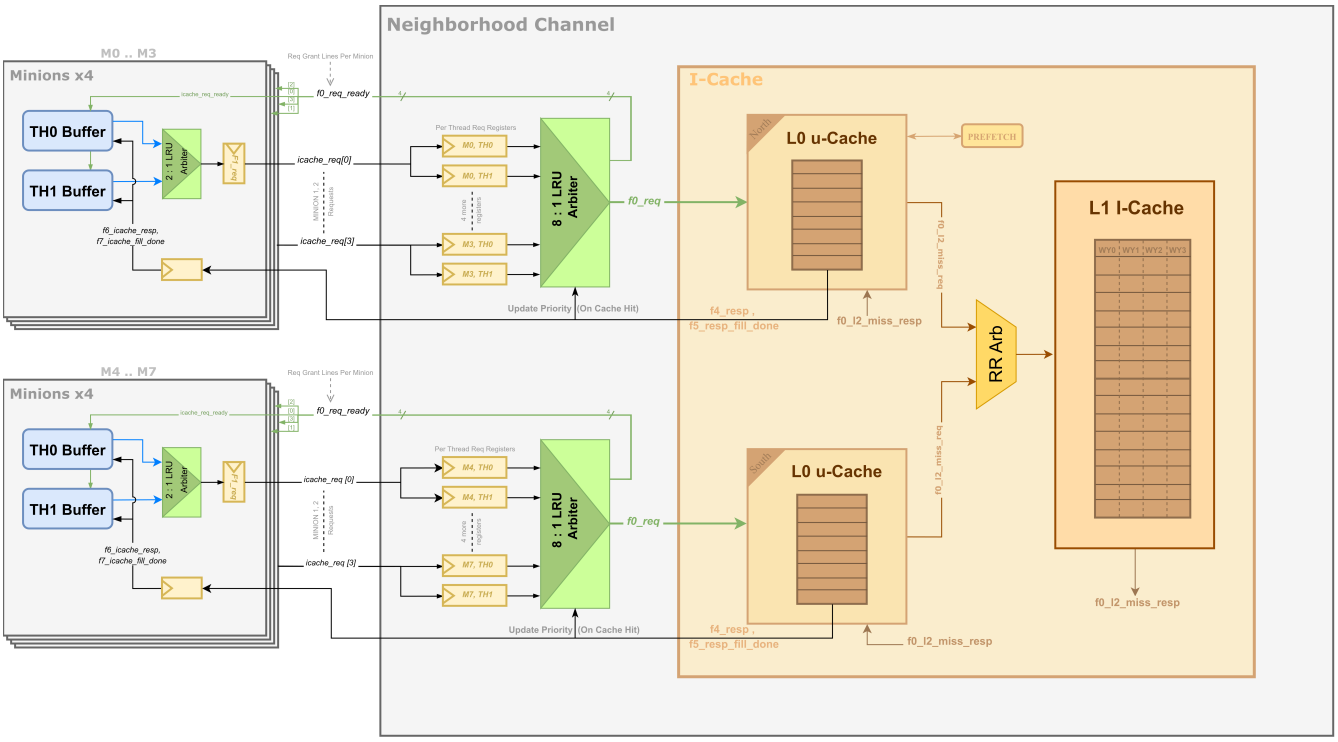


Figure 2. Front End – I-cache interface

A fetch request, that is a request from the Front End of a core for the instruction block at a thread's fetch address, travels as follows:

1. In the core, the 2:1 LRU arbiter selects one of the two threads, and the selected request is flopped into the Front End's F1 request register, which holds it until the Neighborhood accepts it. Each core therefore presents one request at a time.
2. In the Neighborhood, the request is steered into the request register of the thread that made it,

using the thread identifier it carries. One register exists per thread, so eight per L0 micro-cache, which lets the threads of all four cores arbitrate on equal terms.

3. The 8:1 LRU arbiter selects one of the eight captured thread requests and presents it to the L0 micro-cache. The grant is returned to the core in the same cycle, as `icache_req_ready`, and it frees the core's F1 request register: the core's 2:1 arbiter runs in that same cycle and loads the other thread's request, so the core presents it on the next cycle with no idle cycle in between.
4. The L0 micro-cache looks the request up in five stages: arbitration, instruction-TLB translation, tag lookup and PMA check, data-array read, and response.
5. One response is returned a fixed four cycles after the grant, for a hit and for a miss alike, and it reaches the core in its F5 stage. It is shared by the eight threads the micro-cache serves and carries no thread identifier; on a hit it carries the instruction block. How a core recognizes its own response is described in [sec-ch2-fe-track].
6. On a miss, which can be a TLB miss or a line miss, the L0 micro-cache requests a translation from its page-table walker, or requests the line from the L1 instruction cache through a round-robin arbiter that the two micro-caches share. When the fill completes, the micro-cache pulses fill done to all the threads it serves, and the sleeping ones wake and retry.

1.3.2. Interface Signals

Table 2 lists the signals between a single core and the Neighborhood. Directions are given from the Neighborhood side.

Table 2. Front End – I-cache interface signals

Signal	Dir	Width / type	Description
Fetch request			
`icache_req_valid`	in	1	The core presents a fetch request.
`icache_req`	in	`frontend_icache_req`	Requesting thread, its virtual-memory status and the fetch address.
`icache_req_ready`	out	1	The 8:1 arbiter grants the request. Returned in the grant cycle, it frees the core's F1 request register, so the core can load its other thread's request in the same cycle and present it on the next.
Fetch response			
`icache_resp_valid`	out	1	A response is returned in this cycle.
`icache_resp_miss`	out	1	The response is a miss and carries no instruction data.
`icache_resp`	out	`icache_frontend_resp`	32-byte block with its fault, error and cacheable status.
`icache_fill_done`	out	1	A line or translation fill in the L0 micro-cache is complete.
Address translation and maintenance			

Signal	Dir	Width / type	Description
satp_info, matp_info	in	2 × minion_satp_info	Supervisor and machine address-translation registers of the core, used by the instruction TLB.
tlb_invalidate	in	1	Invalidate the instruction-TLB entries of this core.
icache_flush_data	in	1	Invalidate the data stored in the L0 micro-cache.

The fetch response ports are the exception to that one-core view. The Neighborhood drives one set of them per L0 micro-cache, common to the four cores it serves, so icache_resp_valid, icache_resp_miss, icache_resp and icache_fill_done reach all four at the same time. Being common, the response carries no thread identifier and nothing that says which core it belongs to. Each core works that out on its own: it tracks its outstanding request down the pipeline, so it knows in which cycle its response is due and which of its two threads made it, and it ignores the responses of the other three cores ([sec-ch2-fe-track]).

1.3.3. Request Arbitration

This section describes how the Neighborhood decides which thread's request reaches the L0 micro-cache in a given cycle, and when the arbiter updates the priority that decision is based on. Lets start with the registers that hold a thread's request on its way in.

A fetch request is flopped twice before it enters the L0 micro-cache.

First flop, in the core. Two threads share one fetch port of the core, so the 2:1 LRU arbiter selects one of them and its request is held in the Front End's F1 request register. That register keeps the request until the Neighborhood (the L0 micro-cache in this case) accepts it, so a core presents one request at a time, carrying the identifier of the thread that made it.

Second flop, in the Neighborhood. Each micro-cache serves four cores, so it sees at most four requests in a cycle, and it holds one request register per thread, eight in all. Each request is written into the register of the thread its identifier names. At most four of the eight registers are valid at a time, because a core frees its F1 register only when its request is accepted. Eight registers exist nevertheless so that every thread bids for itself, which is what makes the arbitration fair between the two threads of a core as well as between cores.

The eight registers bid through the 8:1 LRU arbiter (arb_lru_delayed_data). A bid is the register's valid qualified with the thread's bid mask. The mask exists to avoid livelock and is described in Section 1.3.8; until then, take it that masking suppresses a thread's valid at the arbiter, so the thread takes no part in the arbitration while its request stays in its register.

The arbiter is stalled while the micro-cache cannot accept a request, which the micro-cache reports with f0_req_ready. Its pipeline never stalls, so this signal is tied high: a request is accepted in every cycle in which at least one thread bids, and the rest of this section takes that for granted.

1.3.3.1. Priority Matrix

The 8:1 LRU arbiter holds its priority in an aging matrix: one bit for every pair of the eight threads it serves, recording which of the two was served more recently. A thread wins when it has priority over every other thread bidding in that cycle, which makes the grant one-hot. Two threads can never hold the same priority, because every pair is ordered by exactly one bit. Granting a thread clears all of its bits, that is it is placed behind every other thread, and the threads with the longest wait move up.

The grant is combinational. In the same cycle it is returned to the core as `icache_req_ready`, the thread's request register is freed, so it reads invalid in the next cycle, and the core may load the request of its other thread into the F1 register. A request is therefore accepted at the earliest in its second cycle in the Front End's F1 stage, because that register has to hold it for one more cycle while it is flopped into the Neighborhood's per-thread register.

1.3.3.2. Deferred Priority Update

Every accepted request gets exactly one response, and it comes back a fixed four cycles after the grant, whether it hits or misses (Section 1.3.4). The arbiter waits for it: priority is not updated when the grant is issued, but when that response arrives.

Waiting means the arbiter has to remember whose response is due, because the response is shared and carries no thread identifier. The grant vector therefore travels down with the request in its own registers, `f1_grant` to `f4_grant`, so that in F4 it can be told which thread the response belongs to (Table 3).

Table 3. Grant pipeline in the Neighborhood

Stage	Register	Content
F1	`f1_grant`	One-hot grant of the arbiter, loaded in the cycle the L0 micro-cache accepts the request
F2	`f2_grant`	Same vector, one cycle later
F3	`f3_grant`	Same vector, one cycle later
F4	`f4_grant`	Thread whose response is returned in this cycle; selects the priority bits and the miss counter to update

The update is enabled by `f4_update_prio`, asserted when a response is valid in F4 and that response is a hit, or reports a page fault or an access fault:

- **Hit.** Priority is updated and the thread moves behind the others, which gives the waiting threads their turn.
- **Miss.** Priority is left unchanged, so the thread keeps its position while it sleeps and is served early when it retries after the fill (Figure 4).
- **Page fault or access fault.** Priority is updated, because the thread does not retry the same fetch: the core records the fault with the instruction and takes the trap (Section 1.3.6).

Bus and ECC errors do not appear in this condition. Both are reported only with a hit, and a hit already updates the priority.

1.3.4. Request and Response Timing

The L0 micro-cache looks a request up in five stages and drives fill done one stage later (Table 4).

Table 4. L0 micro-cache pipeline stages

Stage	Operation
F0	Arbitration; the granted request enters the pipeline.
F1	Instruction-TLB lookup, which translates the virtual address.
F2	Tag lookup, PMA check and LRU update. A miss is detected here, and accepted if the miss handler is idle.
F3	Data array read. On an accepted miss, the request to the L1 instruction cache is driven in this cycle.
F4	Response to the cores (`f4_resp_valid`, `f4_resp_miss`, `f4_resp`).
F5	Fill-done pulse to the cores (`f5_resp_fill_done`), one stage after the response. Not part of the lookup: it reports that a line or translation fill is complete.

These stages are the micro-cache's own, and they are numbered independently of the Front End's F0 to F7: the F1 request register of the core and the F1 stage of the micro-cache are different things.

Every accepted request receives exactly one response from the last stage, for a hit and for a miss alike, so the response arrives a fixed four cycles after the grant. The response is broadcast to the four cores of the L0 micro-cache and carries no thread identifier; each core identifies its own response from the position of its request in time ([sec-ch2-fe-track]).

The fill-done signal is driven one stage later than the response, from the stage after F4.

In the waveforms that follow, a thread is written M0, TH0, meaning thread 0 of Minion 0. Figure 3 shows a request that hits: the core presents it, the 8:1 LRU arbiter grants it in cycle 1, and the response is returned in cycle 5, four cycles later. The grant frees both registers that hold the request, the core's F1 request register and the Neighborhood's per-thread register, which is why both valids fall in the next cycle. `f4_grant` still names M0, TH0, the thread that was granted four cycles earlier, which is how F4 knows whose priority to update: the response is a hit, so `f4_update_prio` is asserted and M0, TH0 moves behind the other threads in the LRU order.

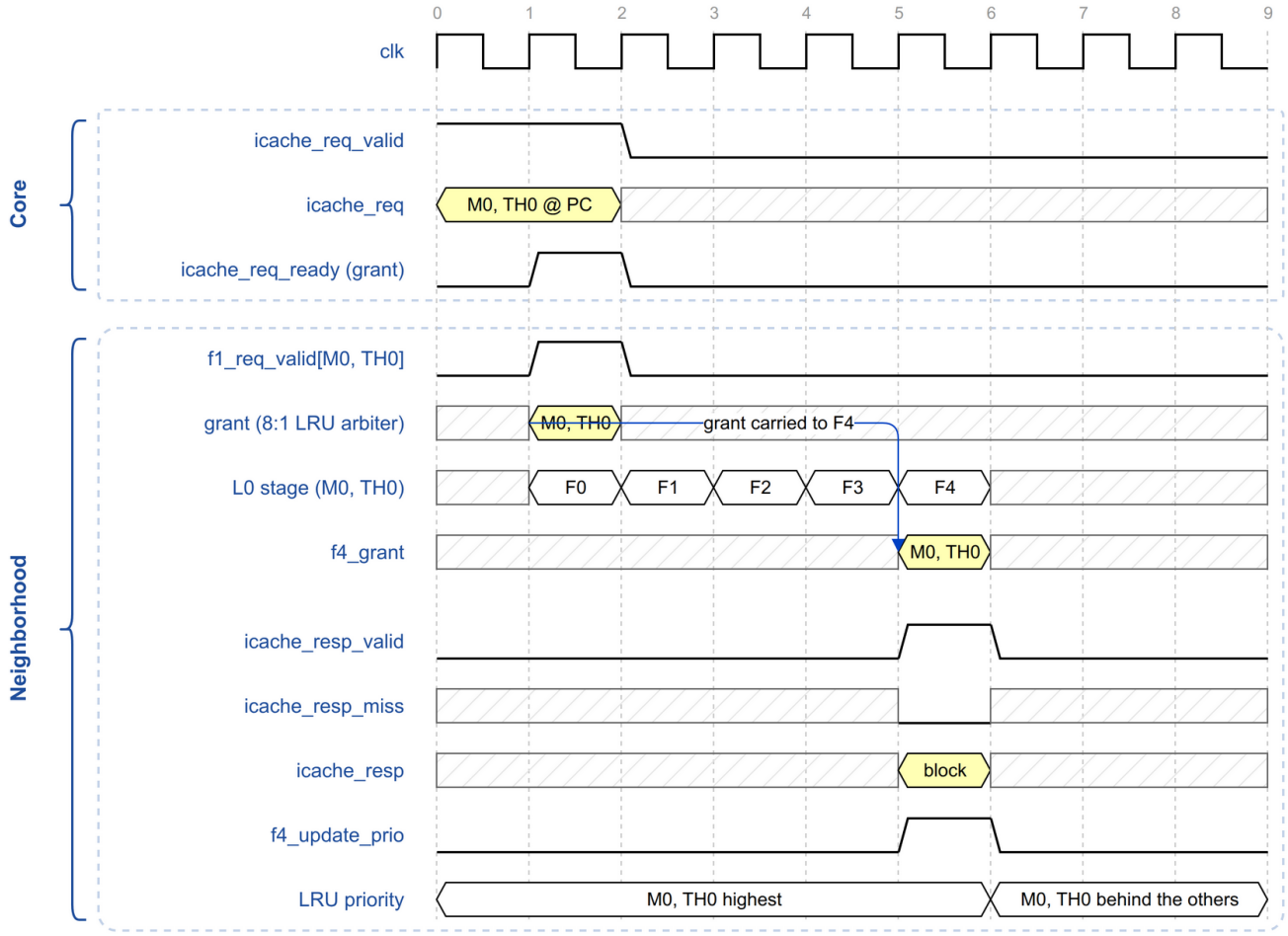


Figure 3. Request and hit response at the L0 micro-cache

1.3.5. Miss, Sleep and Fill-Done Protocol

A request misses in the L0 micro-cache either because the instruction TLB has no translation for it, or because the line is not in the micro-cache:

- **TLB miss.** A request is sent to the shared page-table walker (Section 1.6), and the lookup continues down the pipeline, so the requesting thread receives its miss response with the normal timing.
- **Tag miss.** A request for the line is sent to the L1 instruction cache. The two L0 micro-caches share one request port to the L1 and are selected by a round-robin arbiter, each with a holding register for its request.

In both cases the core that made the request receives a miss response and puts the requesting thread to sleep.

When the fill completes, that is, when the translation is in the TLB or the line is in the micro-cache data array, the L0 micro-cache pulses `icache_fill_done`. The pulse is common to all eight threads it serves and carries no address, so every sleeping thread wakes and retries. A thread whose line is still absent misses again and sleeps until the next fill.

Fill done is deliberately driven one stage later than the response. A lookup that reaches F2 in the cycle the fill is written hits, because the micro-tag array checks the tag being written as well as the tags it already holds. The last lookup that misses on the line is the one in F3 while the line is

written, so its miss response is returned two cycles before `f5_resp_fill_done`. Every thread has therefore entered the sleep state before its wake-up arrives, and no wake-up is lost.

Each L0 micro-cache has one miss handler, which holds one miss at a time; it is described in Section 1.3.8.1. Figure 4 follows one thread through a miss and its retry. The lookup misses in F2, the handler leaves `Ready`, and the request to the L1 instruction cache is driven in F3. The response in F4 is a miss, so the thread sleeps and its priority is left unchanged; the grant has already freed both the core's F1 request register and the Neighborhood's per-thread register.

The break in the waveform stands for the access time of the L1 instruction cache, and of the Shire Cache behind it if the L1 misses as well. That time is not fixed and is far longer than the rest of the sequence. The handler stays in `Fill_Wait` for all of it, so every other lookup that misses meanwhile is turned away, and the core leaves the thread asleep and issues nothing for it. When the line arrives the handler returns to `Ready` and fill done is pulsed; the thread wakes, requests again and hits, and only that second response updates the priority.

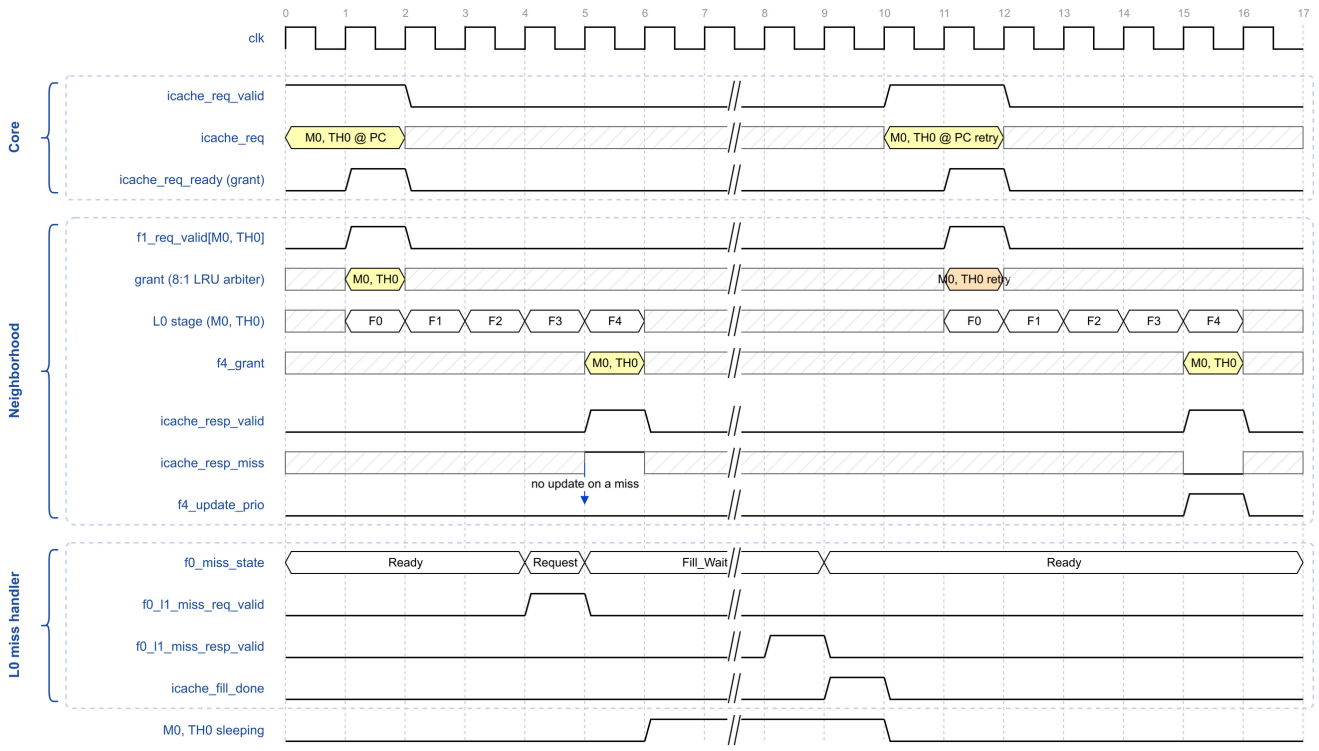


Figure 4. Miss, sleep, fill done and retry

1.3.6. Fault and Error Reporting

The response carries the fault and error status of the fetch together with the data (Table 5):

Table 5. Fetch response fault and error fields

Field	Meaning
`page_fault`	The instruction TLB could not translate the address, or the translation forbids the access. Reported whether the lookup hit or missed.
`access_fault`	The PMA check rejected the physical address. Reported whether the lookup hit or missed.

Field	Meaning
`cacheable`	The block comes from a cacheable region, as reported by the PMA check. Valid on a hit.
`ecc_err`	The line was filled with an ECC error. The micro-cache has no ECC of its own: it stores the error flag that arrived with the fill next to the line and returns it on every hit on that line, until the line is replaced. Only the error classes enabled in the error-log control ESR set the flag. Valid on a hit.
`bus_err`	The line was returned with an error by the memory hierarchy. Valid on a hit.

A response that reports a fault is delivered like any other response, and the core marks the faulting instruction so that the integer pipeline raises the exception ([sec-ch2-fe-status]). Errors detected in the L1 instruction cache data RAM are also logged in the Neighborhood ESRs, which hold a sticky single-bit and double-bit error flag and, for the access that was logged, its way, set and physical address and the ECC blocks of the line in which the error was found.

1.3.7. Complete Path for Four Cores

Figure 5 puts the pieces together: four cores requesting in the same cycle, the two flops each request passes through, the arbitration between the threads, and the five responses, one for each of the five threads that were granted. In the `LRU order` lane alone the threads are written as digits, numbered 2 × Minion + thread, because eight names do not fit in one cycle; the lane lists them highest priority first.

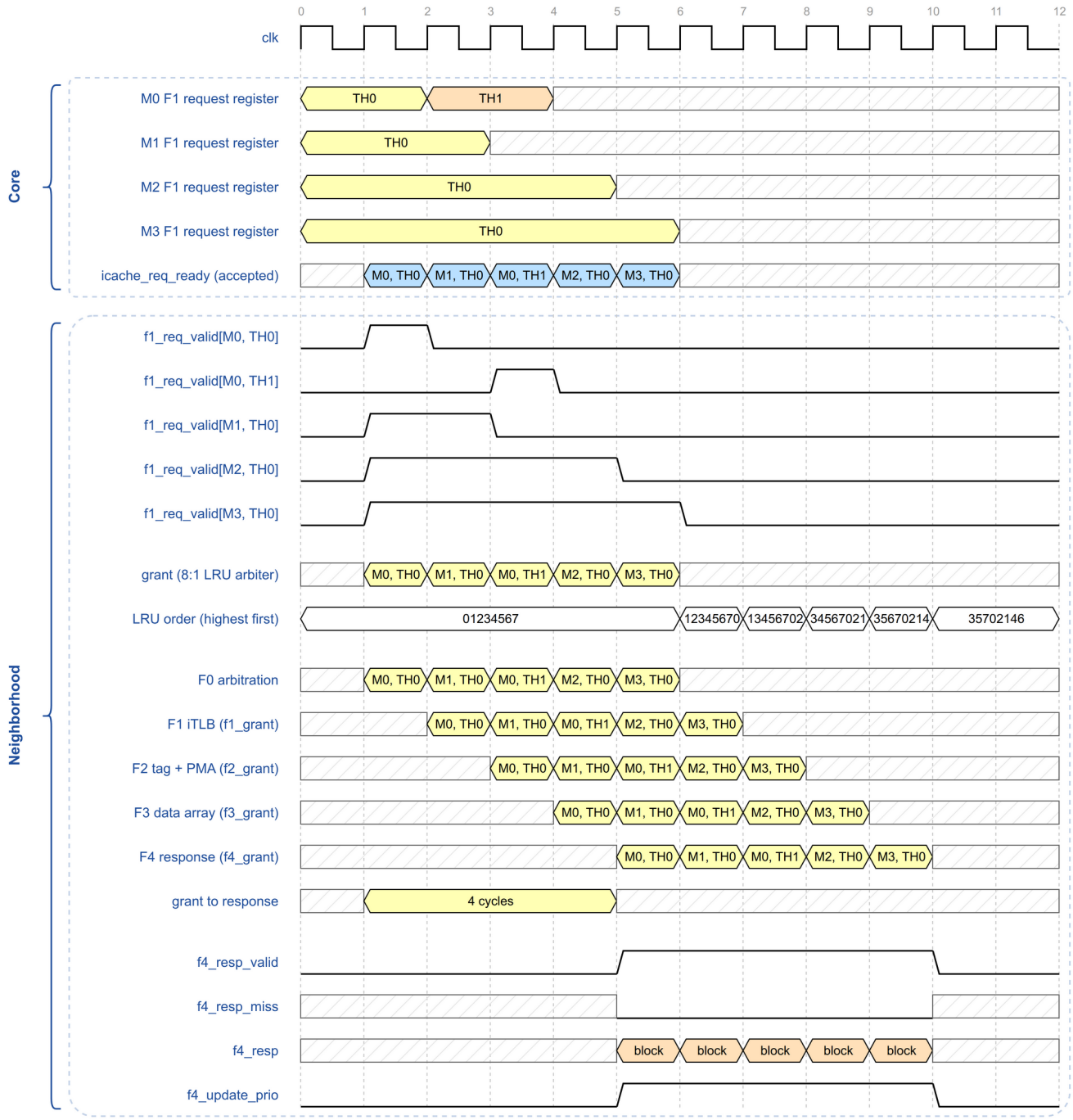


Figure 5. Request capture, arbitration and response for four cores

In cycle 0 the four cores present a TH0 request each, and the F1 registers hold them. The thread registers go valid in cycle 1, M0, TH0 wins and `icache_req_ready` frees M0's F1 register, which reloads with TH1 in cycle 2. M1, TH0 is granted in cycle 2, and in cycle 3 M0, TH1 is granted, while M2 and M3 are still holding their first request.

M0 is therefore served twice before M2 and M3 are served once, and the reason is that the aging matrix has one entry per thread, not one per Minion. Granting M0, TH0 demotes M0, TH0 and nothing else, so M0, TH1 keeps a position that is still ahead of M2 and M3 and wins the arbitration in cycle 3. The deferred update does not cause this: if the priority update occurs at the grant instead of at the response, M0, TH1 still wins in cycle 3.

Per-thread entries are what make the arbitration fair, because the two threads of a core succeed and fail independently. One may hit while the other misses, and a single entry shared by both

would demote the thread that missed every time its sibling was served, costing it the position it needs to win its retry after the fill. What the deferred update adds is the next step: a thread whose own response is a miss keeps its place, so it wins its retry immediately, and the miss-count guard bounds how long any thread can be passed over (Section 1.3.8).

The response for M0, TH0 appears in F4 in cycle 5, four cycles after its grant, with `f4_grant` naming the thread. It hits, `f4_update_prio` is asserted and the order changes from `01234567` to `12345670`.

1.3.8. Livelock and Starvation Avoidance

Up to eight threads share each L0 micro-cache, and the two threads of a core share its fetch port, decoders and issue slot. At every shared point one thread can be delayed by the others. Each shared point therefore has a policy that limits how long a thread can be held back: LRU or round-robin arbitration, a priority rule that favours threads whose requests missed, a wake-up protocol that never loses a wake-up, and a miss-count guard that gives a starving thread exclusive access to the L0 micro-cache. Table 6 lists these mechanisms, and the subsections that follow describe them.

Figure 6 adds to the interface of Figure 2 the two blocks that carry the guard: the per-thread miss counters, one set per micro-cache, and the mask they drive, which gates the bids into the 8:1 LRU arbiter.

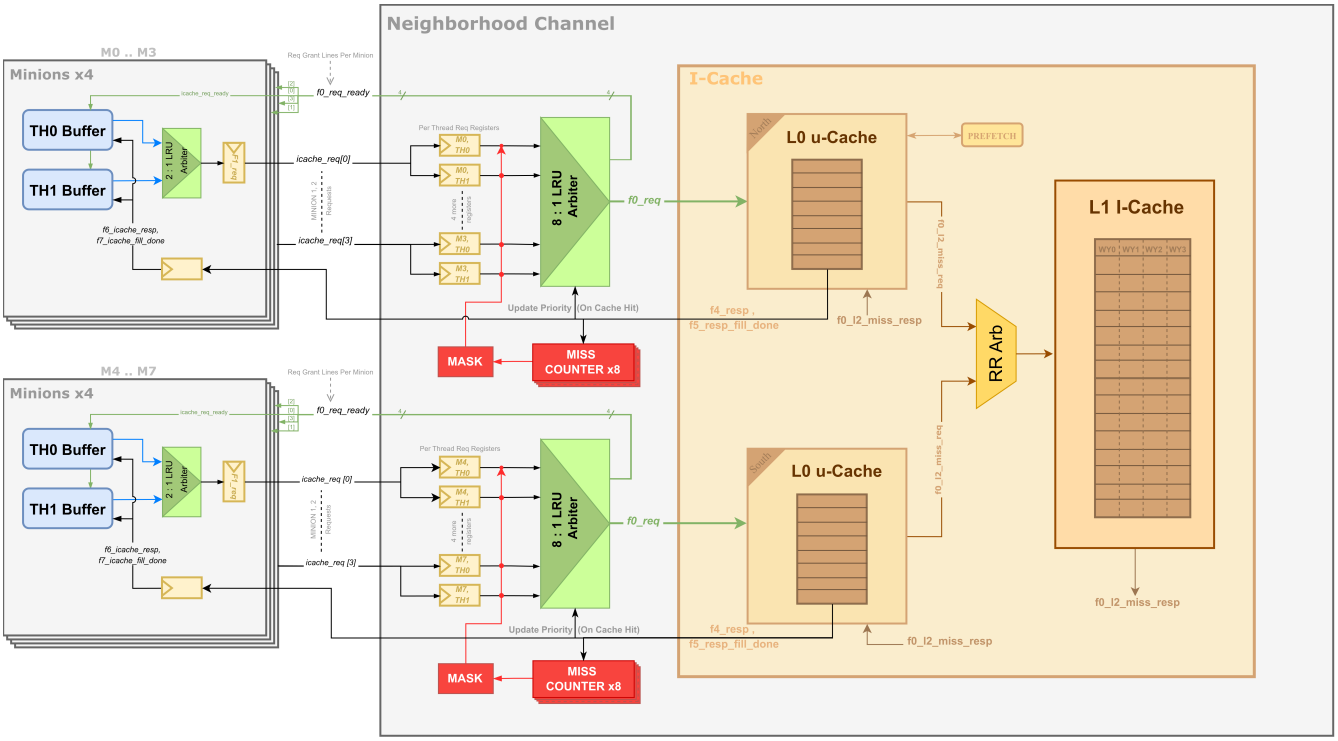


Figure 6. Miss counters and bid mask in the fetch path

Table 6. Starvation avoidance mechanisms in the instruction fetch path

Shared resource	Contenders	Mechanism	Guarantee
Core fetch port (F1 request register)	2 threads of a core	2:1 LRU arbiter, priority updated on every grant	A waiting thread is granted after at most one request of the other thread

Shared resource	Contenders	Mechanism	Guarantee
L0 request arbiter	8 threads of 4 cores	8:1 LRU arbiter whose priority is updated only on a hit or a fault	A thread whose request missed keeps its priority for its retry
L0 miss handler	8 threads and the prefetcher	Fill-done broadcast with ordered timing; thread wakes on fill done, redirect, disable or halt	A thread sleeps only while a fill is in progress, and always wakes
L0 miss handler (repeated misses)	8 threads	Per-thread consecutive-miss counters and bid mask	A thread with 31 consecutive misses gets exclusive access for its core until it hits
L0 miss handler (prefetch)	Prefetcher and threads	Prefetch requests enter the L0 micro-cache only in cycles with no core request	Demand fetches always win the L0 pipeline over prefetch
L0 miss handler (retry windows)	Prefetcher and waking threads	8-cycle prefetch pause after each fill done; prefetch blocked after an unattended miss until the next fill done, then until a core request or 16 cycles	Waking threads get the first chance at the freed miss handler, and prefetching is never blocked for good
L1 I-cache miss port	2 L0 micro-caches	Round-robin arbiter	Each L0 micro-cache waits for at most one miss of the other
Decoders and ID issue slot	2 threads of a core	Alternating decode selection and round-robin thread scheduler, per-thread FIFOs and stalls	Each eligible thread issues at least every other cycle

The mechanisms in the first two rows are described with the hardware that implements them: the 2:1 LRU arbiter of the core fetch port in [sec-ch2-fe-pipeline], and the 8:1 LRU arbiter with its deferred priority update in Section 1.3.3. The code prefetcher and the two timed windows that hold it behind every thread are described in Section 1.4.11, the round-robin arbiter between the two micro-caches in Section 1.5.4, and the decoders, instruction FIFOs and thread scheduler in [sec-ch2-fe-decode] and [sec-ch2-fe-sched]. What remains, and what the rest of this section describes, is the one shared resource with no arbitration of its own: the miss handler of the L0 micro-cache.

1.3.8.1. The L0 Miss Handler

The miss handler is described in full in Section 1.4.8. Enough of it is repeated here for this section to stand on its own.

Each L0 micro-cache has one miss handler: the logic that handles the L0 micro-cache's single outstanding request to the L1 instruction cache. It consists of the following:

1. a three-state machine (`f0_miss_state`: Ready, Request, Fill_Wait),

2. a register holding the physical address of the missing line (`f0_miss_addr`), and
3. a flag indicating the miss as a prefetch miss (`f0_miss_is_prefetch`).

A miss is accepted only in the Ready state. The handler then sends the request to the L1 instruction cache (Request state), waits for the line (Fill_Wait state), writes it into the L0 micro-cache unless it is a prefetch, and returns to the Ready state, at which point fill done is generated.

Behind it, the L1 instruction cache is also single-transaction. It accepts one request, closes its request port until that request's response, and always answers with the block: on an L1 hit from its data RAM, on an L1 miss once the line has come back from the Shire Cache. A micro-cache miss that reaches the L1 therefore occupies the L1 for its whole duration, which on an L1 miss is a full round trip out of the Neighborhood.

The code prefetcher shares this same handler. It never delays a fetch in the same cycle, because a prefetch request enters the L0 pipeline only in cycles with no core request, but after a prefetch takes the handler, the prefetcher holds the handler exactly as a core miss does (Section 1.4.11).

A lookup that does not hit returns a miss response to the thread in one of the following three cases:

1. The line is not present in the L0 micro-cache and the handler is idle, so the handler accepts the miss and sends it to the L1 instruction cache.
2. The line is not present in the L0 micro-cache but the handler is busy with another miss, so the miss is not recorded and no request is sent to the L1 instruction cache for it.
3. The address misses in the L0 TLB, so a translation is requested from the page-table walker (Section 1.6) and no request is sent to the L1 instruction cache for it.

In all three cases the thread enters the sleep state (`f0_miss_pending`) and no request is issued. In the second and third cases nothing is requested for that thread, and it is the second that this section is about.

1.3.8.2. Sleep and Wake-Up

The L0 micro-cache pulses `f6_icache_fill_done` to all its threads whenever an L1 fill completes, whether it is a demand miss or a prefetch, and whenever a translation fill completes. On seeing the fill done every sleeping thread wakes and retries according to the arbitration priority. A thread whose line is now present hits, including any thread that missed on the line already being fetched, and a thread whose line is still absent misses again; if the handler is now idle, its miss is sent to the L1 instruction cache.

A thread sleeps because its own response was a miss; that is the only thing that puts it to sleep. What decides when it is woken is the task of the miss handler at that moment: the thread's own miss, the fill of another thread that keeps the handler busy, or a translation. In all three a fill is in progress, so every sleeping thread is woken when it receives the fill-done signal. The fill-done timing ([sec-ch2-fe-miss]) makes sure that no wake-up is lost: fill done is issued one cycle after the last response the fill can affect, and it is registered again in the Front End. A redirect, a thread disable or a pending halt clears the sleep state of the affected thread only; fill done is the only event that wakes all sleeping threads.

Which thread is served in a round is decided in F2. That is the stage where the tag is compared,

where a miss is therefore first indicated, and where the handler accepts it and flops its state so that the request to the L1 instruction cache goes out in F3 (Table 4). A lookup that reaches F2 any later in the round finds the handler no longer Ready.

Two properties of this protocol follow, and they set up the rest of the section. Because the handler accepts a miss only while it is Ready, and because it is Ready only between one fill completing and the next miss being accepted, each fill-done round serves exactly one new miss. And because the whole set of sleeping threads is woken at once, the thread that is served in a round is whichever one reaches F2 first with a miss.

1.3.8.3. How a Thread Can Be Left Behind

The thread that reaches F2 first is not always the thread that waited longest. A thread that was woken by something other than fill done is already moving while the others are still asleep, and it keeps that head start all the way down the pipeline.

The following sequence is built from ordinary events and needs no fault, no error and no unusual configuration.

Starting point. Each fetch returns a 32-byte block, half an L0 line, and a thread buffer holds two of them ([sec-ch2-fe-tb]). Thread A has the first half of line X in its buffer and executes from it. It requests the second half of X. While A executes from the first half, a fill for another thread selects X as its LRU victim and evicts it, so A's request misses. The miss handler is busy with that other fill, so A's miss is **not recorded**. A receives a miss response and sleeps.

The redirect. While A sleeps, the integer pipeline resolves a branch in the instructions A already consumed and redirects it. The redirect clears A's sleep bit at once, and A presents a request for the branch target without waiting for any fill.

The capture. A's new request is in F1 when `icache_fill_done` arrives for the other thread's line. The seven other threads wake on that pulse, but they are a cycle behind: they must be granted before they enter F0, while A is already one stage inside the pipeline. A reaches F2 first, misses on the branch target, finds the handler just returned to `Ready`, and takes it.

The others. Each of the seven woken threads reaches F2 one or more cycles later and finds the handler in `Request` or `Fill_Wait`. Every one of their misses is discarded. They receive miss responses, sleep again, and because a miss response does not update the arbitration priority, the next round begins in exactly the same order as this one.

No part of the design is at fault: the arbiter grants A on merit, and the miss handler accepts its miss on merit. The point is that a redirect let A bypass the queue of sleeping threads entirely, and that one fill-done round can serve only one miss. A thread that is redirected often enough, or a prefetch engine that is already in the pipeline when the threads wake, can take that single opportunity repeatedly.

The result is a thread that is granted in every round, answered in every round, and never once has a line fetched on its behalf. Its arbitration priority does not change, because priority is only updated on a hit or a fault, so the arbiter cannot lift it out of the position it is in. Raising its priority would not be enough in any case: priority decides who enters the pipeline first, and the thread that enters first is not necessarily the thread that finds the handler idle.

This is the case the miss-count guard exists for.

1.3.8.4. Miss-Count Livelock Guard

The Neighborhood counts, for each thread, the number of consecutive miss responses to that thread's own requests (`th_miss_cnt`, 5 bits, saturating at 31). The counter advances only when that thread's own response returns, so a thread that is not being granted does not accumulate.

While no counter is saturated, every thread may bid. When one or more saturate, a mask register (`th_miss_mask`) allows only the two threads of one Minion: the lowest-numbered Minion that has a saturated thread. Both of its threads are allowed, because the Minion holds one request at a time and the sibling's request may have to be accepted before the starving thread's can be presented at all. The mask is recomputed one cycle after every response.

The mask does not discard anything. A bid is the thread's request-register valid qualified with the mask, so masking only removes the thread's valid at the arbiter: the request itself stays in its register, untouched, and bids again in the cycle the mask is released. Nothing is re-issued and no state is lost.

With every other Minion masked, only the selected Minion's misses can reach the handler, so the starving thread's line is fetched and cannot be displaced before the thread retries. The thread then hits, which clears its counter, and the mask passes to the next Minion with a saturated thread or is removed if there is none. The mask is released by a hit, and only by a hit.

Figure 7 shows the guard at work, with the waits between rounds compressed into breaks. Every miss costs M0, TH0 a full round: the miss response puts it to sleep, fill done wakes it, it requests again and is granted, and its next response arrives four cycles after that grant.

The first response shown takes the counter to 31. The counter is loaded at the end of that cycle, so `th_miss_cnt_sat` only reads one in the next, and `th_miss_mask_en` is a registered copy of the counter enable, so the mask itself changes one cycle after that: the restrictive mask is in force two cycles after the response that saturated the counter. From that point only the two threads of Minion 0 bid.

Being alone does not give M0, TH0 its line, it gives it the miss handler. On the next round its lookup is the only one to reach F2, so its miss is accepted: `f0_l1_miss_req_valid` is driven in F3 and `f0_l1_miss_req` carries that thread's line address out to the L1 instruction cache. The response in F4 is still a miss and the thread sleeps again; the counter holds at 31 because it saturates. Only when that fill completes and the thread retries does it hit, and that hit clears the counter and releases the mask.

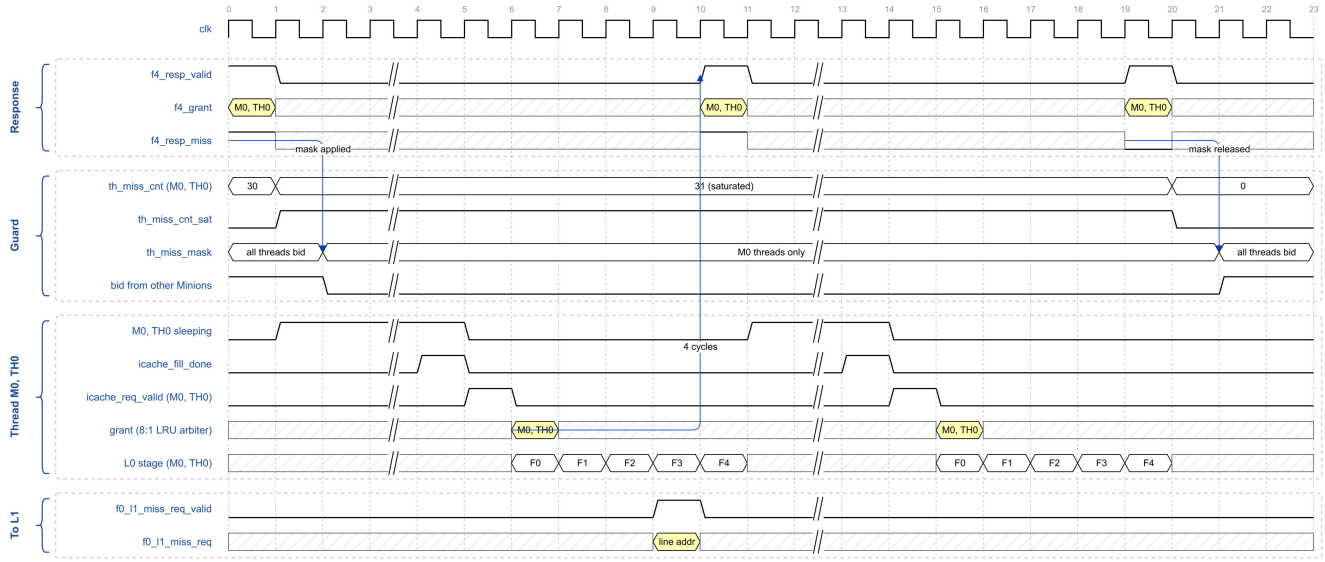


Figure 7. Miss-count guard: saturation, bid mask and release

Counters and Error Responses

A response updates the counters of more threads than the one it belongs to. Table 7 lists the cases.

Table 7. Effect of a response on the per-thread miss counters

Response returned in F4	Counter of the thread it belongs to	Counters of the other seven threads
Hit	Cleared	Unchanged
Miss, no fault or error	Incremented, saturating at 31	Unchanged
Page fault or access fault	Cleared	Cleared
Bus error or ECC error	Cleared	Cleared

The first two rows are the counter doing its job. The last two are deliberate: an error clears every thread's counter, not only the counter of the thread that received it.

The reason is a corner case in which a core would otherwise hang on a faulting address. A faulting thread never hits on that address, so its counter would saturate and it would hold exclusive access, which is only ever released by a hit. Clearing the counters and the mask lets the fault be delivered and handled normally rather than leaving a mask in place based on stale counts. If the fetch pattern that caused the pressure persists, the counters saturate again and the guard is triggered again.

The cost of clearing all eight is stated in Section 1.3.8.5: a sustained stream of faulting fetches from any thread defers the guard for all of them.

1.3.8.5. Assumptions

These mechanisms guarantee forward progress in the fetch path under the following conditions:

- Every L1 miss request eventually returns a fill, and every translation request eventually completes.

- Software does not keep a thread's exclusive mode, `WFI` or other architectural stalls asserted indefinitely. These deliberately stop the affected thread and are not starvation.
- Error responses are not sustained. Because an error clears every thread's miss counter, a continuous stream of faulting fetches from other threads would defer the livelock guard. In practice each fault redirects the faulting thread to its trap handler, so such a stream does not persist.

1.3.9. Known Limitations

The properties below follow from the design of the interface. They are not defects, but they constrain how the interface may be changed and how it behaves under load.

- **Response carries no thread identifier.** A core matches a response to its request by counting cycles, so the fixed response latency is part of the interface contract. A change to the depth of the L0 pipeline has to be matched in the Front End.
- **Fill done carries no address.** Every fill wakes every sleeping thread of the micro-cache. Threads waiting for a different line arbitrate, miss again and sleep again, which costs L0 bandwidth on every fill and grows with the number of threads waiting on different lines.
- **One outstanding miss per L0 micro-cache.** A miss that arrives while the miss handler is busy is answered with a miss response but is not recorded, so its line is not requested. That thread makes progress only after an unrelated fill wakes it and it retries.
- **Starving Minions are served in a fixed order.** When several Minions have a saturated thread, the mask always selects the lowest-numbered one first. Its waiting time is bounded, because each exclusive period ends when the selected thread hits, but the order is not fair.
- **The L1 instruction cache is blocking.** It accepts one request at a time and holds its request port closed until that request's own response, which on an L1 miss waits for the line to come back from the Shire Cache. The round-robin bound of one transaction of the other micro-cache is therefore correct as a count but stands for a full round trip out of the Neighborhood, during which the other micro-cache cannot even begin a lookup that might have hit.

1.4. L0 Micro-Cache

1.4.1. Overview

The L0 micro-cache (`icache_micro_cache`) is the first level of the Neighborhood's shared instruction cache: a fully associative, physically tagged cache of 16 lines of 64 bytes, 1 KB in all. Each Neighborhood has two instances, each shared by four Minions, eight threads: the north micro-cache serves Minions 0 to 3 and the south micro-cache serves Minions 4 to 7. The cache is split in two so that each instance is next to the Minions it serves, which reduces congestion in the Neighborhood channel. The two instances are identical except for the code prefetcher, which loads code into the L1 instruction cache at the request of software and is active only in the north micro-cache.

The micro-cache has a single request port, so the eight threads of an instance are arbitrated in the Neighborhood by an 8:1 LRU arbiter (Section 1.3.3), which presents the virtual fetch address of one request per cycle. The micro-cache translates the address with its own instruction TLB and, on a hit,

returns a 32-byte instruction block, half a line. On a miss it requests the complete 64-byte line from the L1 instruction cache, the next level, which both micro-caches share.

The design rests on four decisions:

1. **Never stalls, fixed latency:** a miss does not block the requests behind it, so the cache accepts one request in every cycle, serves hits while a miss is outstanding, and returns a response for every request, hit or miss, four cycles after the grant. The Front End depends on the fixed latency, because it recognizes its responses by timing alone ([sec-ch2-fe-track]).
2. **Translate, then look up:** the tags are physical, so the instruction TLB translates the fetch address in the cycle after the request is accepted, and the physical tag is compared with all 16 lines in the next cycle.
3. **One fill at a time:** one line request to the L1 instruction cache and one page-table walk can be outstanding. A miss that finds the resource it needs busy is answered but not recorded, and every completed fill wakes all sleeping threads (Section 1.3.5).
4. **One TLB for four Minions:** each Minion owns four of the 16 TLB entries, except in cooperative mode, a Shire-wide setting held in the `shire_coop_mode` ESR, in which the four Minions share all 16 entries.

Table 8 lists the submodules. The sections that follow describe the interface (Section 1.4.2), the organization and address decode (Section 1.4.3), the pipeline (Section 1.4.4), the instruction TLB (Section 1.4.5), the arrays and the replacement policy (Section 1.4.6), the PMA check (Section 1.4.7), miss handling (Section 1.4.8), flush (Section 1.4.9), bypass mode (Section 1.4.10), the code prefetcher (Section 1.4.11) and debug access (Section 1.4.12).

Table 8. L0 micro-cache submodules

Submodule	Function
Instruction TLB (`icache_tlb_array`, wrapping `tlb_top`)	Translates the fetch address. Requests a page-table walk on a miss.
Micro-tag array (`icache_micro_tag_array`)	16 tags with their valid bits. Compares the physical tag with all lines in parallel and generates the one-hot hit vector.
Micro-data array (`icache_micro_data_array`)	16 lines with their error flags. Read with the one-hot hit vector that the micro-tag array generates.
PMA unit (`icache_pma_unit`)	Checks the physical address and reports an access fault and whether it is cacheable.
LRU selector (`lru_sel`)	Chooses the victim among the 16 lines.
Miss handler (inline)	Holds the one outstanding line request to the L1 instruction cache.
Code prefetcher (inline)	Generates prefetch requests. Active only in the north micro-cache.
Bypass register (inline)	Holds the last fetched line while the cache is bypassed.
APB debug access (inline)	Debug reads and writes of the micro-tag and micro-data arrays.

Figure 8 shows how the submodules connect. A request from a core or from the code prefetcher passes the arbiter and the instruction TLB, and its physical address goes to the micro-tag array, the bypass register and the miss handler. The hit and miss logic forms the response to the Minions. The miss handler sends the line request to the L1 instruction cache, and supplies the victim and the tag when the line returns. The translation returns from the shared page-table walker at the top, and the line returns from the L1 instruction cache at the right. The walk state machine of the instruction TLB is drawn as a separate block, PTW FSM.

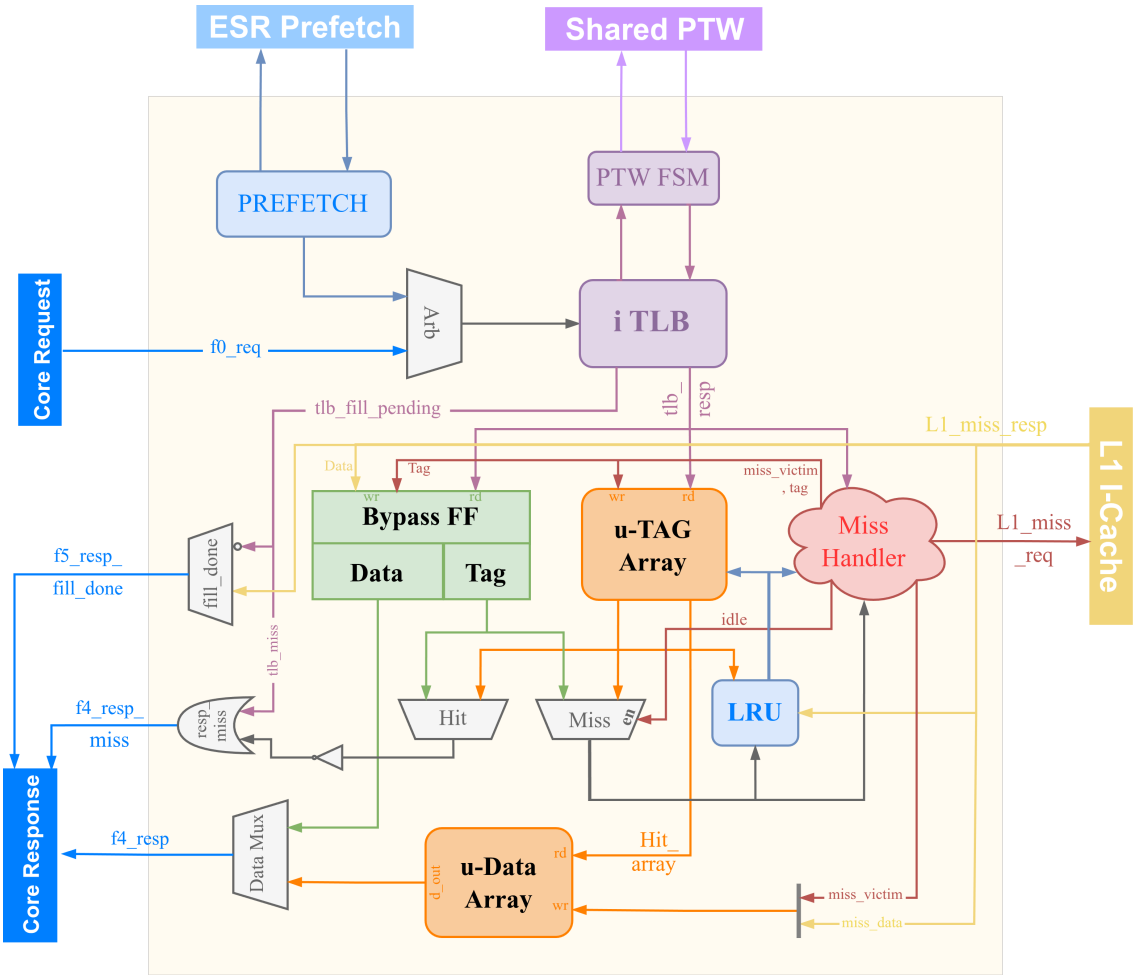


Figure 8. L0 micro-cache functional block diagram

The functional block diagram does not show the PMA unit (Section 1.4.7) or the APB debug access (Section 1.4.12). Both appear in Figure 10, the detailed pipeline diagram, which places every block in the stage where it works. The sections that follow draw each transaction on the functional block diagram, starting with a core request hit in Section 1.4.4.

1.4.2. Interface

This section defines the boundary of `icache_micro_cache`. Section 1.4.2.1 lists its ports in eleven groups: system, ESR configuration, fetch request, fetch response, flush, line request to the L1 instruction cache, line response from it, address-translation control, page-table walker, APB debug access and the Status Monitor outputs. Section 1.4.2.2 gives the field layout of the packed structures that cross these ports.

1.4.2.1. I/O Signals

Table 9 lists the ports of `icache_micro_cache`. The module is instantiated with `NR_MINIONS` set to 4, so the per-Minion ports are four wide and `f0_req_min_id` is 2 bits.

Table 9. L0 micro-cache I/O signals

Signal	Dir	Width / type	Description
System			
`clock`	in	1	Clock.
`reset`	in	1	Reset. Synchronous, active high.
ESR configuration			
`esr_prefetch_conf`	in	`icache_prefetch_conf_t`	Privilege, start line and line count of a prefetch operation. Zero in the south instance.
`esr_prefetch_start`	in	1	Starts a prefetch operation. A pulse during an operation cancels that operation and starts the new one. Tied low in the south instance.
`esr_prefetch_done`	out	1	Cleared when the prefetcher starts an operation, set when the operation completes. Resets high. The north instance drives `esr_prefetch_done` of `icache_top`.
`esr_mprot`	in	`esr_mprot_t`	Memory-protection setting for the PMA check.
`esr_vmspagesize`	in	`tlb_entry_type`	Page size of every TLB entry: 4 KB, 2 MB or 1 GB.
`esr_bypass_icache`	in	1	Bypasses both cache levels. Driven by the `bypass_icache` chicken bit. Sampled with each request in F0.
`esr_shire_coop_mode`	in	1	Cooperative mode: the four Minions share all TLB entries.
Fetch request			
`f0_req_valid`	in	1	Request granted by the 8:1 LRU arbiter of the Neighborhood.
`f0_req`	in	`frontend_icache_req`	Thread, virtual-memory status and fetch address.
`f0_req_min_id`	in	2	Minion that made the request, 0 to 3 within the instance. Selects its translation registers and, outside cooperative mode, its TLB entries.

Signal	Dir	Width / type	Description
f0_req_ready	out	1	Tied high. The pipeline never stalls.
Fetch response			
f4_resp_valid	out	1	Response to a core request, four cycles after the grant. Not driven for prefetch requests.
f4_resp_miss	out	1	Line or TLB miss. The response carries no instruction data.
f4_resp	out	icache_frontend_resp	32-byte block and status.
f5_resp_fill_done	out	1	A line or translation fill is complete. For a line, pulsed three cycles after f0_l1_miss_resp_valid.
Flush			
f0_flush_data	in	1	Invalidates all 16 lines. It also flushes the L1 instruction cache, and it is the OR of the flush requests of the eight Minions, registered once in neigh_shared_icache.
Line request to the L1 instruction cache			
f0_l1_miss_req_valid	out	1	The miss handler requests a line. Driven from F3 of the missing request until accepted.
f0_l1_miss_req_addr	out	40	Physical address of the missing line.
f0_l1_miss_req_ready	in	1	The holding register of this instance in icache_top is free.
Line response from the L1 instruction cache			
f0_l1_miss_resp_early_valid	in	1	Asserted one cycle before f0_l1_miss_resp_valid. Early write enable of the latch-based tag and data arrays.
f0_l1_miss_resp_valid	in	1	The line is returned and written into the victim entry in this cycle. Steered to the instance that made the request.
f0_l1_miss_resp_data	in	512	Line data. Driven to both micro-cache instances.
f0_l1_miss_resp_ecc_err	in	1	ECC error in the L1 data RAM, of a class enabled in the error-log control ESR. Stored with the line.
f0_l1_miss_resp_l2_err	in	1	Error response from the Shire Cache for this line. Stored with the line.

Signal	Dir	Width / type	Description
Address-translation control			
satp_info	in	4 x minion_satp_info	Supervisor address-translation register of each of the four Minions of this instance.
matp_info	in	4 x minion_satp_info	Machine address-translation register of each of the four Minions of this instance, used for fetches in M-mode.
tlb_invalidate	in	4	Invalidates the TLB entries of one of the four Minions of this instance, or the whole TLB in cooperative mode.
Page-table walker			
ptw_req_valid	out	1	Translation request. Held until accepted.
ptw_req_data	out	minion_ptw_req	Translation mode, root page number, privilege and virtual page number.
ptw_req_ready	in	1	The walker accepts the request.
ptw_invalidate	out	1	Any tlb_invalidate, registered once and forwarded to the walker.
ptw_resp_valid	in	1	The walk is complete.
ptw_resp_data	in	minion_ptw_pte	Leaf page-table entry, access fault or cancellation.
APB debug access			
apb_paddr	in	8	Address in the debug space of the micro-cache: micro-tag array, or micro-data array and error flags.
apb_psel, apb_penable, apb_pwrite	in	1 each	APB control. apb_psel and apb_penable are decoded for this instance by icache_top.
apb_pwdata	in	64	Write data.
apb_pready	out	1	Transfer complete.
apb_prdata	out	64	Read data.
apb_pslverr	out	1	Tied low.
Status Monitor			
dbg_sm_signals	out	icache_dbg_sm_t	Prefetch and miss state-machine states, F2 valid, F2 hit vector and F2 line address, for the Status Monitor debug logic. icache_top merges both instances.

1.4.2.2. Data Structures

The fetch request and response, `frontend_icache_req` and `icache_frontend_resp`, are the structures of the Front End interface and are given in [tab-ch2-fe-icache-req] and [tab-ch2-fe-icache-resp]. The structures below are used only by the instruction cache and its page-table walker.

Table 10. `icache_prefetch_conf_t`: prefetch operation (50 bits)

Field	Width	Description
`prv`	2	Privilege level used to translate the prefetch addresses.
`start_addr`	42	Virtual address of the first line, bits 47:6.
`num_lines`	6	Number of lines minus one: 0 prefetches one line, 63 prefetches 64.

Table 11. `minion_satp_info`: address-translation register (32 bits)

Field	Width	Description
`mode`	4	Translation mode: Bare (0), Sv39 (8) or Sv48 (9).
`ppn`	28	Physical page number of the root page table.

Table 12. `minion_ptw_req`: page-table walk request (70 bits)

Field	Width	Description
`satp_mode`	4	Translation mode of the register used for this fetch, `satp_info` or, in M-mode, `matp_info`.
`satp_ppn`	28	Root page-table page number from the same register.
`prv`	2	Privilege level of the fetch.
`addr`	36	Virtual page number, virtual address bits 47:12.

Table 13. `minion_ptw_pte`: page-table walk response (64 bits)

Field	Width	Description
`canceled_req`	1	The walk was cancelled. The TLB is not written.
`access_fault`	1	The walk ended in an access fault. Recorded in the TLB entry and reported as an access fault when the entry hits.
`rsvd_for_hw`	8	Reserved for hardware. Bit 0 returns the execution mode of the walk, 1 for M-mode (`matp_info`) and 0 for S-mode or U-mode (`satp_info`); the TLB stores it, and an entry hits only for a fetch in the same mode. Bits 7:1 are not used by the TLB.
`ppn`	44	Physical page number. The TLB keeps the low 28 bits.
`rsvd_for_sw`	2	Not used by the TLB.
`d`, `a`, `g`, `u`, `x`, `w`, `r`, `v`	1 each	Page-table entry flags. The TLB keeps `d`, `a`, `u` and the `r`, `w`, `x` permissions; a fetch needs `x` and `a`.

Table 14. `esr_mprot_t`: memory-protection setting (9 bits)

Field	Width	Description
mprot_en	1	Enables the M-mode and S-mode protection boxes checked by the PMA unit (Section 1.4.7).
mmode_size	4	Size of the M-mode box, encoded as a power of two.
smode_size	4	Upper bound of the S-mode box, encoded as a power of two.

tlb_entry_type is a 2-bit page size: 0 for 4 KB, 2 for 2 MB and 3 for 1 GB; 1 is reserved.

1.4.3. Cache Organization and Address Decode

This section gives the organization of the micro-cache, the parameters that size it, the storage they result in, and how a fetch address is divided on its way through the TLB and the tag lookup.

Table 15 summarizes the organization of one micro-cache.

Table 15. L0 micro-cache organization

Property	Value
Capacity	1 KB: 16 lines of 64 bytes
Associativity	Fully associative. The lookup has no index; the tag is compared with all 16 lines.
Tag	Physical address bits 39:6, 34 bits, and one valid bit for each line (Section 1.4.6)
Access size	32-byte block, half a line, selected by physical address bit 5
Replacement	LRU over all 16 lines (Section 1.4.6.3)
Allocation	A line is allocated only for a core miss accepted by the miss handler. Prefetch and bypassed fills are not allocated.
Write policy	Read-only. Lines are written only by fills and by APB debug writes, and nothing is written back.
Coherence	None. The micro-cache does not snoop. A change to code in memory is seen only after a flush (Section 1.4.9).

1.4.3.1. Configuration Parameters

Table 16. L0 micro-cache configuration parameters

Define	Value	Description
ICACHE_NR_REQS	2	Micro-cache instances per Neighborhood.
NR_MINIONS (module parameter)	4	Minions served by one instance, eight threads.
ICACHE_MICRO_CACHE_SIZE	16	Lines.
ICACHE_MICRO_CACHE_ADWIDTH	4	Bits that number a line. log2(ICACHE_MICRO_CACHE_SIZE) = log2(16) = 4
ICACHE_BLOCK_BITS	512	Line size, 64 bytes.

Define	Value	Description
ICACHE_BLOCK_ADDR_SIZE	6	Byte-offset bits in a line. log2(ICACHE_BLOCK_BITS / 8) = log2(64) = 6
FE_FETCH_READ_SIZE	256	Response block, 32 bytes.
ICACHE_CHUNK_ADDR_SIZE	1	Bits that select the block within a line. log2(ICACHE_BLOCK_BITS / FE_FETCH_READ_SIZE) = log2(512 / 256) = 1
ICACHE_MICRO_TAG_SIZE	34	Tag bits. PA_SIZE - ICACHE_BLOCK_ADDR_SIZE = 40 - 6 = 34
ICACHE_TLB_ENTRIES	16	Instruction TLB entries.
ICACHE_PREFETCH_MAX_LINES	64	Maximum lines in one prefetch operation.
VA_SIZE	48	Virtual-address bits.
VA_SIZE_EXT	49	Fetch-address bits: the 48-bit virtual address and the extension bit, bit 48. VA_SIZE + 1 = 49. The TLB detects a non-canonical address from VA[48] XOR VA[47].
PA_SIZE	40	Physical-address bits.
VA_UNTRANS_SIZE, PA_UNTRANS_SIZE	12	Page-offset bits, not translated.
PTW_PG_IDX_SZ	9	Virtual-page-number bits per page-table level. Places the TLB index and tag for 2 MB and 1 GB pages.

1.4.3.2. Storage

Table 17 lists the storage of one micro-cache. The tag array, the data array and the TLB are latch-based register files (rf_latch_1r_1w_par, rf_latch_1r_1w_dec and rf_latch_1r_1w); the tag array reads all its entries in parallel for the fully associative compare, and the data array is read with a one-hot entry select. The valid bits are flip-flops beside the arrays.

Table 17. L0 micro-cache storage

Structure	Organization	Bits
Micro-tag array	16 entries of a 34-bit tag, and 16 valid bits	560
Micro-data array	16 entries of 512 data bits, an ECC-error flag and a bus-error flag	8,224
Instruction TLB	16 entries of 70 bits: a 34-bit tag and 36 bits of entry data, which are the 28-bit physical page number, the x, w, r, u, a and d bits, the access-fault flag and the execution mode (Section 1.4.5.1); and 16 valid bits	1,136
Bypass register	34-bit tag, 512 data bits, two error flags and a valid bit	549

1.4.3.3. Address Decode

The micro-cache receives a 49-bit fetch address, the 48-bit virtual address and its extension bit, and generates a 40-bit physical address (Figure 9). With 4 KB pages the low 12 bits, the page offset, pass through unchanged. The virtual page number, bits 47:12, goes to the instruction TLB with the extension bit, and the TLB returns a 28-bit physical page number (Section 1.4.5). The micro-cache divides the physical address into three fields: bits 39:6 are the 34-bit tag, compared with the tags of all 16 lines; bit 5 selects the 32-byte half of the line that is returned; and bits 4:0 are not used by the micro-cache, because the Front End extracts the instruction from the block ([sec-ch2-fe-extract]).

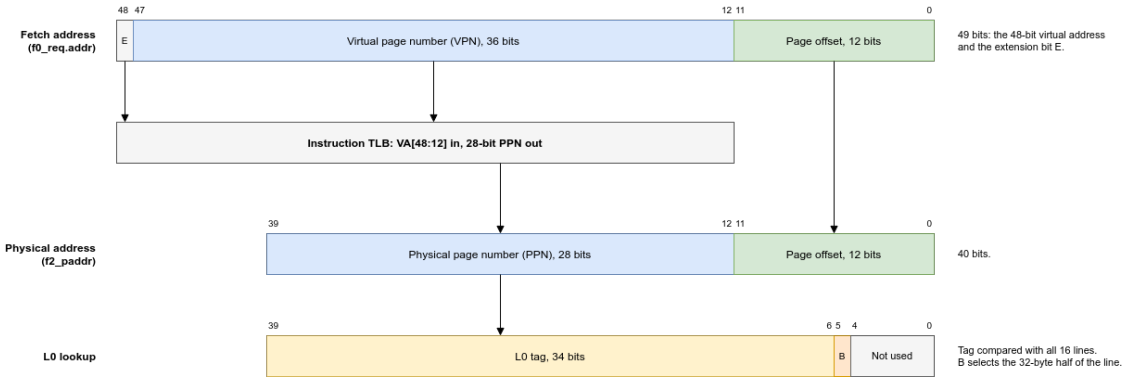


Figure 9. Address decode from fetch address to L0 lookup

1.4.4. Pipeline Stages

This section describes the five L0 micro-cache stages, F0 to F4, and the fill-done stage F5. A lookup runs from F0 to F4: a request enters in F0, is translated by the instruction TLB in F1, is compared with the tags and checked by the PMA unit in F2, reads the data array in F3, and returns its response in F4. A miss accepted in F2 sends its line request to the L1 instruction cache in F3. The fill runs independently of the lookup, and fill done is driven in F5, three cycles after the fill finishes.

Figure 10 shows the north micro-cache, the instance that contains the prefetcher, with each block drawn in the stage in which it operates. Blocks outside the frame are its neighbors: the cores, the per-thread request registers and the 8:1 LRU arbiter of the Neighborhood, the page-table walker, the L1 instruction cache, and the ESR and APB interfaces.

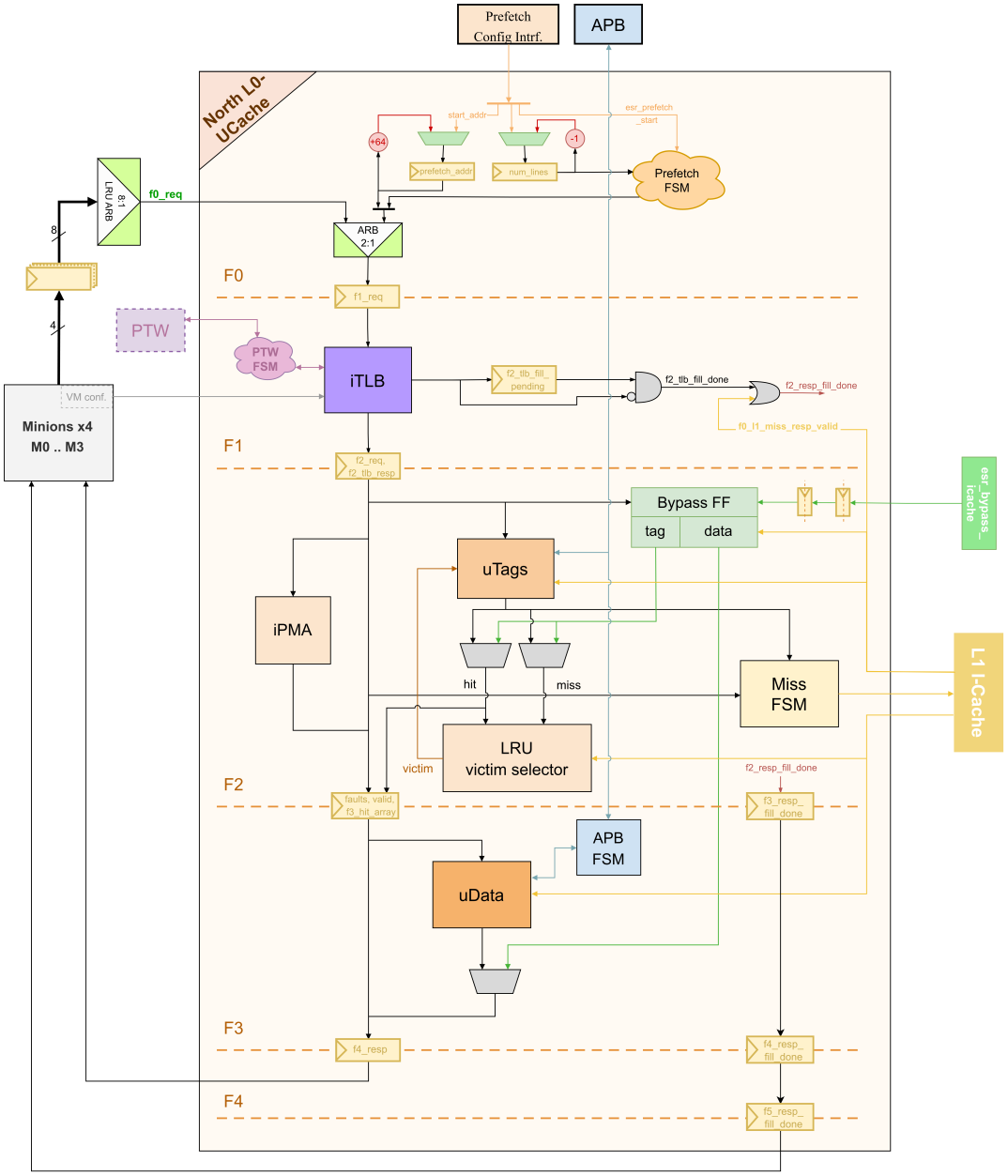


Figure 10. L0 micro-cache detailed pipeline diagram

F0 Stage

The F0 stage arbitrates between the core request granted by the 8:1 LRU arbiter and a request from the code prefetcher, and always gives priority to the core request. In a cycle with no core request, the code prefetcher can enter its request when the request is qualified (unattended-miss state machine). A prefetch request uses the same path through the pipeline as a core request, so a prefetch flag goes with each request to the next stages to tell the two apart. The request, the identifier of its Minion, the bypass bit and the prefetch flag are registered into F1.

F1 Stage

The F1 stage translates the virtual address of the request to a physical address in the instruction TLB. The translation uses the supervisor and machine address-translation registers that each Minion sends to the micro-cache (`satp_info` and `matp_info`), and the privilege level of the fetch selects one of the two (Section 1.4.5.2). The TLB returns the physical page number, a miss indication, and page and access faults. A TLB miss starts a page-table walk if no walk is in progress and the address is valid (canonical), and the walk request goes to the page-table walker one cycle later (Section 1.4.5.3). The request continues to F4 and receives a miss response, and

the thread retries after fill done.

F2 Stage

The F2 stage forms the physical address from the page number and the page offset. The tag is compared with all 16 lines of the micro-tag array, and with the line that a fill writes in the same cycle, while the PMA unit checks the address. For a core request outside bypass mode, the LRU state is updated on a hit, on an accepted miss, and when the L1 response writes the line into the victim entry. A line miss with no TLB miss and no fault is accepted if the miss handler is Ready: the handler records the physical address, the LRU victim and the bypass and prefetch flags. For a core miss outside bypass mode, the victim's valid bit is also cleared. A miss that finds the handler busy is not recorded and receives only the miss response in F4. In bypass mode the tag is compared with the bypass register instead of the micro-tag array, and a prefetch request is forced to miss.

F3 Stage

The F3 stage reads the micro-data array with the one-hot hit vector that the micro-tag array generates in F2, and selects the 32-byte half-line named by address bit 5; in bypass mode the half comes from the bypass register. The response is a miss when the tag misses or the TLB misses. The ECC and bus-error flags stored with the line are valid only on a hit. If F2 accepted a miss, the miss handler is in its Request state in this cycle and drives the line request to the L1 instruction cache.

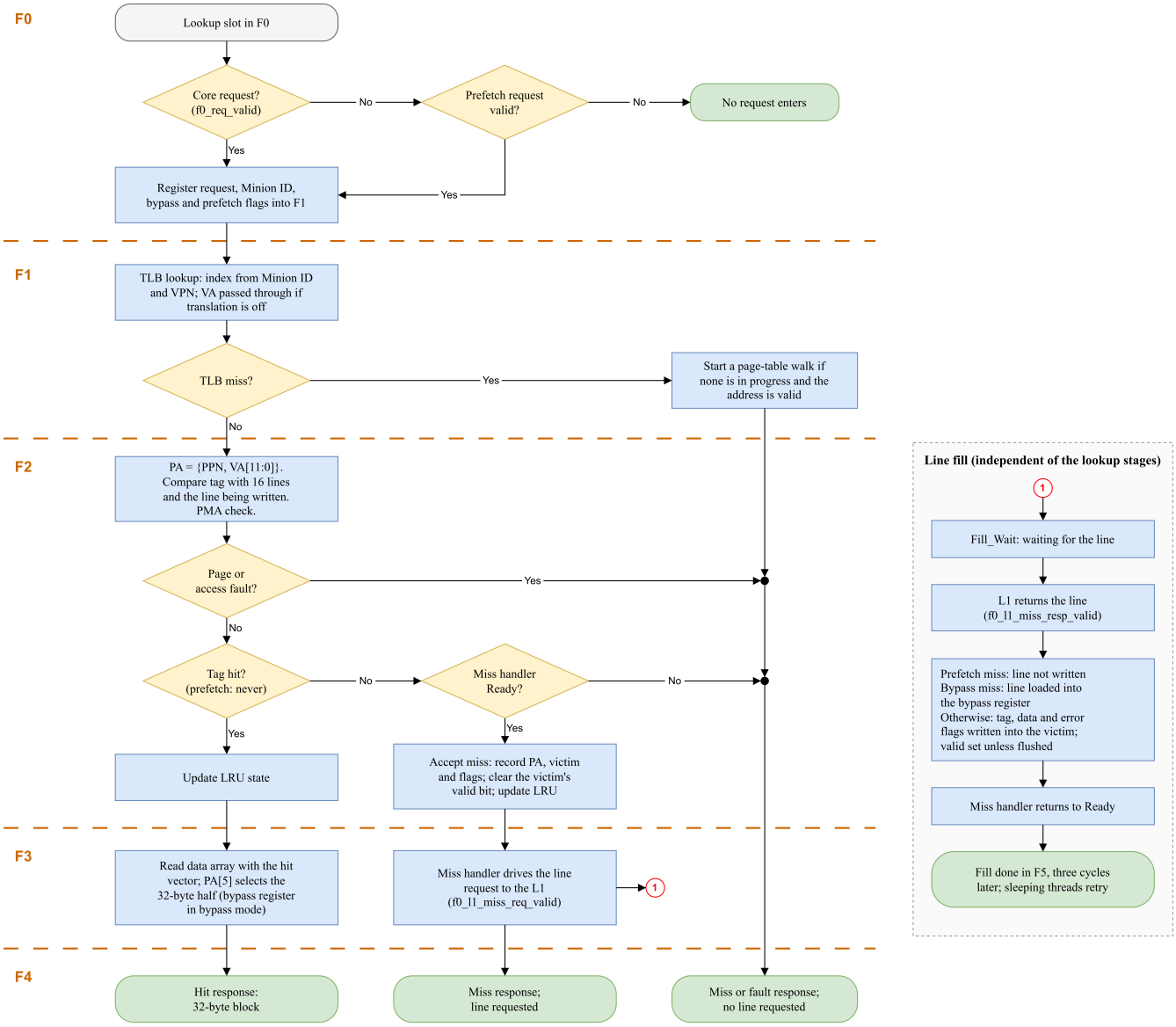
F4 Stage

The F4 stage registers the response and drives it to the four Minions, four cycles after the grant, for a hit and for a miss alike. A prefetch request produces no response.

F5 Stage

The F5 stage drives fill done, which tells the sleeping threads that a line or translation fill is complete. Fill done is generated in F2 and passes through F3 and F4 to F5. F5 is not part of a lookup.

Figure 11 shows the path of one request from F0 to F4 and the fill that a miss starts. Every request leaves the pipeline at F4 with a hit, miss or fault response. The fill path on the right runs independently of the lookup and ends with fill done in F5. The time to fill a line depends on the L1 instruction cache and is given in Section 1.5.



Prefetch requests take the miss paths and produce no response. A thread that receives a miss response sleeps until fill done.

Figure 11. L0 micro-cache pipeline flow

The transaction figures of the L0 micro-cache draw one transaction on the functional block diagram (Figure 8):

- **Path:** the dashed green line is the path of the transaction.
- **Steps:** a numbered green circle marks each step, in order.
- **Unused blocks:** a red circle or a dashed red line marks a block that the transaction does not use, or whose result it ignores.
- **Cycles:** two steps with different numbers can occur in the same cycle. The table under each figure gives the cycle of each step: a request counts its cycles from the grant, with the stage in brackets, and a fill counts them from the cycle *N* of its response.

Figure 12 shows a core request that hits in the instruction TLB and in the micro-tag array, the common case, and Table 18 lists its steps.

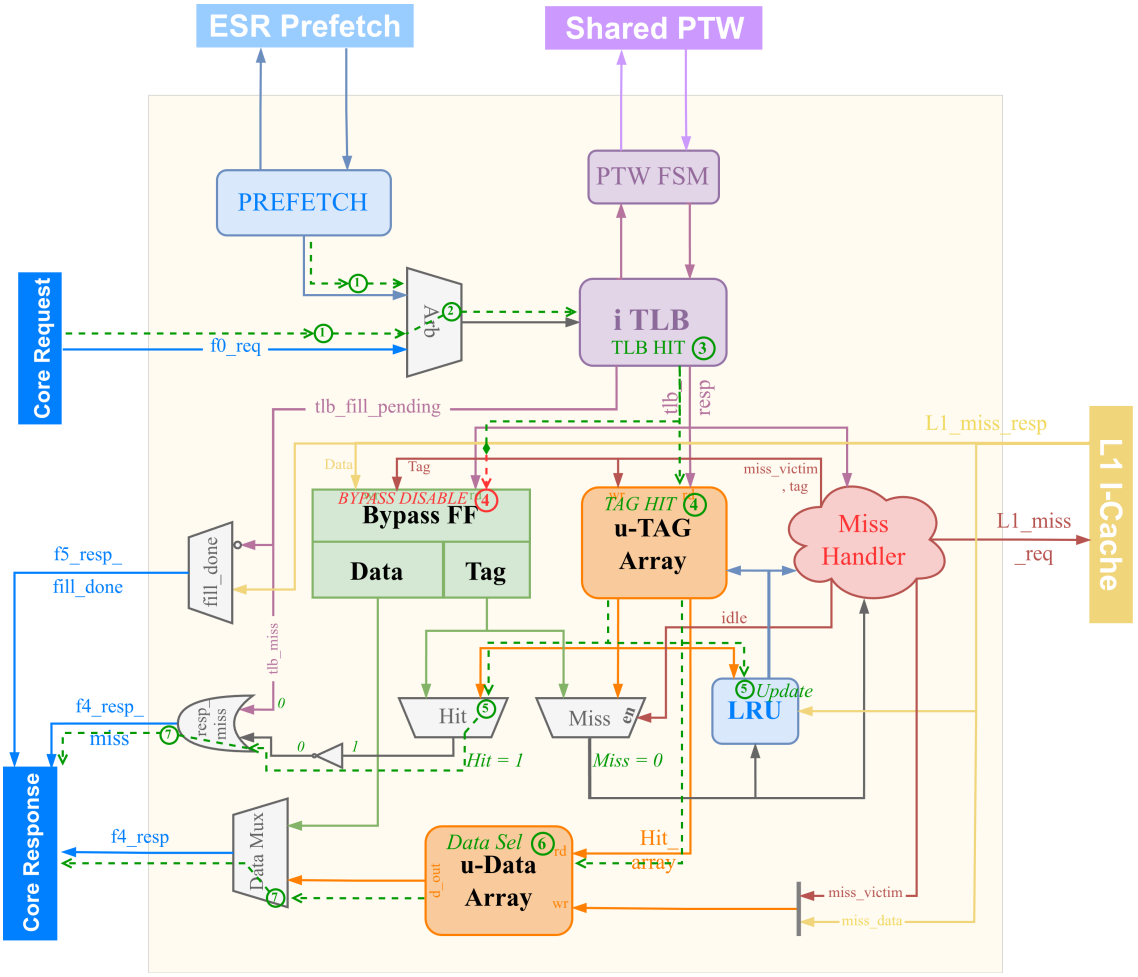


Figure 12. Core request hit

Table 18. Core request hit, steps by cycle

Cycle	Step	Block	Action
0 (F0)	1	Core request, code prefetcher	A core request and, if one is qualified, a prefetch request reach the arbiter.
0 (F0)	2	Arbiter	Grants the core request, which always has priority.
1 (F1)	3	Instruction TLB	Hits and returns the physical page number.
2 (F2)	4	Micro-tag array	The physical tag hits one line. The bypass register is not used outside bypass mode.
2 (F2)	5	Hit and miss logic, LRU	Hit is 1 and miss is 0. The LRU state records the line that hits.
3 (F3)	6	Micro-data array	Reads the line with the one-hot hit vector and selects the 32-byte half.
4 (F4)	7	Response	Drives the instruction block to the Minions, with the miss bit clear.

1.4.5. Instruction TLB

The tags of the micro-cache are physical, so every request, from a core or from the code prefetcher, is translated from its virtual address to a physical address in F1 by the instruction TLB. The TLB is

icache_tlb_array, a wrapper around tlb_top, the TLB design that the data cache of each Minion also uses (dcache_tlb_array); the two caches share the design but not the translations. The instruction wrapper sets 16 entries for four Minions, marks every request as an instruction fetch, and registers satp_info, matp_info and esr_vmspagesize; tlb_top registers esr_shire_coop_mode. A change to any of these settings takes effect one cycle later.

A TLB miss needs a page-table walk, and the TLB has at most one walk outstanding (Section 1.4.5.3). The TLB does not block during a walk: it continues to look up every request, and a request that hits receives its translation. A request that misses during a walk starts no walk and receives a miss response with no data, and its thread retries after fill done.

Table 19 lists the interface of icache_tlb_array, grouped by the block at the other end. The request and the response connect to F1 of the micro-cache pipeline: the TLB answers in the same cycle, and the micro-cache registers the response into F2. The configuration inputs, the invalidation pulses and the walker ports pass through to ports of the micro-cache (Table 9).

Table 19. Instruction TLB interface signals

Signal	Dir	Width / type	Description
Request from the pipeline, F1			
req_valid	in	1	A request is in F1, from a core or from the code prefetcher.
req_min_id	in	2	Minion of the request. Selects its address-translation registers and, in per-Minion mode, its four TLB entries. A prefetch request uses Minion 0.
req_data.status	in	minion_vm_status	Privilege level, mprv, mpp, sum, mxr and debug mode of the requesting thread.
req_data.vpn	in	36	Virtual page number, VA[47:12].
req_data.msb_err	in	1	VA[48] differs from VA[47]. Used for the canonical-address check.
req_data.passthrough	in	1	Disables translation. Tied low in the micro-cache.
Response to the pipeline, F1			
resp_data.ppn	out	28	Physical page number. VA[39:12] when translation is disabled.
resp_data.miss	out	1	TLB miss.
resp_data.xcpt_if	out	1	Page fault (Table 21).
resp_data.access_fault	out	1	Access fault from the walk that filled the entry.
resp_data.fill_pending	out	1	A walk is in progress. Its fall marks the end of a translation fill.
Configuration and invalidation			

Signal	Dir	Width / type	Description
satp_info	in	4 × minion_satp_info	Supervisor address-translation register of each of the four Minions (Table 11). Registered in the wrapper.
matp_info	in	4 × minion_satp_info	Machine address-translation register of each of the four Minions, used for fetches in M-mode. Registered in the wrapper.
esr_vmspagesize	in	tlb_entry_type	Page size of every entry: 4 KB, 2 MB or 1 GB. Registered in the wrapper.
esr_shire_coop_mode	in	1	Cooperative mode: the four Minions share all 16 entries. Registered in tlb_top.
tlb_invalidate	in	4	One pulse per Minion. Clears the entries of that Minion, or all 16 entries in cooperative mode (Section 1.4.5.5).
Page-table walker			
ptw_req_valid	out	1	Walk request. Held until the walker accepts it.
ptw_req_ready	in	1	The walker accepts the request.
ptw_req_data	out	minion_ptw_req	Translation mode, root page number, privilege level and virtual page number (Table 12).
ptw_invalidate	out	1	OR of the four tlb_invalidate pulses, registered once.
ptw_resp_valid	in	1	The walk is complete.
ptw_resp_data	in	minion_ptw_pte	Last page-table entry of the walk, with the access-fault and cancel flags (Table 13).

1.4.5.1. Organization

The TLB has 16 entries, and all of them hold translations of one page size, 4 KB, 2 MB or 1 GB, set by esr_vmspagesize. Figure 13 shows the format of an entry and how the 16 entries are divided between the Minions. An entry holds 70 bits in a latch-based register file: the 34-bit tag, the 28-bit physical page number, the x, w, r, u, a and d bits of the page-table entry, the access-fault flag of the walk, and the execution mode of the walk. The valid bits are separate flip-flops. The tag field is 34 bits because that is the longest tag, the one of 4 KB pages in per-Minion mode; cooperative mode and larger pages use fewer bits and store them with zero bits (Table 20).

A walk can end without a translation. The walker returns the last page-table entry that it reads, and that entry is not a valid leaf when v is clear, when r and x are both clear, or when w is set and r is clear, a reserved encoding. A walk that ends in an access fault gives no translation either. The TLB still writes the entry, with x, w and r as 0. The thread's retry then hits that entry and takes a page fault, and also an access fault if the walk ended in one, instead of starting the same walk again.

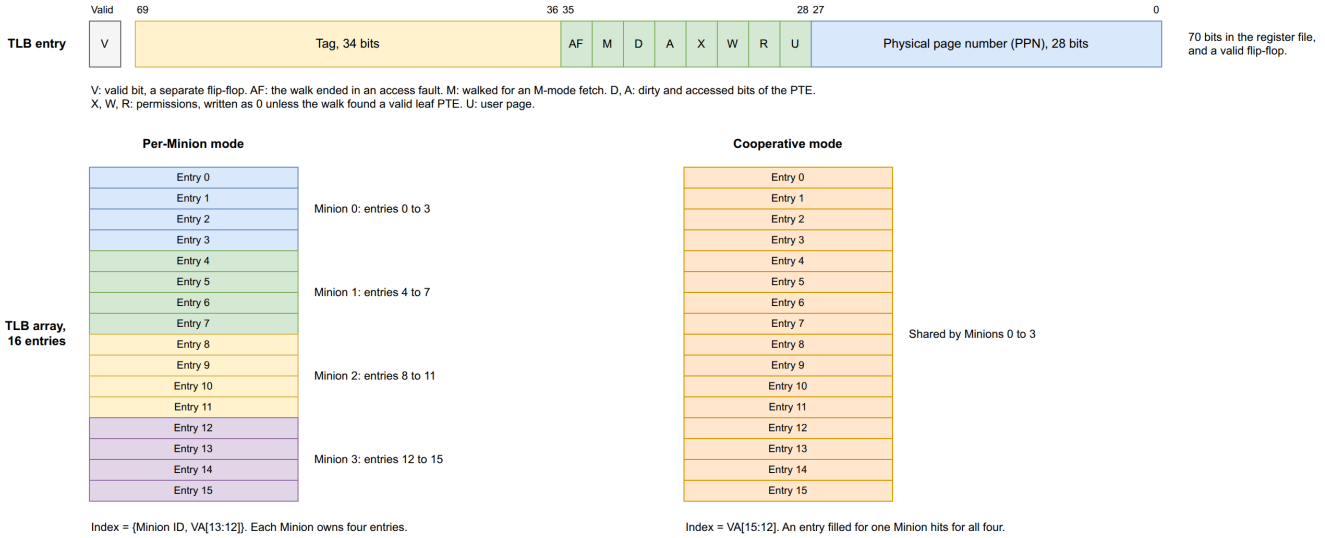


Figure 13. Instruction TLB organization

The TLB is direct-mapped in both of its modes: the index selects one entry, and a translation is either in that entry or not in the TLB. The mode, set by `esr_shire_coop_mode`, decides where the index comes from:

- **Per-Minion mode:** the micro-cache serves four Minions, so the 16 entries are divided into four groups of four, one group for each Minion. The upper two index bits are the Minion identifier, and the lower two come from the virtual page number. A Minion replaces only its own entries, so a Minion that touches many pages cannot evict the translations of the other three.
- **Cooperative mode:** all 16 entries are shared, and all four index bits come from the virtual page number. An entry carries no Minion identifier, so an entry filled for one Minion hits for the other three. Cooperative mode therefore assumes that the four Minions use the same translation, and in return the group has 16 entries instead of four for each Minion.

Figure 14 shows how the index and the tag are taken from the virtual page number for 4 KB pages, and Table 20 lists them for all page sizes. Each stored tag is written most-significant field first. The tag field is always 34 bits wide, and a shorter tag fills the rest of the field with zero bits. Cooperative mode adds two zero bits at the top, because its index takes two more address bits; a 2 MB or 1 GB page adds 9 or 18 zero bits at the bottom, one page-table level of 9 bits for each step above 4 KB.

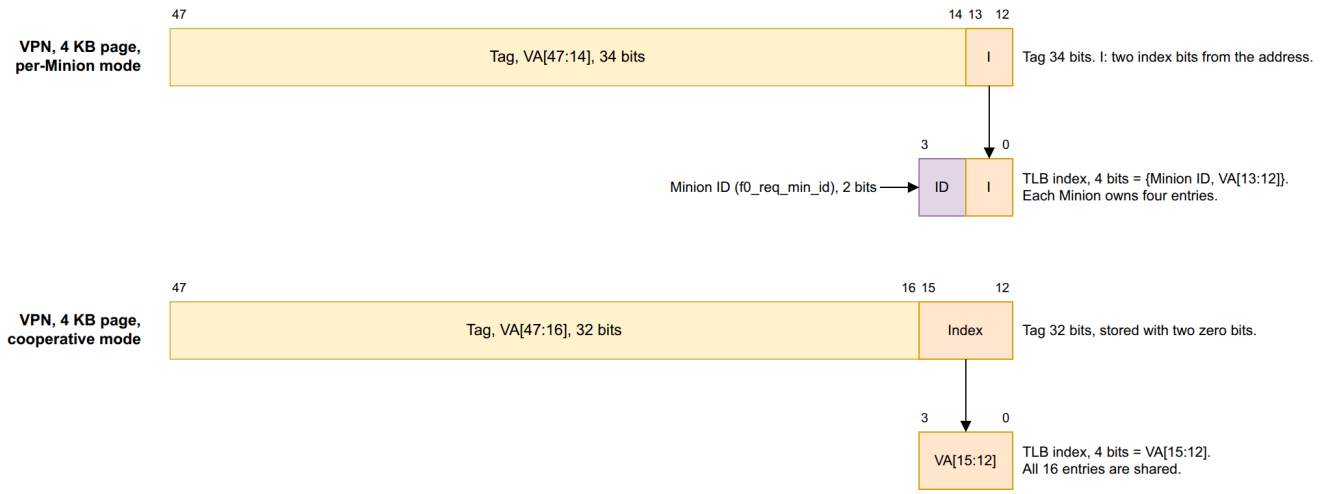


Figure 14. Instruction TLB index and tag for 4 KB pages

Table 20. Instruction TLB index and tag

Page size	Mode	Index (4 bits)	Stored tag, 34 bits (significant bits)
4 KB	Per-Minion	Minion identifier, VA[13:12]	{VA[47:14]} (34)
4 KB	Cooperative	VA[15:12]	{2 zeros, VA[47:16]} (32)
2 MB	Per-Minion	Minion identifier, VA[22:21]	{VA[47:23], 9 zeros} (25)
2 MB	Cooperative	VA[24:21]	{2 zeros, VA[47:25], 9 zeros} (23)
1 GB	Per-Minion	Minion identifier, VA[31:30]	{VA[47:32], 18 zeros} (16)
1 GB	Cooperative	VA[33:30]	{2 zeros, VA[47:34], 18 zeros} (14)

A change of `esr_shire_coop_mode` does not invalidate the entries. An entry filled in one mode stays valid in the other mode and can give a false hit, so the TLB must be invalidated after a change of mode (Section 1.4.5.5).

1.4.5.2. Lookup and Permission Check

Figure 15 shows the lookup in F1 as a sequence of decisions from top to bottom. The canonical-address check comes first because it depends only on the mode of the selected register, not on whether translation is enabled. The privilege level of the fetch, `vm_status.prv`, selects the translation register: `matp_info` of the requesting Minion in M-mode, `satp_info` otherwise. For an instruction fetch `mprv` is ignored. Translation is enabled when the selected register is not in Bare mode and the thread is not in debug mode. When translation is disabled, the TLB returns VA[39:12] as the physical page number and never misses.

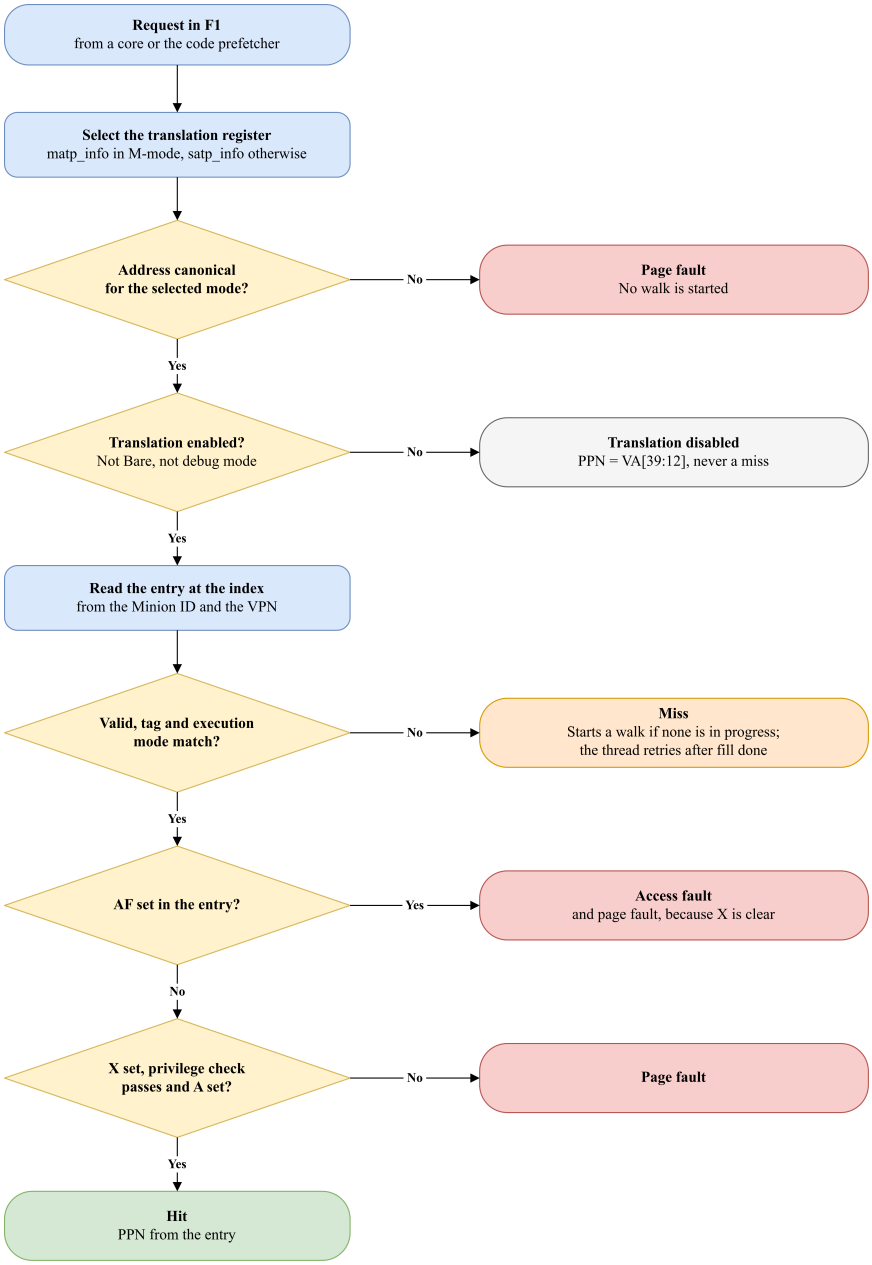


Figure 15. Instruction TLB lookup and permission check

An entry hits when it is valid, its tag matches, and its execution mode matches the fetch: an entry walked for M-mode hits only for M-mode fetches, and an entry walked for S-mode or U-mode only for those. A miss goes to the walk state machine (Section 1.4.5.3).

The TLB reports two faults, listed in Table 21. The page fault comes from three checks:

- **Privilege check:** in U-mode the entry must have `u` set; in S-mode and M-mode it must have `u` clear. `mstatus.SUM` does not apply to instruction fetches, so an S-mode fetch from a page with `u` set always fails the check.
- **Accessed bit:** the TLB does not set `a`. It raises a page fault instead, and software updates the page-table entry.
- **Canonical address:** in Sv48, VA[48] must equal VA[47]; in Sv39, VA[47:38] must also all be equal. A non-canonical address raises the page fault whether or not the entry hits, and starts no walk.

Table 21. Instruction TLB faults

Signal	Condition
`xcpt_if`, instruction page fault	The entry hits and `x` is clear, the privilege check fails, or `a` is clear; or the address is not canonical. The privilege check fails for an S-mode fetch from a page with `u` set, whatever the value of `mstatus.SUM`. `x` is also clear when the walk finds no valid leaf entry (Section 1.4.5.1).
`access_fault`, instruction access fault	The entry hits and its walk ended in an access fault.

A walk that ends in an access fault leaves `x` clear and AF set, so the retry reports both faults. The integer pipeline gives the access fault priority, and the thread takes an instruction access fault.

1.4.5.3. Page-Table Walk

A page-table walk reads the page tables in memory, from the root table of the selected translation register down to the leaf entry that holds the translation. The TLB does not read the page tables itself: it sends the walk to the shared page-table walker (Section 1.6).

For the walk the TLB uses a three-state machine, shown in Figure 16:

- **Ready:** no walk is in progress. A TLB miss on a canonical address captures the Minion, the virtual page number and the walk request, and moves the state machine to Request. The request carries the translation mode and root page number of the selected register, the privilege level and the virtual page number (Table 12).
- **Request:** sends the walk request to the walker. `ptw_req_valid` is held until the walker accepts the request with `ptw_req_ready`, and the state machine then moves to Wait. An invalidation withdraws the request and returns the state machine to Ready (Section 1.4.5.5).
- **Wait:** waits for the walker's response. The response is registered for one cycle, and in the next cycle the entry is written at the index of the request's Minion and virtual page number, unless the walk is cancelled. The state machine returns to Ready in the same cycle.

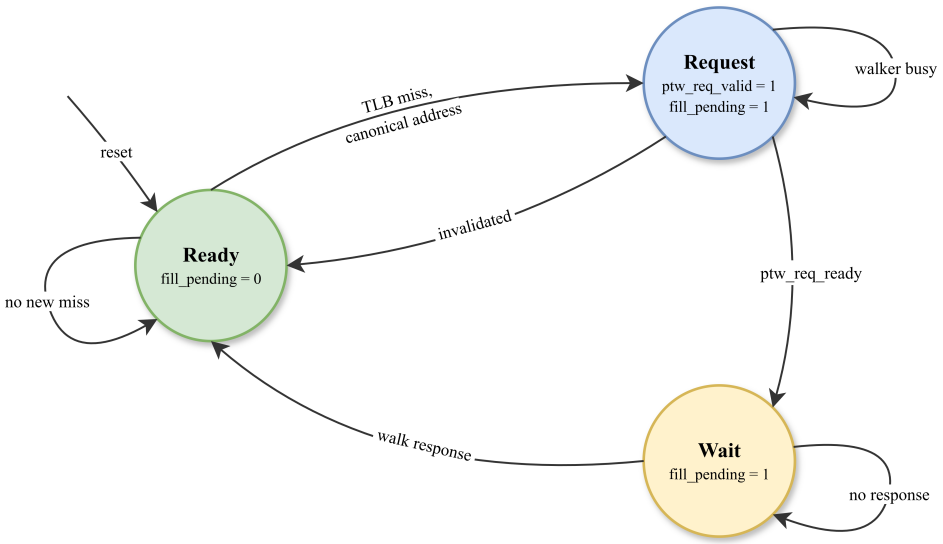


Figure 16. Instruction TLB walk state machine

A walk starts only from Ready, so one walk at most is in progress. A miss that arrives in Request or Wait receives a miss response and starts no walk. `fill_pending` is set in Request and Wait. The

micro-cache detects the end of the walk as the fall of `fill_pending` and turns it into `f2_tlb_fill_done`, which reaches the cores as `f5_resp_fill_done` three cycles later.

1.4.5.4. TLB Miss and Translation Fill

A request that misses in the TLB receives a miss response and starts a page-table walk if none is in progress. It never takes the miss handler of the micro-cache, because it has no physical address to send to the L1 instruction cache. The thread sleeps until the translation fill and then retries. Figure 17 shows a core request that misses in the TLB, and Table 22 lists its steps.

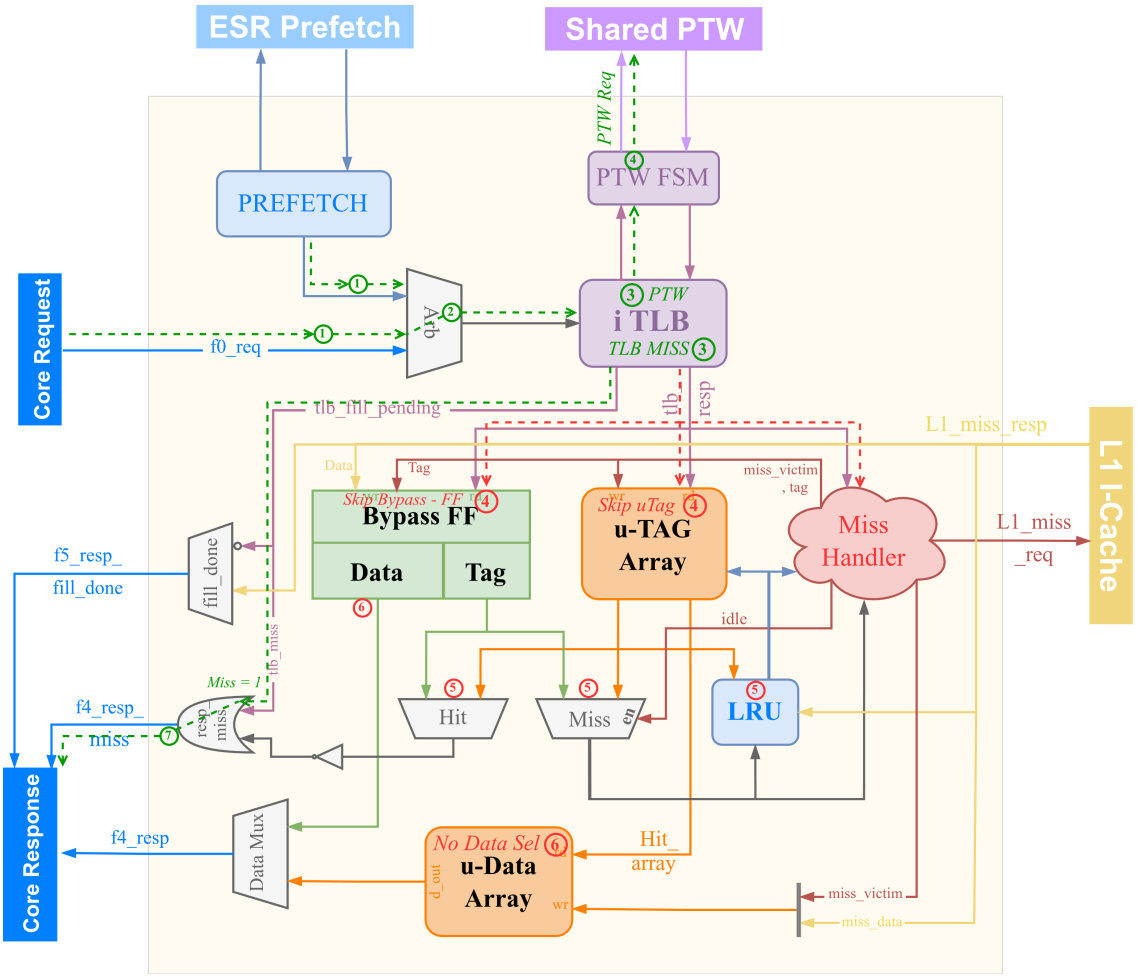


Figure 17. Core request TLB miss

Table 22. Core request TLB miss, steps by cycle

Cycle	Step	Block	Action
0 (F0)	1–2	Arbiter	Grants the core request, which always has priority.
1 (F1)	3	Instruction TLB	Misses. The walk state machine is in Ready, so it captures the request and moves to Request.
2 (F2)	4	Walk state machine	Drives the walk request to the shared page-table walker. The micro-tag array and the bypass register compare an address that is not valid, and their results are ignored.
2 (F2)	5	Hit and miss logic, LRU	No miss is accepted, because the TLB missed. The LRU state does not change.

Cycle	Step	Block	Action
3 (F3)	6	Micro-data array	Not read.
4 (F4)	7	Response	The TLB-miss bit forces a miss response. The thread sleeps until fill done.

Figure 18 shows a fetch by M0, TH0 that misses in the TLB. The miss is seen in F1 in cycle 1, the walk request is driven in cycle 2, and the miss response leaves F4 in cycle 4. The break stands for the walk. The walker responds in cycle 8, the entry is written in cycle 9, and fill done is driven in cycle 13. The thread then retries, hits in the TLB, and proceeds to the line lookup.

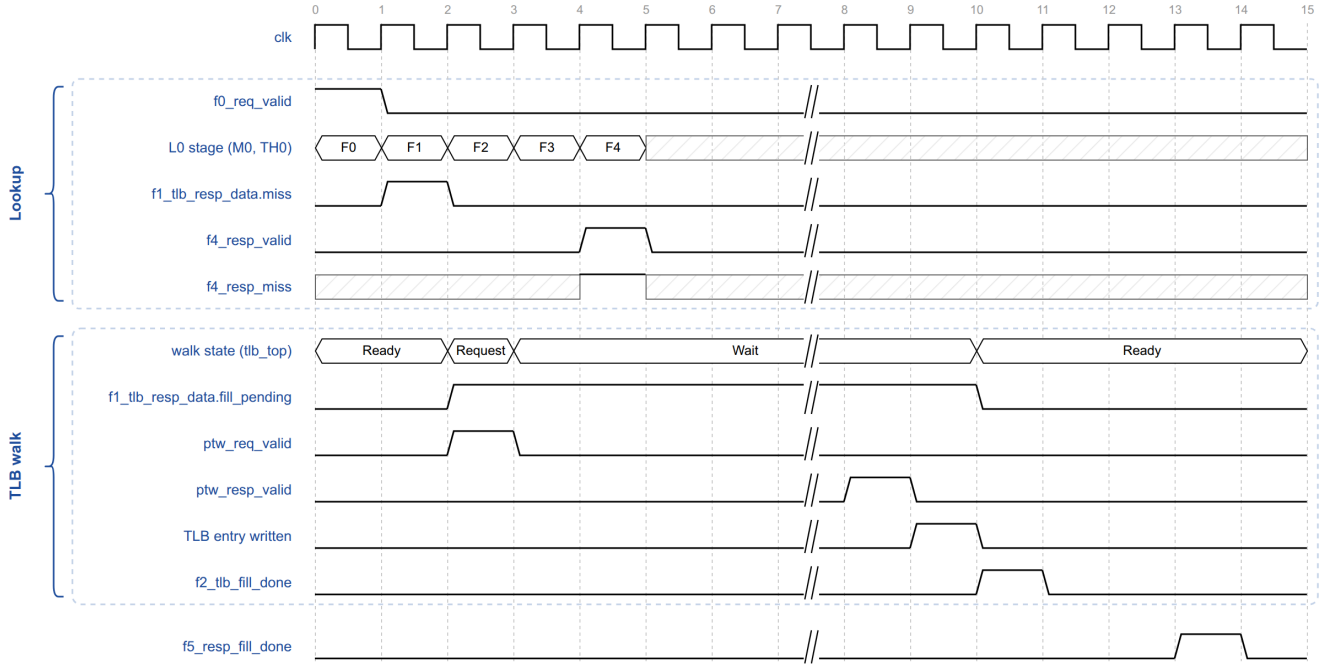


Figure 18. TLB miss, page-table walk and fill done

Figure 19 shows the end of a page-table walk, and Table 23 lists its steps. Cycle *N* is the cycle in which the walker response arrives. Fill done reaches the cores five cycles later: the response is registered, the entry is written, the fall of `fill_pending` is detected (Figure 25), and fill done passes F3 and F4.

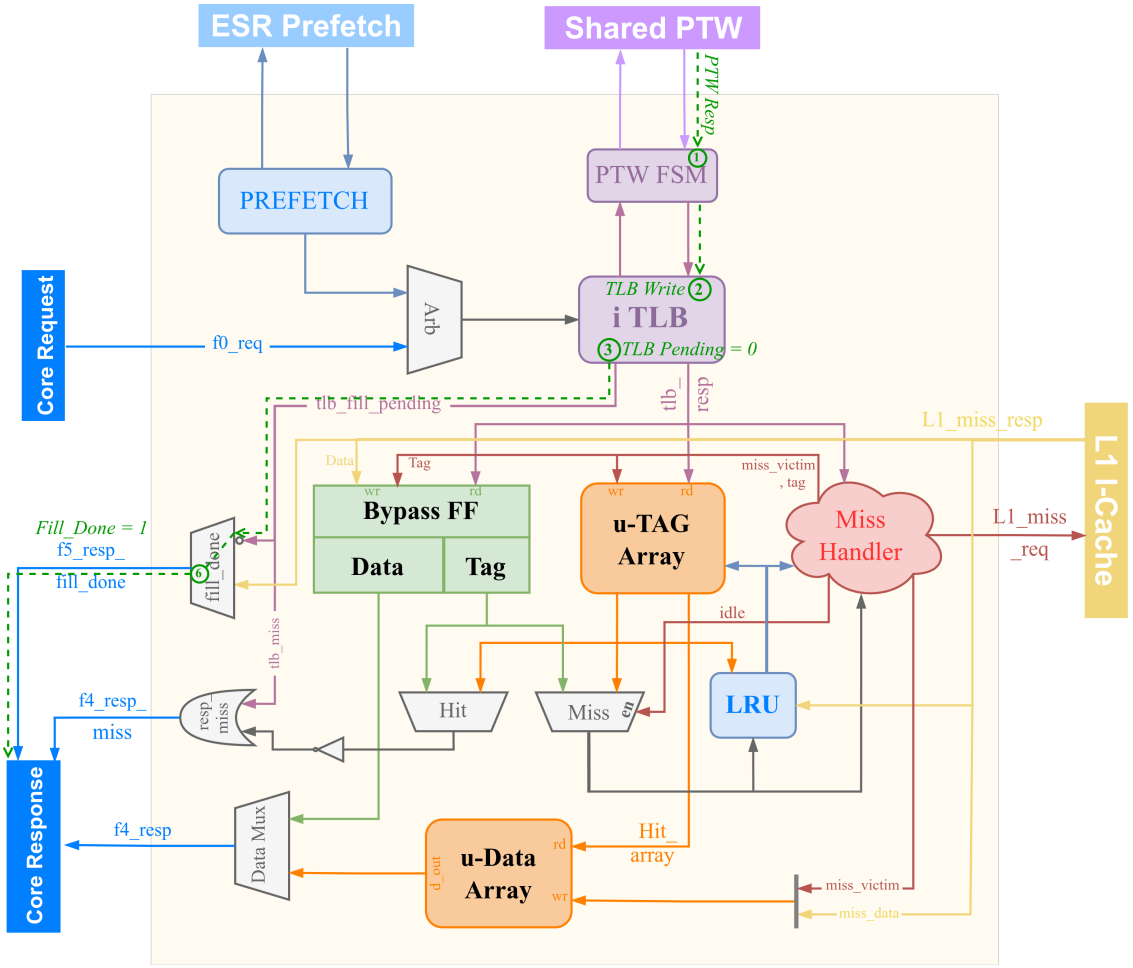


Figure 19. Translation fill

Table 23. Translation fill, steps by cycle

Cycle	Step	Block	Action
N	1	Walk state machine	In Wait, receives the walker response and registers it.
N + 1	2	Instruction TLB	Writes the entry at the index of the request, unless the walk is cancelled. The walk state machine returns to Ready.
N + 2	3	Instruction TLB	`fill_pending` falls. The micro-cache detects the fall and generates fill done.
N + 5	6	Fill done	`f5_resp_fill_done` wakes the sleeping threads.

1.4.5.5. Invalidation

`tlb_invalidate` carries one pulse per Minion, driven when the Minion writes 1 to bit 1 of its `cacheInvalidate` CSR (0x7D0). Outside cooperative mode a pulse clears the four entries of that Minion; in cooperative mode any pulse clears all 16. A walk in progress for the invalidated Minion, or any walk in cooperative mode, is cancelled: in Request it is withdrawn, and in Wait its response is discarded. The invalidation is held until the outstanding walk ends, so a response from before the invalidation can never be written. The OR of the four pulses, registered once, is forwarded to the walker on `ptw_invalidate`.

The entries carry no address-space identifier, and a change of `satp_info` or `matp_info` does not invalidate them. Software must invalidate the TLB through `cacheInvalidate` after it changes the translation.

1.4.5.6. Page Size

`esr_vmspagesize` selects 4 KB, 2 MB or 1 GB for every entry, and moves the index and the tag accordingly (Table 20). A change of `esr_vmspagesize` does not invalidate the entries either. The walker ends the walk at the page-table level of that size. A leaf found at a higher level is split into pages of the configured size, with the walker supplying the lower page-number bits from the virtual address.

1.4.6. Micro-Tag and Micro-Data Arrays

The micro-tag array and the micro-data array hold the 16 lines of the micro-cache, and the LRU selector chooses the line that a new miss replaces. The micro-tag array works in F2: it holds the physical tag of each line, and compares the tag of the request, translated by the instruction TLB in F1, with all 16 tags at once. The micro-data array works in F3: it holds the lines, and returns the 32-byte half that the request needs. The two arrays have the same 16 entries, so the entry that holds a tag also holds its line, and one one-hot vector selects both. Figure 20 shows the format of an entry of each array, and how a lookup passes from one array to the other.

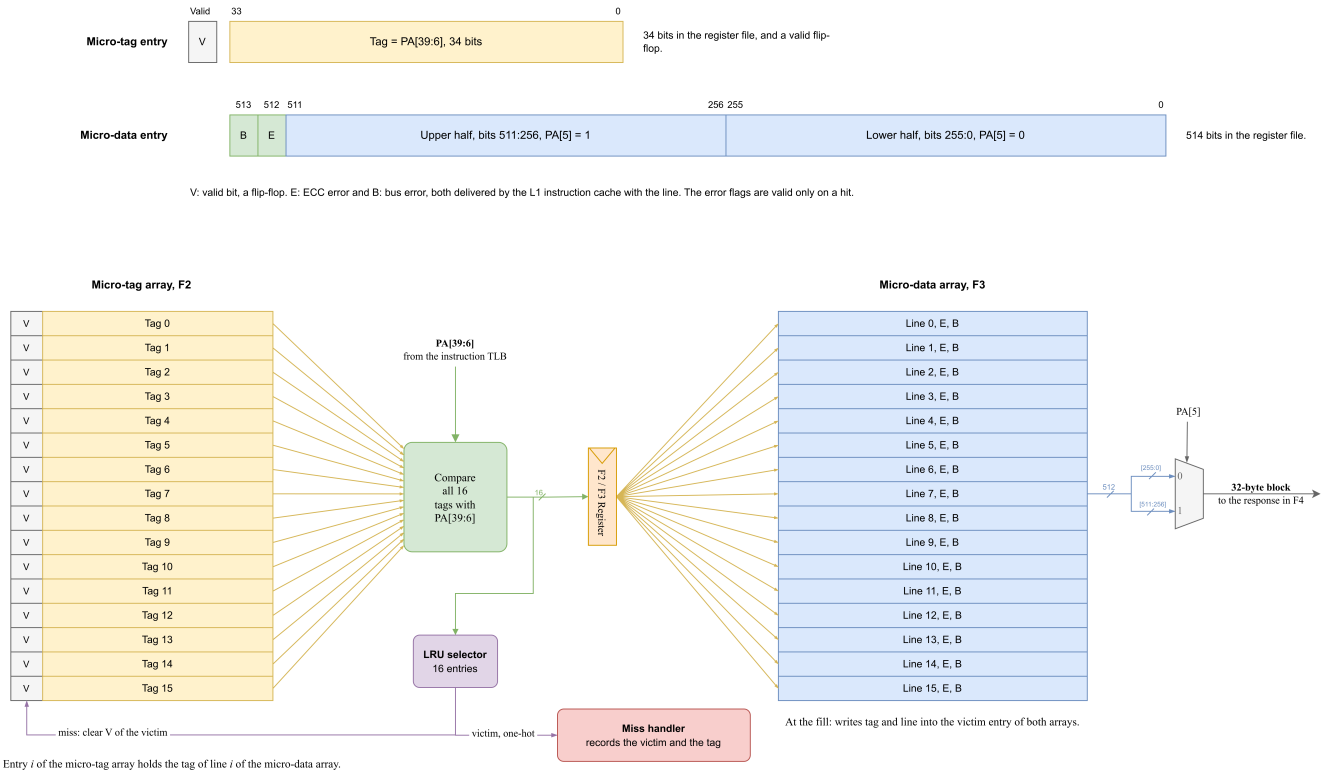


Figure 20. Micro-tag and micro-data arrays

1.4.6.1. Micro-Tag Array

The micro-tag array holds 16 tags, one for each line. It has no index: the cache is fully associative, so a line can be in any entry. Each tag is the line address, PA[39:6], 34 bits, stored in a latch-based register file that reads all 16 entries at once (`rf_latch_1r_1w_par`). Each entry also has a valid bit in a flip-flop beside the register file. Table 24 lists the ports of the array.

Table 24. Micro-tag array ports

Port	Stage	Function
Read	F2	Receives PA[39:6] of the request and returns the one-hot hit vector.
Victim	F2	Receives the LRU victim of an accepted miss and clears its valid bit.
Write	Fill cycle	Writes the tag that the miss handler recorded into the victim entry, and sets its valid bit. APB debug writes share the port, and the fill has priority.
Flush	F0	`f0_flush_data` clears all valid bits (Section 1.4.9).
APB	–	Reads and writes the tag and the valid bit of one entry (Section 1.4.12).

Lookup. In F2 the array compares the tag of the request with all 16 tags in parallel. An entry hits when it is valid, its tag equals PA[39:6], it receives no write in this cycle, and no flush is in progress. A line is allocated only after a miss, so at most one entry hits, and the result is a one-hot hit vector of 16 bits. The vector goes to two places in the same cycle: to the LRU selector, which records the entry as the most recent, and through the stage register to the micro-data array, which reads the line with it in F3.

Lookup in the fill cycle. While a fill writes a tag, the array also compares the request with the tag that is written. The entry under write cannot hit on its old tag, but a request whose tag matches the new tag hits on that entry. A request that arrives in the fill cycle therefore sees the new line and does not miss on it (Figure 21).

Miss. When no entry hits and the miss handler accepts the miss, the array frees the victim that the LRU selector chooses: it clears the valid bit of the victim in F2, so the entry cannot hit while its new line is outstanding. The miss handler records the victim and the tag, and requests the line from the L1 instruction cache in F3 (Section 1.4.8.1). A bypassed or prefetch miss frees no victim, because its fill writes no array.

Fill. When the L1 instruction cache returns the line for the outstanding miss request, the array writes the tag that the miss handler recorded into the victim entry, and sets the valid bit (Section 1.4.6.4).

The valid bits change on four events:

- **Accepted miss:** the valid bit of the victim is cleared.
- **Fill:** the valid bit of the victim is set, unless a flush arrived while the fill was outstanding.
- **Flush:** all valid bits are cleared.
- **Debug write:** the valid bit is written with the tag.

1.4.6.2. Micro-Data Array

The micro-data array holds the 16 lines, entry for entry with the micro-tag array. Each entry is 514 bits in a latch-based register file with one-hot read and write selects (`rf_latch_1r_1w_dec`): the 512-

bit line, the ECC-error flag (bit 512) and the bus-error flag (bit 513) that the L1 instruction cache delivers with the line. Table 25 lists the ports of the array.

Table 25. Micro-data array ports

Port	Stage	Function
Read	F3	Receives the one-hot hit vector from F2 and returns the selected entry. APB debug reads share the port and wait for a cycle with no request in F3.
Write	Fill cycle	Writes the line and its error flags into the victim entry. APB debug writes share the port, and the fill has priority.
APB	–	Reads and writes one entry (Section 1.4.12).

Read. In F3 the array reads the entry that the hit vector selects, and PA[5] selects the 32-byte half that goes to the response in F4. On a miss the hit vector is all zeros, and the response carries no instruction block. The error flags go to the response with the block, and they are valid only on a hit.

Write. The fill writes the line and its error flags into the victim entry, in the same cycle as the tag (Section 1.4.6.4).

1.4.6.3. LRU Selector

The LRU selector (`lru_sel`) decides which line a new miss replaces. It keeps the exact order of use of all 16 entries, and the victim is always the entry that was used least recently. For each pair of entries the selector holds one bit that tells which of the two was used more recently. An entry is the victim when it is older than each of the other 15, and the selector presents the victim as a one-hot vector, in every cycle, to the micro-tag array and the miss handler.

Three events in F2 count as a use, and each makes the entry the most recent one:

- **Hit:** the entry that hits.
- **Accepted miss:** the victim, when the miss handler takes the miss.
- **Line fill:** the victim again, when its line is written. While the line was outstanding, other entries continued to hit and became more recent, so without this second update the new line could be the next victim as soon as it arrives.

The selector is not updated for a prefetch request or a bypassed request, because neither keeps a line in the arrays. The selector also ignores the valid bits, so it does not look for an empty entry first. Empty entries exist only after reset or a flush, and after reset the selector hands the entries out in order, entry 0 first.

1.4.6.4. Fill Write

A line from the L1 instruction cache is written into the tag and data arrays in the cycle `f0_l1_miss_resp_valid` is asserted, at the victim recorded when the miss was accepted. The latch-based register files need their write enable one cycle ahead of the write, and `f0_l1_miss_resp_early_valid` supplies it; the L1 derives it from the next-cycle value of its response valid. Prefetch and bypassed fills write neither array.

Figure 21 shows the write in cycle 3. All the requests in the figure are for the same line, line A: the outstanding miss in `f0_miss_addr`, which the L1 instruction cache answers in cycle 3, and the lookups of M1, TH0 and M2, TH0. M1, TH0 reaches F2 in cycle 2, before the write, and misses; the miss handler is still in Fill_Wait, so nothing is recorded for it. M2, TH0 reaches F2 in cycle 3, the write cycle, and hits on the line being written. Their responses leave F4 in cycles 4 and 5, the handler returns to Ready in cycle 4, and fill done is driven in cycle 6, three cycles after the write.

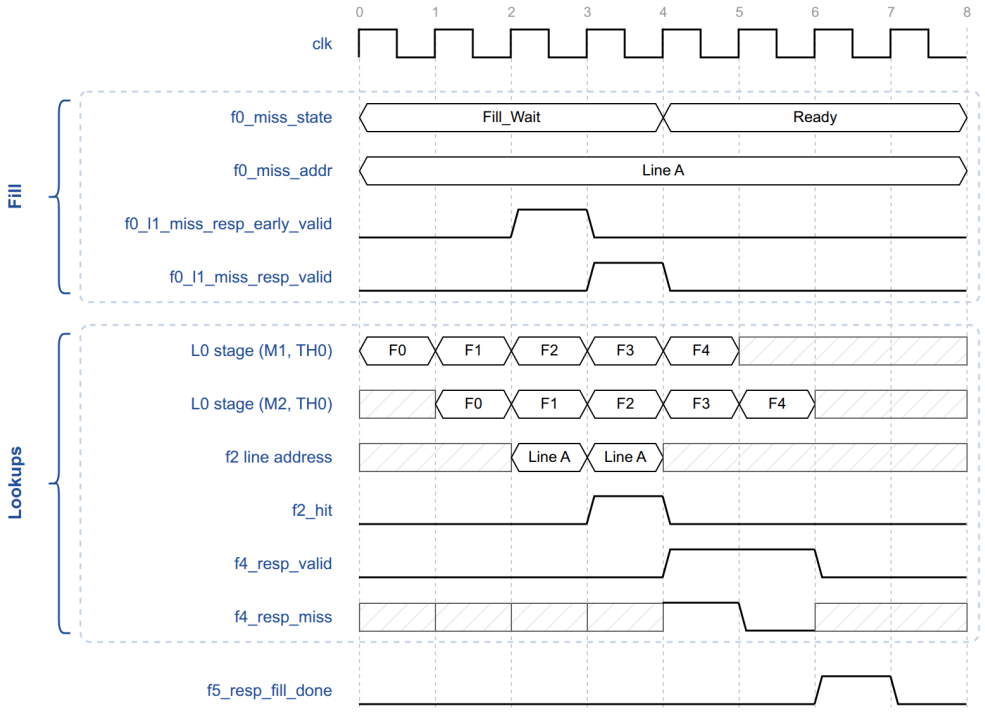


Figure 21. Line fill and lookup in the fill cycle

1.4.7. PMA Check

The PMA unit (`icache_pma_unit`) checks the physical address in F2 against the regions from which instructions may be fetched, and reports an access fault and whether the address is cacheable. Table 26 lists the regions, decoded from the address-map defines in `shire_defines.vh`. The privilege used is the privilege level of the fetch, `vm_status.prv`; for a prefetch request it is the privilege of the prefetch operation.

Table 26. Instruction-fetch regions checked by the PMA unit

Region	Physical address	Fetch permitted	Cacheable
BootROM	0x0_0200_8000 to 0x0_0200_9FFF, 8 KB	All privilege levels	Yes
SRAM	0x0_0200_C000 to 0x0_0200_CFFF, 4 KB	All privilege levels	Yes
MRAM, M-mode box	From 0x0_4000_0000, the first 2^{mmode_size} 4-KB granules	M-mode only	Yes
MRAM, S-mode box	Above the M-mode box, up to 2^{smode_size} granules from the base	M-mode and S-mode	Yes

Region	Physical address	Fetch permitted	Cacheable
MRAM, other	The rest of 0x0_4000_0000 to 0x0_40FF_FFFF, 16 MB	All privilege levels	Yes
Any other address		Access fault	No

The two boxes exist only while `esr_mprot.mprot_en` is set; with it clear, the whole MRAM region is in the last MRAM row. The granule number is PA[23:12], so the box sizes are powers of two from 4 KB to 16 MB.

The access fault from the PMA unit is suppressed when the TLB misses, because the physical address is not valid then. Every region that passes the check is cacheable, so a fetch that does not fault is always reported as cacheable.

1.4.8. Miss Handling

1.4.8.1. Miss Handler

The miss handler fetches missing lines from the L1 instruction cache. It sends the line request, and it keeps the information about the miss that the fill needs when the line comes back. It serves three kinds of miss: a core request that misses in the micro-tag array, a bypassed core request that misses in the bypass register, and a prefetch request. A TLB miss never reaches the miss handler. Without a translation there is no physical address to send to the L1, so the walk state machine of the instruction TLB handles the TLB miss instead (Section 1.4.5.4).

The miss handler holds one line request at a time and uses a three-state machine, shown in Figure 22:

- **Ready:** no line request is outstanding, and the handler can take a miss. It takes one in F2, asserts `f2_miss` and moves to Request, when the request in F2 is valid, does not hit, has a translation, and has no page fault or access fault.
- **Request:** drives the line request on `f0_l1_miss_req_valid` with the physical address, and holds it until the holding register of the instance in `icache_top` takes it (`f0_l1_miss_req_ready`). The handler then moves to Fill_Wait.
- **Fill_Wait:** waits for the line. The line is written in the cycle `f0_l1_miss_resp_valid` arrives, and the handler is back in Ready in the next cycle.

A miss that arrives while the handler is in Request or Fill_Wait still receives its miss response, but nothing is recorded for it (Section 1.4.8.2).

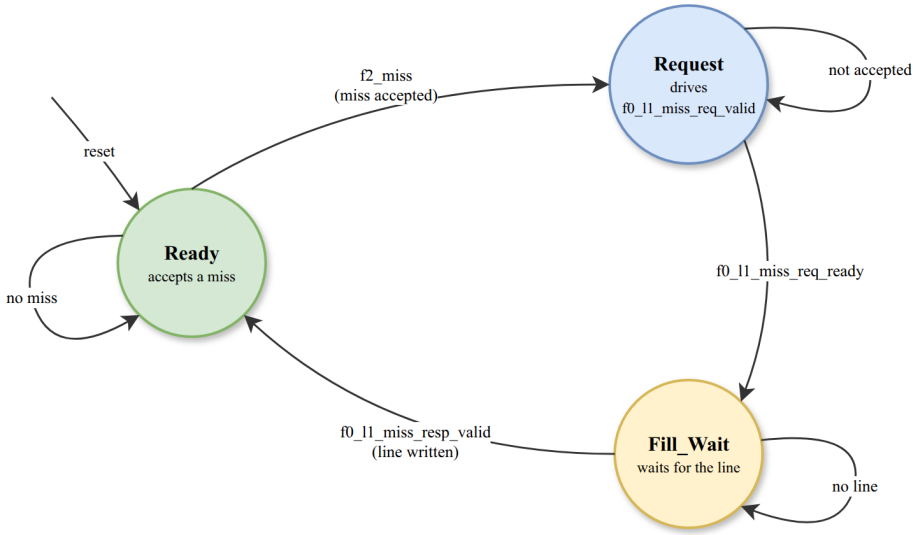


Figure 22. Miss handler state machine

The response from the L1 instruction cache carries the line and its two error flags, but not its address. So when the handler takes a miss, it records everything the fill needs later:

- **Tag:** PA[39:6] of the miss, in `f0_miss_addr`. It is the address of the line request, and the tag that the fill writes into the micro-tag array or into the bypass register.
- **Victim:** the one-hot entry that the LRU selector chose, in `f0_miss_victim`. The fill writes the tag and the line into this entry.
- **Flags:** bypassed, prefetch, and L0 write disabled, the last set for a bypassed or prefetch request. They decide where the line goes: into the arrays, into the bypass register, or nowhere in the micro-cache.

1.4.8.2. Line Miss

A request that translates but does not hit receives a miss response. If the handler is in Ready, the miss is accepted and the line is requested; if it is busy, nothing is recorded, and the thread fetches the line after a later fill done, when it retries and finds the handler idle. A core miss that is not recorded also opens the window in which the code prefetcher holds back (unattended-miss state machine). Figure 23 shows a line miss that finds the miss handler in Ready, and Figure 24 one that finds it busy. Table 27 lists the steps of both side by side. The busy case is the path that the livelock guard of the Front End bounds (Section 1.3.8).

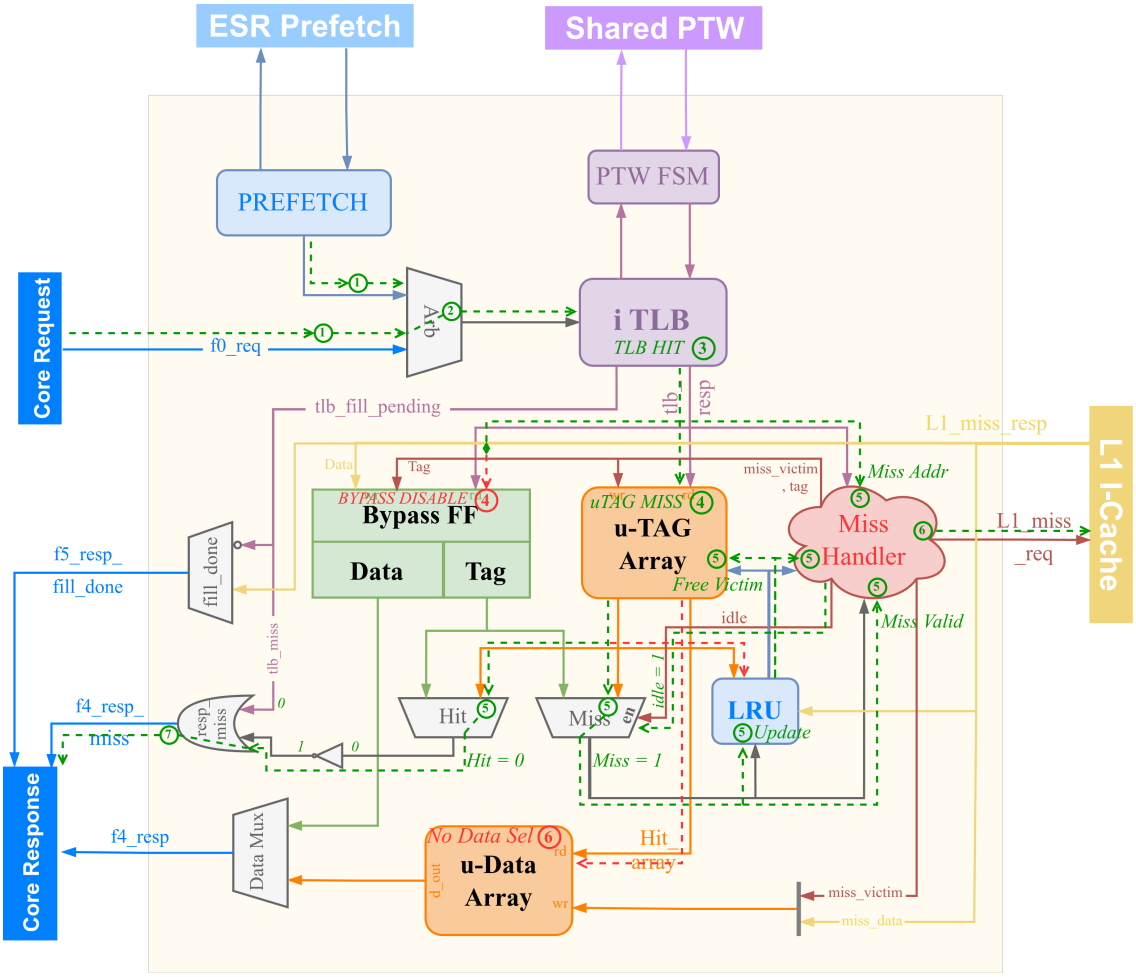


Figure 23. Core request line miss

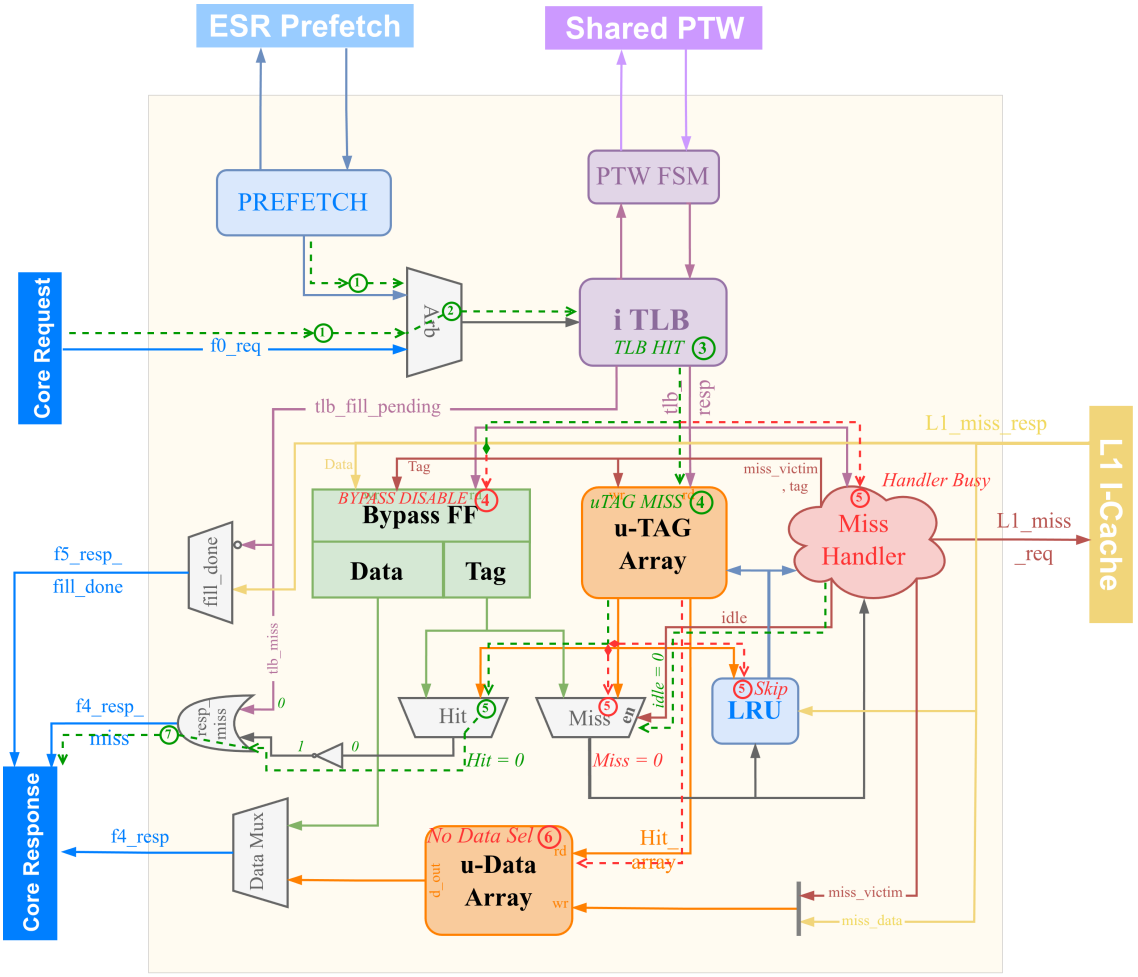


Figure 24. Line miss with the miss handler busy

Table 27. Core request line miss, steps by cycle

Cycle	Step	Miss handler in Ready (Figure 23)	Miss handler busy (Figure 24)
0–1 (F0–F1)	1–3	The arbiter grants the core request, and the instruction TLB translates its address.	
2 (F2)	4	The physical tag hits no line in the micro-tag array.	
2 (F2)	5	Hit is 0 and miss is 1. The miss handler records the tag, the LRU victim and the flags of the request. The valid bit of the victim is cleared, and the LRU state records the victim.	Hit is 0, and miss is 0 because the miss handler is not in Ready. Nothing is recorded, no victim is cleared, and the LRU state does not change. The unattended-miss state machine moves to FillDone_Wait and holds the code prefetcher back (unattended-miss state machine).
3 (F3)	6	The miss handler, in Request, drives the line request to the L1 instruction cache. The micro-data array is not read.	The micro-data array is not read, and no line request is made.
4 (F4)	7	The response is a miss. The thread sleeps until the next fill done, and then retries.	

1.4.8.3. Fill and Thread Wake-Up

Fill done tells the sleeping threads that a line or a translation has arrived, so that they retry. Figure 25 shows how the micro-cache generates it from two sources:

- **Line fill:** `f0_l1_miss_resp_valid`, in the cycle in which the L1 instruction cache returns a line.
- **Translation fill:** `f2_tlb_fill_done`, a one-cycle pulse at the end of a page-table walk. The micro-cache registers `fill_pending` from the instruction TLB into `f2_tlb_fill_was_pending`, and the pulse is high when the registered value is 1 and the current value is 0. This is the fall of `fill_pending`, one cycle after the TLB entry is written.

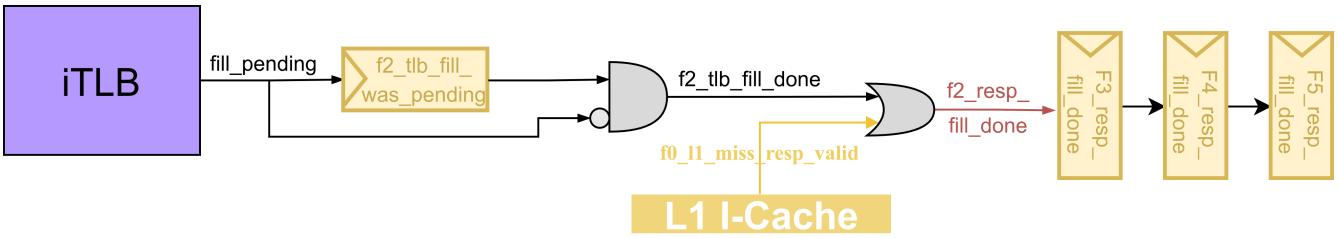


Figure 25. Fill-done generation

The OR of the two sources is `f2_resp_fill_done`. It passes through the F3, F4 and F5 stage registers and reaches the cores as `f5_resp_fill_done`, three cycles later. Every line fill produces fill done, including prefetch and bypassed fills, which write no array. The reason fill done follows the response by one stage is given in Section 1.3.5.

A core line fill writes the line into the arrays. Figure 26 shows the transaction, and Table 28 lists its steps; cycle *N* is the cycle in which `f0_l1_miss_resp_valid` is asserted. The fill of a bypassed miss and the fill of a prefetch miss are explained in their own sections (Section 1.4.10, Section 1.4.11), and the translation fill in Section 1.4.5.4.

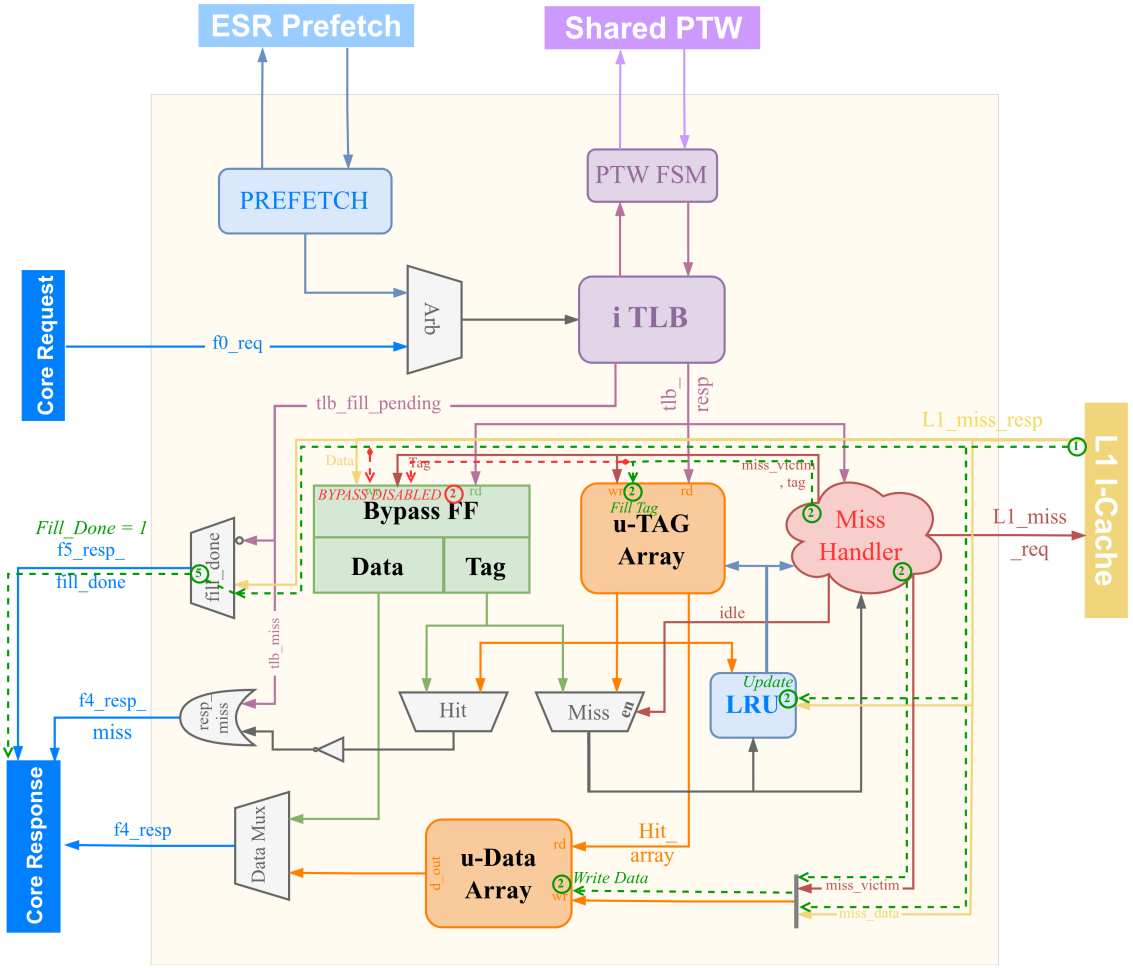


Figure 26. Core line fill

Table 28. Core line fill, steps by cycle

Cycle	Step	Block	Action
N	1	L1 instruction cache	Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags.
N	2	Micro-tag array, micro-data array, LRU, miss handler	The line and its error flags are written into the victim entry of the micro-data array, and the tag recorded by the miss handler into the micro-tag array, which makes the entry valid. The LRU state records the entry. The miss handler returns to Ready, and fill done is generated.
N + 3	5	Fill done	`f5_resp_fill_done` wakes the sleeping threads.

1.4.9. Flush

The micro-cache does not snoop memory, so a change to code in memory is not seen by the lines that it already holds (Table 15). After software changes code, it flushes the instruction caches by writing 1 to bit 0 of the `cacheInvalidate` CSR (0x7D0). Each Minion sends this request to the Neighborhood, where `neigh_shared_icache` forms `f0_flush_data` as the OR of the requests of the eight Minions, registered once. It drives the flush to both micro-caches and to the L1 instruction cache.

In the flush cycle the valid bits of all micro-tag entries are cleared, and no lookup hits. A line that is outstanding at the flush was requested before it, so it can hold the old code. Its fill is still written when it arrives, but the entry stays invalid: the flush is remembered until the miss handler returns to Ready. The flush does not affect the TLB or the bypass register; TLB entries are invalidated through bit 1 of the same CSR (Section 1.4.5.5).

1.4.10. Bypass Mode

Bypass mode turns off instruction caching in the Neighborhood. The cores still fetch their instructions, but neither the L0 micro-cache nor the L1 instruction cache keeps a line, so every new line comes from the Shire Cache, the shared L2 and L3 cache of the Minion Shire. Bypass mode is a chicken bit, a switch that disables a feature in silicon. It is set by the `bypass_icache` bit of the Neighborhood chicken ESR, which reaches the shared instruction cache as `esr_bypass_icache`.

1.4.10.1. Request Path in Bypass Mode

The Front End is not connected to the Shire Cache directly. Its fetch port connects to the L0 micro-cache, the L0 micro-cache connects to the L1 instruction cache, and only the L1 instruction cache connects to the Shire Cache. A bypassed request therefore still passes through both caches, and the design must respect two rules of that path:

- **Physical addresses.** The fetch address is virtual, but the L1 instruction cache and the Shire Cache take physical addresses.
- **Fixed latency.** The Front End expects every response a fixed four cycles after the grant, for a hit and for a miss alike, and recognizes its own response by that timing alone ([sec-ch2-fe-track]). A hit that is served from the Shire Cache takes many more cycles than that.

A core request first enters the L0 micro-cache, because the fetch port of the core is connected to it. The micro-cache pipeline already solves the translation: the instruction TLB translates the address in F1, and the PMA unit checks it in F2, before any request leaves for the L1 instruction cache. In F2 the micro-cache also sees that the request is bypassed. It skips the micro-tag array and, if the miss handler is in Ready, the miss handler registers the line request (Section 1.4.8.1). The request is sent to the L1 instruction cache in the next cycle, in F3.

The L1 instruction cache also sees the bypass bit. It treats every request as a miss and sends it to the Shire Cache. The line returns through its data RAM and reaches the micro-cache with its ECC and bus-error flags, as for any line fill. The L1 instruction cache writes no tag and does not update its LRU state, so it keeps no copy of the line.

1.4.10.2. Bypass Register

The line request leaves the micro-cache in F3, and the core expects its response in the next cycle, in F4, as a hit or a miss. In one cycle the request cannot reach the L1 instruction cache, look up the Shire Cache and come back to the micro-cache, so the core always receives a miss. The line then arrives after a number of cycles that is not fixed, and by that time the thread sleeps. The micro-cache must hold the line until the thread retries, but its arrays must not keep it.

To hold the line, the micro-cache adds the bypass register, a small cache of one line built from flip-flops. A bypassed fetch of a new line then runs as follows:

1. The core request is translated and checked, and in F3 the miss handler sends the line request to the L1 instruction cache.
2. The response in F4 is a miss, and the thread sleeps.
3. When the line arrives from the L1 instruction cache, the micro-cache loads the line, its error flags and the tag that the miss handler recorded into the bypass register. It then pulses fill done, as for a normal line fill, and the thread wakes.
4. The thread retries. Its request hits in the bypass register in F2, and the response in F4 carries the instruction block.

The next request, for the other 32-byte half of the same line, also hits in the bypass register. A request for a different line misses: the thread sleeps, the bypass register is loaded with the new line, fill done wakes the thread, and its retry hits. Figure 27 shows the complete flow.

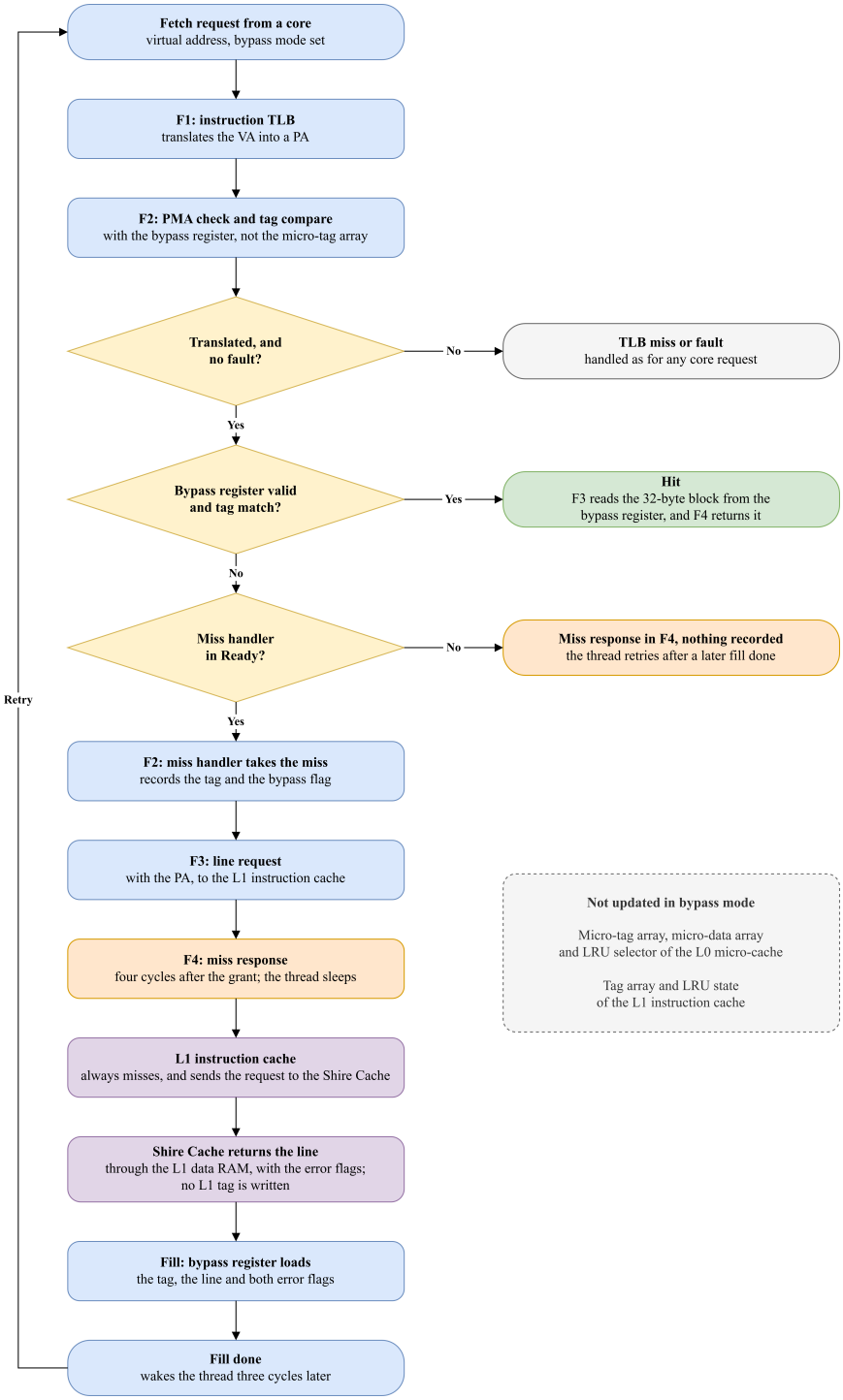


Figure 27. Bypassed fetch flow

Figure 28 shows the format of the bypass register: the tag, the 512-bit line, the two error flags and a valid bit.

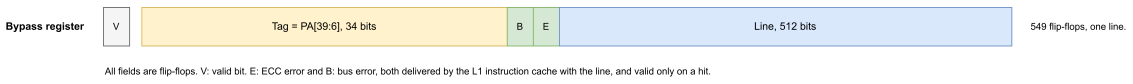


Figure 28. Bypass register

The bypass register is used as follows:

- **Lookup.** In F2 a bypassed request compares its tag with the bypass register instead of the micro-tag array. The register does not hit in the cycle in which it is loaded.
- **Fill.** The miss handler records the bypass flag with the miss, so the fill loads the bypass register

instead of the arrays. The micro-tag array, the micro-data array and the LRU selector are not updated (Table 30).

- **Read.** In F3 the 32-byte half and the error flags come from the bypass register.
- **Invalidation.** The register is invalidated when a request that is not bypassed passes F2.

Bypass mode has two further effects:

- **Mode changes.** A change of bypass mode flushes neither cache level. The micro-tag and micro-data arrays keep the lines that they held before bypass mode, and those lines hit again when it ends. If the code in memory changed meanwhile, software must flush the instruction caches (Section 1.4.9).
- **Prefetch.** A prefetch request in bypass mode loads nothing, because the L1 instruction cache keeps no line and the bypass register takes only the lines of core misses. The code prefetcher is therefore of no use in bypass mode.

1.4.10.3. Bypassed Transactions

Figure 29 shows a bypassed core request that hits in the bypass register, and Figure 30 one that misses. Table 29 lists the steps of both side by side. In both, the result of the micro-tag array is not used, the micro-data array is not read, and the LRU state does not change.

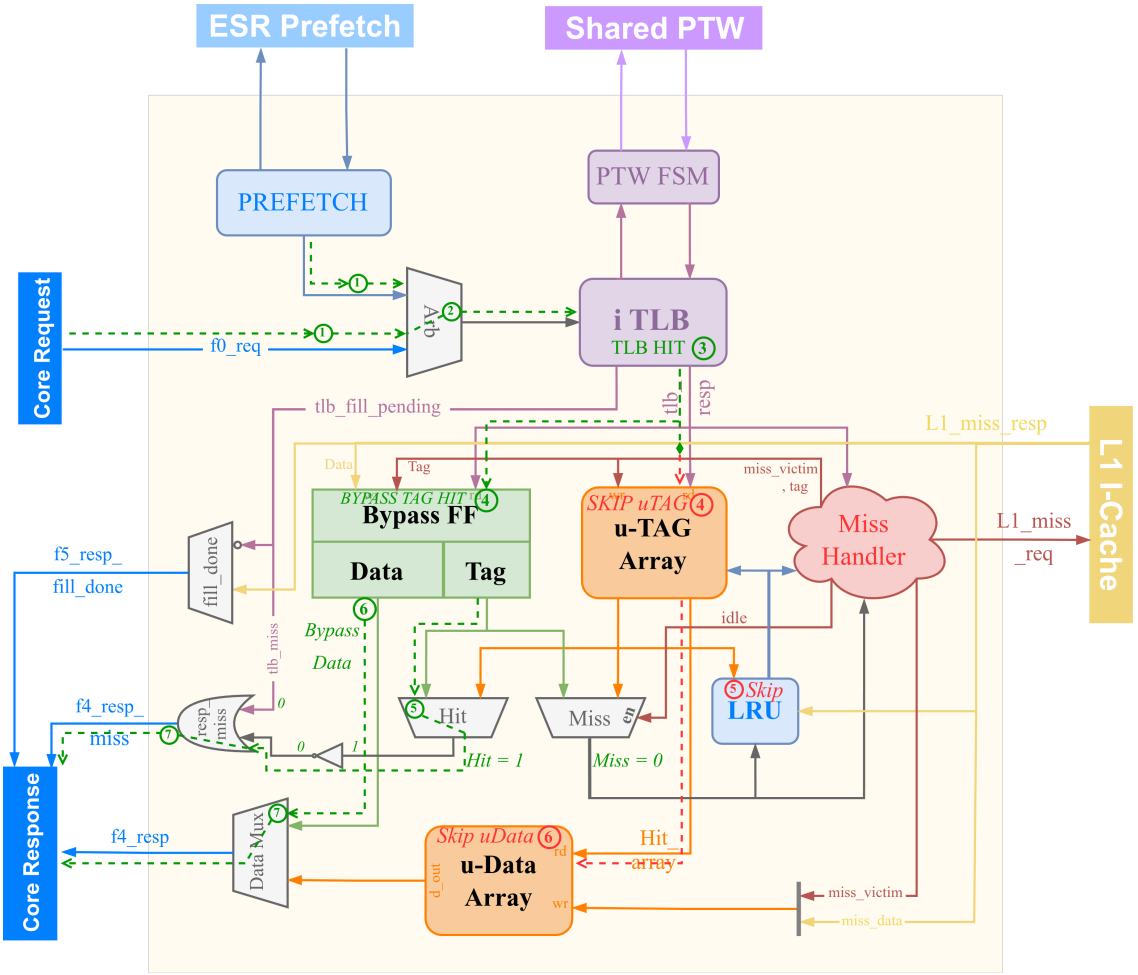


Figure 29. Bypassed core request hit

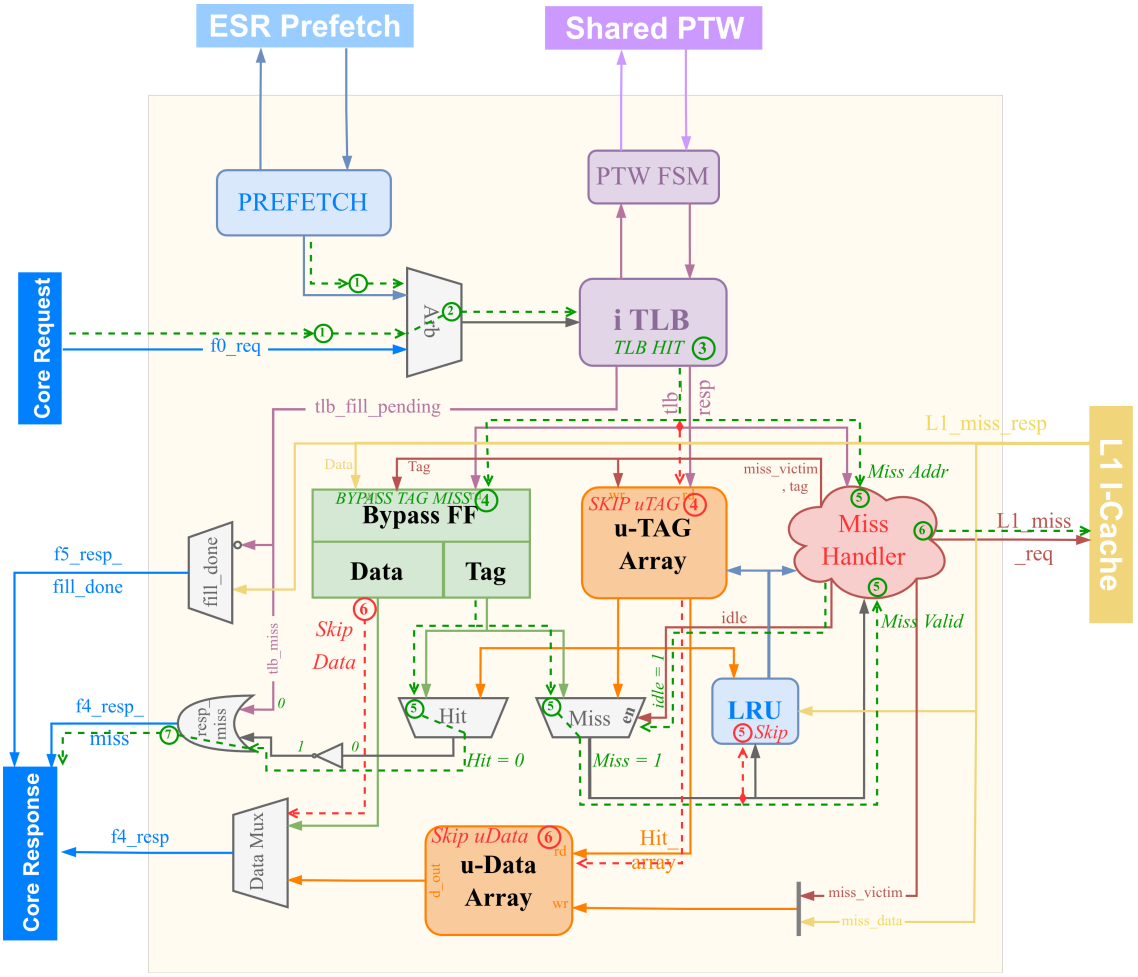


Figure 30. Bypassed core request miss

Table 29. Bypassed core request, steps by cycle

Cycle	Step	Bypass register hit (Figure 29)	Bypass register miss (Figure 30)
0–1 (F0–F1)	1–3	The arbiter grants the core request, and the instruction TLB translates its address.	
2 (F2)	4	The physical tag matches the bypass tag.	The physical tag does not match the bypass tag.
2 (F2)	5	Hit is 1 and miss is 0.	Hit is 0 and, with the miss handler in Ready, miss is 1. The miss handler records the tag and the bypass flag, and no victim.
3 (F3)	6	The bypass register supplies the 32-byte half and the error flags.	The miss handler, in Request, drives the line request to the L1 instruction cache.
4 (F4)	7	The response carries the instruction block, with the miss bit clear.	The response is a miss. The thread sleeps until fill done.

Figure 31 shows the fill of the bypassed miss, and Table 30 lists its steps. The line goes into the bypass register, and the thread then retries and hits there.

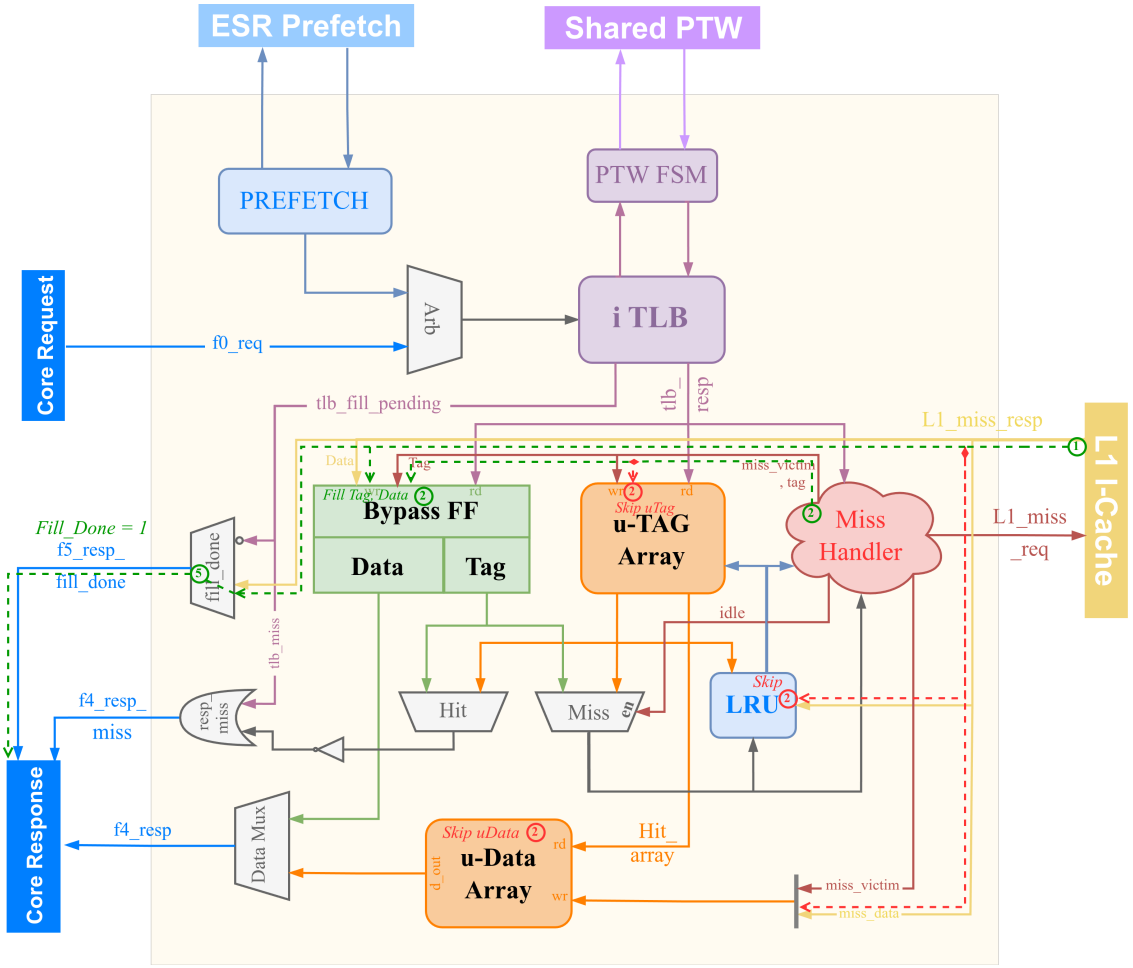


Figure 31. Bypassed line fill

Table 30. Bypassed line fill, steps by cycle

Cycle	Step	Block	Action
N	1	L1 instruction cache	Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags.
N	2	Bypass register, miss handler	The bypass register loads the line data and its two error flags from the L1 instruction cache response, and the tag recorded by the miss handler, and becomes valid. The arrays are not written, and the LRU state does not change. The miss handler returns to Ready, and fill done is generated.
N + 3	5	Fill done	`f5_resp_fill_done` wakes the sleeping threads.

1.4.11. Code Prefetcher

The code prefetcher is a service that software uses to preload critical code into the L1 instruction cache before the cores execute it. Without it, the first fetch of each new line misses in both cache levels, and the thread sleeps until the line comes from the Shire Cache. A prefetch operation gives a start virtual address, a number of lines and a privilege level, and the prefetcher loads that many consecutive lines. A single operation prefetches at most 64 lines into the L1 instruction cache. Software writes the number of lines minus one, so the maximum is configured as 63 (Table 10).

The prefetcher is built into the north micro-cache, which serves Minions 0 to 3; the south micro-cache has no prefetcher (Section 1.4.11.1). The prefetcher and the cores work in parallel. The Minions do not stop during an operation: they continue to fetch, and their requests always have priority. The prefetcher uses only the cycles in which no core request enters the micro-cache, and it holds back while a core waits for the miss handler (unattended-miss state machine).

1.4.11.1. Prefetch Path

To load a line into the L1 instruction cache, a request for that line is enough. The L1 instruction cache answers every request with the line: on a hit it reads the line from its data RAM, and on a miss it fetches the line from the Shire Cache, stores it and returns it. Whether the request hits or misses, the line is in the L1 instruction cache when the response returns, which is all that a prefetch needs. A prefetch operation is therefore a sequence of line requests, one for each consecutive line.

Two facts decide where these requests come from. The L1 instruction cache has one request port, shared by the two micro-caches through a round-robin arbiter (Section 1.5.4), and a prefetcher with a physical address could simply be a third input of that arbiter. But the operation gives a virtual address. For its translation, the prefetcher can use the instruction TLB of the micro-cache, as a bypassed request does (Section 1.4.10.1). The prefetcher is therefore built into a micro-cache, and sends its requests through the micro-cache pipeline beside the core requests. One requester is enough, so only the north micro-cache has it.

Figure 32 shows the prefetcher in F0. The prefetch address and line-count registers load the operation from `esr_prefetch_conf` on `esr_prefetch_start`. For each new line, the address moves to the next line, 64 bytes higher, and the line count decreases by one. The prefetch state machine presents its request with `f0_prefetch_req_valid` only when the request is qualified (unattended-miss state machine). The 2:1 arbiter always selects the core request, so the prefetch request passes only in a cycle with no core request, with Minion identifier 0. `f1_is_prefetch` marks the prefetch request for the later stages.

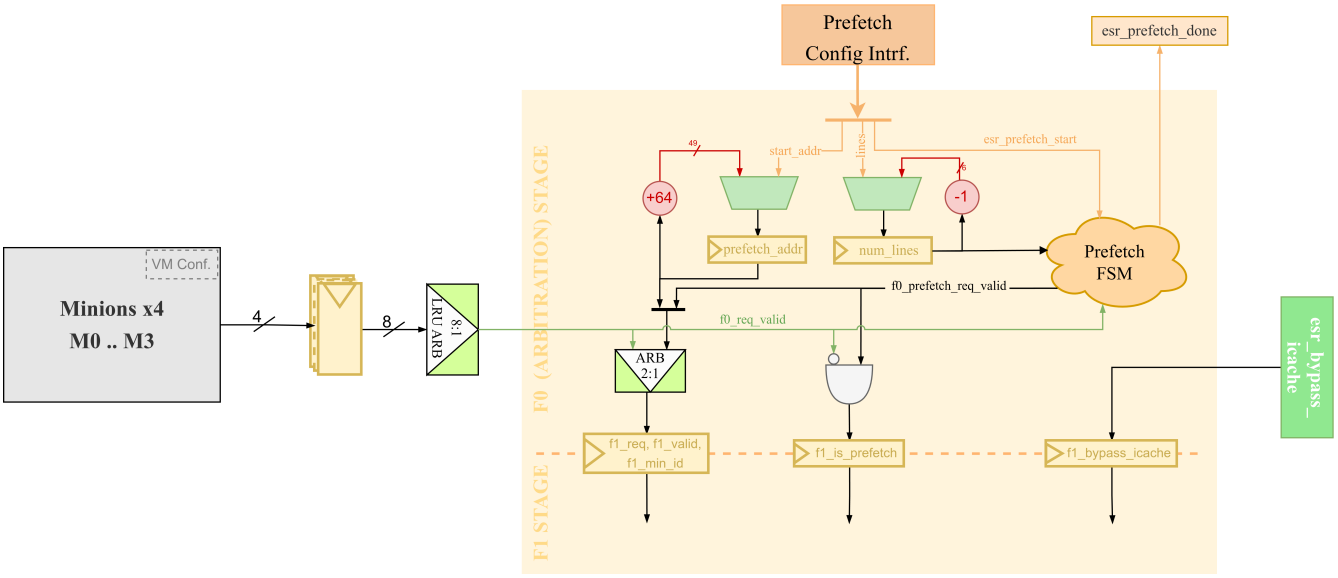


Figure 32. Code prefetcher in the F0 stage

A prefetch request passes the pipeline as follows:

- **F0.** The pipeline arbitrates between the core request and the prefetch request, and always gives

priority to the core request (Figure 32). A prefetch request enters only in a cycle with no core request, so it never delays a fetch.

- **F1.** The instruction TLB translates the address. A prefetch request carries Minion identifier 0, so its translation uses the translation configuration of Minion 0, with the privilege of the operation. On a TLB miss a page-table walk starts as for a core request.
- **F2.** The PMA unit checks the physical address. The request is forced to miss, and the result of the micro-tag array is not used, because the purpose of the request is a line request to the L1 instruction cache. If the miss handler is in Ready, it takes the request and records that it is a prefetch.
- **F3.** The miss handler sends the line request to the L1 instruction cache.
- **F4.** No response is sent, because no thread waits for the line.

The L1 instruction cache handles the line request with its normal behavior, and returns the line. The miss handler sees that the request was a prefetch and discards the line: the micro-tag array, the micro-data array and the LRU selector are not updated. The line stays only in the L1 instruction cache, where both micro-caches can use it. The fill still generates fill done to the Minions, as every line fill does.

1.4.11.2. Starting an Operation

Software starts an operation by writing the start virtual address and the number of lines minus one to one of three Shire ESRs: `icache_uprefetch`, `icache_sprefetch` or `icache_mprefetch`. The ESR written sets the privilege of the operation. A write is accepted only in cooperative mode. Pending requests are served in privilege order: an M-mode request at once, and an S-mode or U-mode request only when no operation is in progress or the current operation has the same or lower privilege. The served request reaches the north micro-cache as `esr_prefetch_conf` (Table 10), with a pulse on `esr_prefetch_start` that starts the operation.

Figure 33 shows the format of the prefetch ESR that software writes, and of the configuration that the north micro-cache receives.

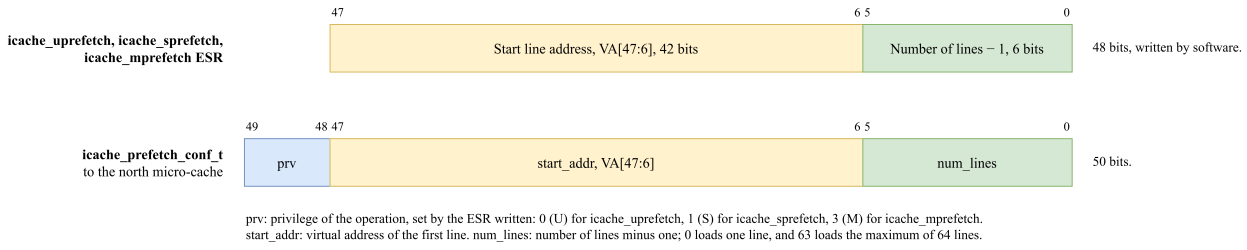


Figure 33. Prefetch configuration

1.4.11.3. Prefetch State Machines

The prefetcher uses two state machines, shown in Figure 34:

- **Prefetch state machine.** Requests the lines of the operation one by one, and waits for each line before it requests the next.
- **Unattended-miss state machine.** Keeps the prefetcher from reducing the performance of the

Minions. The miss handler holds one line request at a time, and a core miss that finds it busy is not recorded: the thread sleeps and retries after the next fill done. This state machine notes that such a thread waits for the miss handler, and holds the prefetcher back until the thread retries.

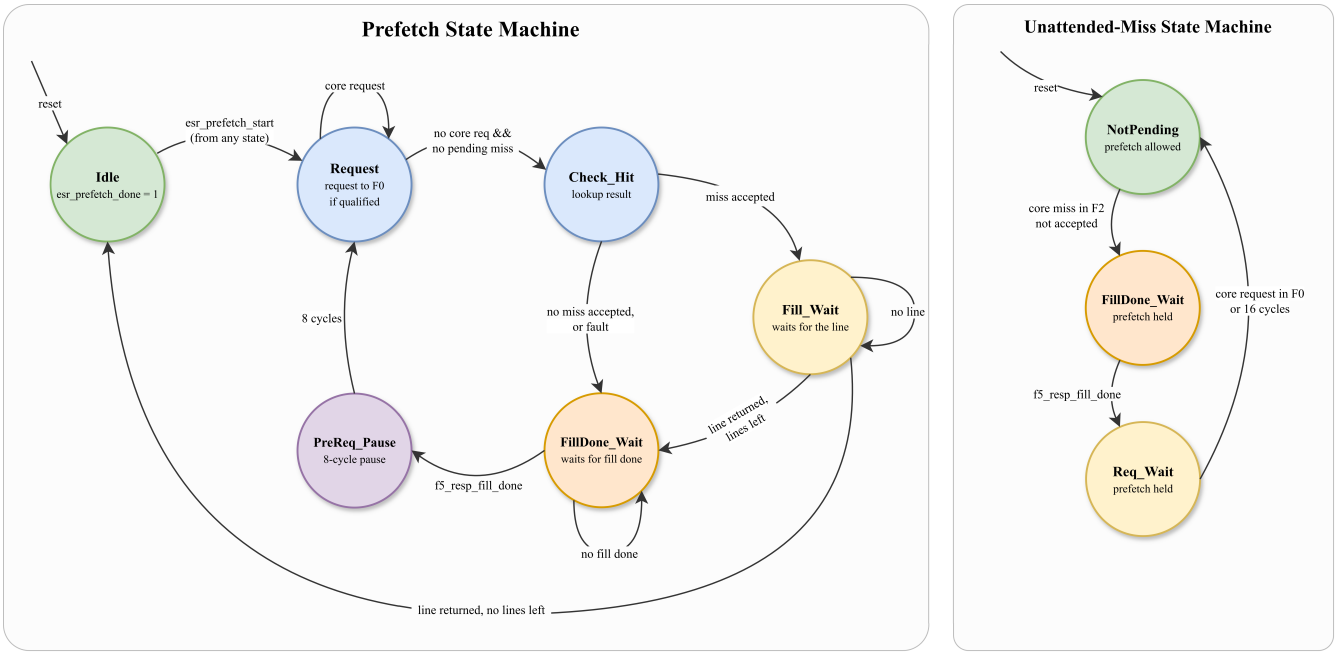


Figure 34. Code prefetcher state machines

The prefetch state machine has six states:

- **Idle.** No operation is in progress, and `esr_prefetch_done` is 1. A pulse on `esr_prefetch_start` loads the start address and the line count, and moves the state machine to Request. The pulse has this effect in every state, so a new operation cancels the current one and starts at once.
- **Request.** Clears `esr_prefetch_done` and presents the prefetch request to F0 when it is qualified. The state machine moves to Check_Hit when the request enters F0, in a cycle with no core request.
- **Check_Hit.** Waits for the request to reach F2. If the miss handler takes it, the state machine moves to Fill_Wait. If the request takes a page fault or an access fault, the line is skipped, and the state machine moves to the next line and to FillDone_Wait. If neither, because the miss handler is busy or the TLB missed, it moves to FillDone_Wait and requests the same line again later.
- **Fill_Wait.** Waits for the line from the L1 instruction cache. After the last line it moves to Idle and sets `esr_prefetch_done`; otherwise it moves to the next line and to FillDone_Wait.
- **FillDone_Wait.** Waits for `f5_resp_fill_done`. Fill done wakes the threads that sleep on a miss, and it also marks the end of the fill for which a busy miss handler or a TLB walk was waiting.
- **PreReq_Pause.** Waits eight cycles and then returns to Request. The north micro-cache serves eight threads, two in each of its four Minions, and the pause gives all eight threads the time to wake up and retry after fill done, before the next prefetch request.

The unattended-miss state machine has three states:

- **NotPending.** No core miss waits for the miss handler. A core request that misses in F2 while the miss handler is busy is not recorded, and moves the state machine to FillDone_Wait.

- **FillDone_Wait.** The thread of that core request sleeps. The state machine waits for `f5_resp_fill_done`, which wakes the thread, and then moves to Req_Wait.
- **Req_Wait.** The thread is awake, and its retry is expected. The state machine returns to NotPending when a core request enters F0, so that a new unattended miss can be noted. If no core request comes, it returns to NotPending after 16 cycles, so that the prefetcher is not held back for good.

A prefetch request is qualified when both state machines allow it. The prefetch state machine is in Request, so an operation is in progress and waits to request its next line, and the unattended-miss state machine is in NotPending, so no thread waits for the miss handler after a miss that was not recorded. In the first cycle in which no core request uses F0, the prefetcher then sends the request for its line into the pipeline.

1.4.11.4. Prefetch Transactions

Figure 35 shows one prefetch request, and Table 31 lists its steps. The request is forced to miss, so it always takes the miss handler if the handler is in Ready, and it produces no response. Figure 36 shows its fill.

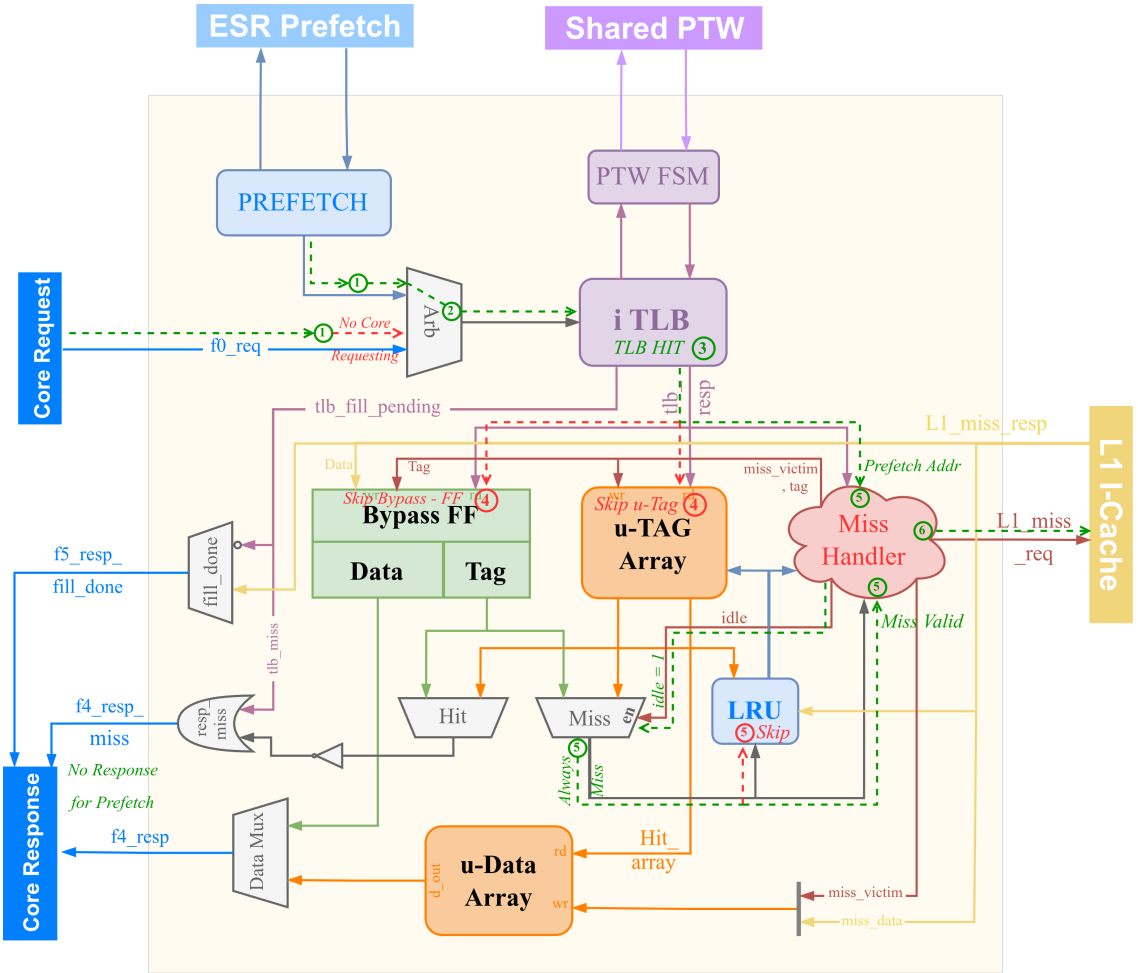


Figure 35. Prefetch request

Table 31. Prefetch request, steps by cycle

Cycle	Step	Block	Action
0 (F0)	1	Code prefetcher	The prefetcher is in Request and its request is qualified, and no core request is present.
0 (F0)	2	Arbiter	Grants the prefetch request.
1 (F1)	3	Instruction TLB	Hits and returns the physical page number.
2 (F2)	4	Micro-tag array, bypass register	Their results are not used, because a prefetch request is forced to miss.
2 (F2)	5	Hit and miss logic, miss handler	Miss is 1 whenever the miss handler is in Ready. The miss handler records the physical address and the prefetch flag. No victim is cleared, and the LRU state does not change.
3 (F3)	6	Miss handler	In Request, drives the line request to the L1 instruction cache.
4 (F4)	–	Response	None. A prefetch request produces no response.

Figure 36 shows the fill of a prefetch miss, and Table 32 lists its steps. The miss handler recorded the prefetch with L0 writes disabled, so nothing is written in the micro-cache and only the L1 instruction cache keeps the line. The fill still produces fill done, and fill done starts the pause of the prefetcher.

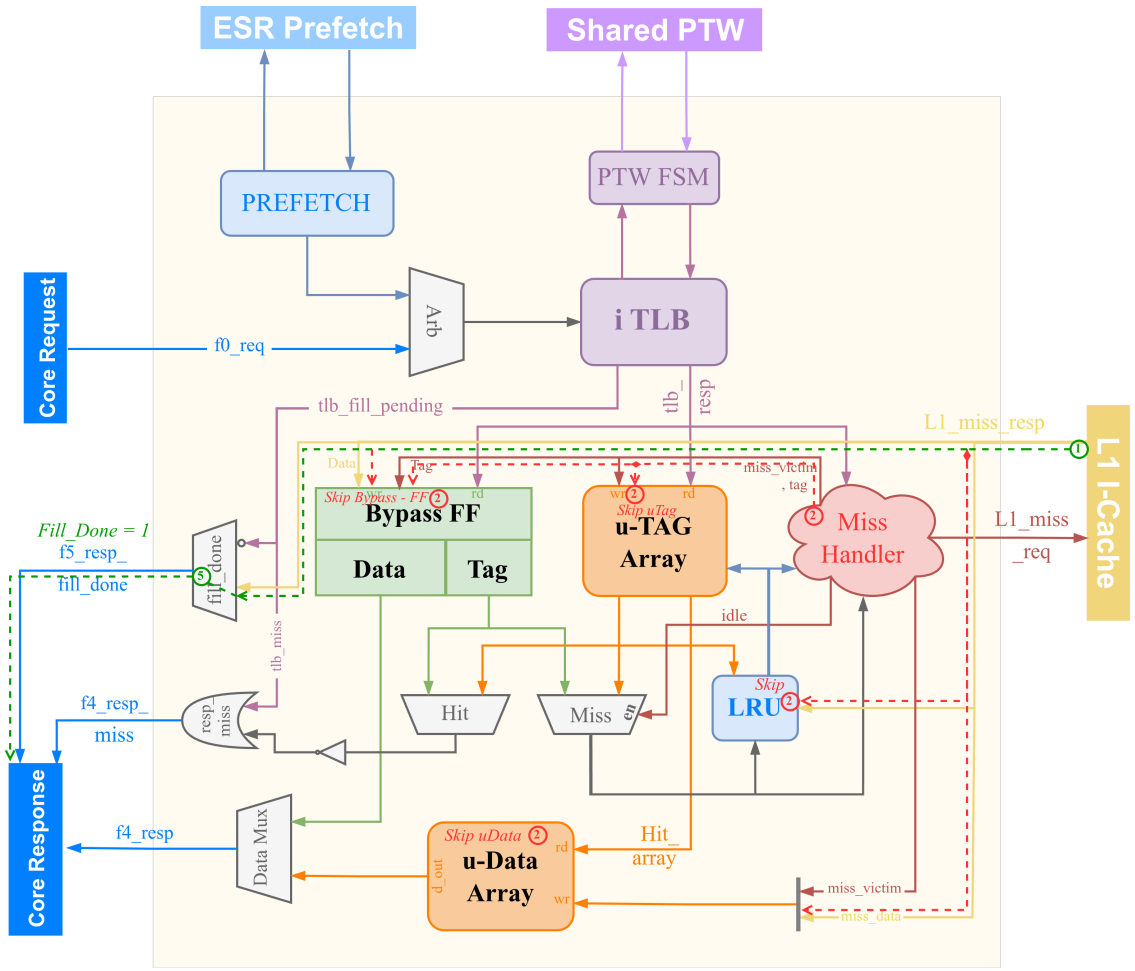


Figure 36. Prefetch line fill

Table 32. Prefetch line fill, steps by cycle

Cycle	Step	Block	Action
N	1	L1 instruction cache	Asserts `f0_l1_miss_resp_valid` and returns the line and its two error flags.
N	2	Miss handler, code prefetcher	Nothing is written in the micro-cache, and the LRU state does not change. The miss handler returns to Ready, and fill done is generated. The code prefetcher leaves Fill_Wait for FillDone_Wait, or for Idle after the last line.
N + 3	5	Fill done	`f5_resp_fill_done` wakes the sleeping threads, and the code prefetcher starts its eight-cycle pause.

Figure 37 shows the first two lines of a two-line operation. The start pulse in cycle 0 moves the prefetcher to Request, and its request enters F0 in cycle 1, a cycle with no core request. The miss is accepted in F2 in cycle 3, and the L1 request is driven in cycle 4. The line returns in cycle 9 and the prefetcher advances to line A+1. Fill done in cycle 12 starts the eight-cycle pause, shown compressed by the second break. The request for line A+1 enters F0 in cycle 18.

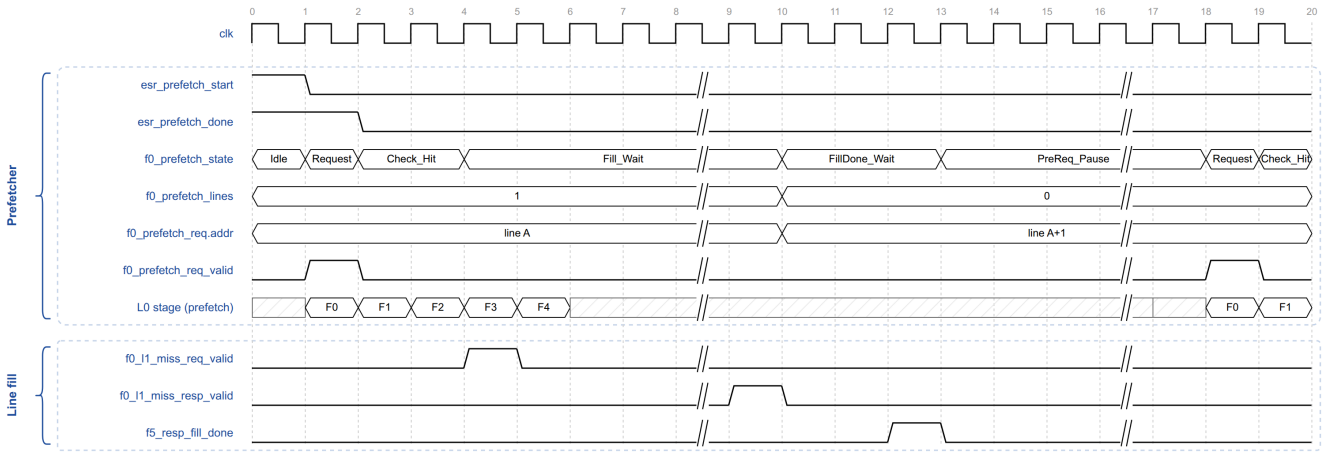


Figure 37. Code prefetch sequence

1.4.11.5. Completion

`esr_prefetch_done` resets high. It falls when an operation starts, and rises again when the fill of the last line returns. It is returned to the Shire ESR block, where software reads it to know that the operation is complete.

1.4.12. Debug Access

The micro-tag and micro-data arrays of both instances can be read and written over APB. `icache_top` decodes the upper bits of its 10-bit APB address and passes the low eight bits to the selected instance (Table 33).

Table 33. Micro-cache debug address

Bits	Use
9	1: micro-cache space. 0 selects the L1 tag array.

Bits	Use
8	Instance: 0 for the north micro-cache, 1 for the south.
7	0: micro-data array. 1: micro-tag array or error flags.
6:3	Line number, 0 to 15.
2:0	With bit 7 = 0: 64-bit word within the line. With bit 7 = 1: bit 2 selects the tag (0) or the error flags (1).

- **Tag access:** a read returns the tag in bits 33:0 and the valid bit in bit 34; a write sets both from the same bits. A write waits for a cycle in which no fill writes the array.
- **Data access:** a read returns one 64-bit word of the line. A write is a read-modify-write of the whole 514-bit entry: the line is read, the selected word replaced, and the entry written back. Reads wait for cycles in which F3 does not read the array, and writes for cycles in which no fill writes it.
- **Error flags:** bit 0 is the ECC-error flag and bit 1 the bus-error flag of the line, read or written in the same way.

`apb_pslverr` is never asserted.

1.5. L1 Instruction Cache

1.5.1. Overview

The L1 instruction cache is the second level of the Neighborhood's shared instruction cache. It holds 32 KB of code in 128 sets of four 64-byte lines, and both L0 micro-caches share it. A micro-cache sends it a line request only when a lookup misses (Section 1.4.8.1), and the code prefetcher of the north micro-cache uses the same request to load lines (Section 1.4.11). Every address that reaches the L1 instruction cache is physical, because the micro-cache translates it first, so the L1 instruction cache has no TLB.

The L1 instruction cache is split across the two voltage regions, for the reasons given in Section 1.2. Its tags, its LRU state and its control logic are in `icache_top`, inside the Neighborhood. Its data RAM is in the high-voltage region of the Shire Channel, and every access to the data RAM passes through the voltage-domain crossing (Section 1.1.3). A line that is not in the cache comes from the Shire Cache, over the memory request path of the Neighborhood (Section 1.7).

The design rests on three decisions:

1. **One request at a time:** the cache accepts a new line request only after it returns the response to the previous one. A hit returns its line 13 cycles after the line request, and a miss after the Shire Cache access.
2. **Write by address only:** the line from the Shire Cache does not pass through the L1 instruction cache on its way to the data RAM. The L1 instruction cache sends only the address of the victim entry, and the Shire Channel writes the line from the Shire Cache response.
3. **ECC on the data RAM:** each 64-bit block of a line carries its own 8-bit code. A single-bit error is corrected, and a double-bit error removes the line from the cache. Both are reported to the

micro-cache and logged (Section 1.5.9).

Figure: L1 instruction cache block diagram.

The sections that follow describe the interface (Section 1.5.2), the organization (Section 1.5.3), the arbitration between the micro-caches (Section 1.5.4), the pipeline of a hit (Section 1.5.5), the arrays (Section 1.5.6), the data RAM (Section 1.5.7), the miss path (Section 1.5.8), errors (Section 1.5.9), flush and the two special requests (Section 1.5.10) and debug access (Section 1.5.11).

1.5.2. Interface

The L1 instruction cache is part of `icache_top`, the module that also contains the two micro-caches. Table 34 lists the ports of `icache_top` that belong to the L1 instruction cache. The line request and the line response between a micro-cache and the L1 instruction cache are internal to `icache_top` and are given with the micro-cache ports (Table 9).

Table 34. L1 instruction cache I/O signals

Signal	Dir	Width / type	Description
Configuration and control			
`esr_bypass_icache`	in	1	Bypass mode. Every lookup misses, and no tag or LRU state is written (Section 1.5.10).
`esr_err_log_ctl`	in	`esr_icache_err_log_ctl_t`	Error-log control ESR, 3 bits. Bit 0 enables the report of single-bit ECC errors to the micro-cache, and bit 1 of double-bit ECC errors. Bit 2 is used only by the Neighborhood ESR block.
`f0_flush_data`	in	1	Invalidates every line of the cache (Section 1.5.10).
Shire Cache request			
`f0_l2_miss_req_valid`	out	1	Request for a line from the Shire Cache. Held until accepted.
`f0_l2_miss_req`	out	`et_link_nodata_req_info_t`	Read of a full line at the line-aligned physical address. The Neighborhood channel sets the source field.
`f0_l2_miss_req_ready`	in	1	The memory request arbiter of the Neighborhood accepts the request.
`f0_l2_miss_req_disable_next`	in	1	Holds the request back for one cycle. The Neighborhood channel asserts it while a 512-bit eviction takes its second grant from the memory request arbiter.
Shire Cache response			

Signal	Dir	Width / type	Description
f0_l2_miss_resp_valid	in	1	The Shire Cache response arrives from the fill FIFO of the Neighborhood channel.
f0_l2_miss_resp	in	et_link_rsp_info_t	Response. The L1 instruction cache uses only its opcode, which tells an error response from a data response. The line is written into the data RAM by the Shire Channel.
f0_l2_miss_resp_ready	out	1	Tied high.
Data RAM request			
f2_sram_req_valid	out	1	Request to the data RAM.
f2_sram_req_write	out	1	1: write the line of a fill into the entry. 0: read the entry.
f2_sram_req_addr	out	9	Entry of the data RAM: the set and the way, {set, way}.
f2_sram_req_ready	in	1	The data RAM accepts the request.
Data RAM response			
f0_sram_resp_valid	in	1	A line returns from the data RAM.
f0_sram_resp_dout	in	576	Line with its check bits: four 144-bit words, each with 128 data bits and 16 check bits.
f0_sram_resp_ready	out	1	Tied high.
Error logging			
esr_err_log_sbe	out	1	One-cycle pulse: a single-bit ECC error is found in a line read on a hit.
esr_err_log_dbe	out	1	One-cycle pulse: a double-bit ECC error is found in a line read on a hit.
esr_err_log_info	out	icache_err_log_info_t	Location and error bits of the error (Table 35). Loaded with each pulse.
APB debug access			
apb_paddr	in	10	Debug address. Bit 9 selects the L1 tag array (0) or a micro-cache (1) (Section 1.5.11).
apb_psel, apb_penable, apb_pwrite	in	1 each	APB control, shared by the tag array and both micro-caches.
apb_pwdata	in	64	Write data.

Signal	Dir	Width / type	Description
apb_pready, apb_pslverr	out	1 each	APB response. The responses of the tag array and both micro-caches are merged; apb_pslverr is tied low.
apb_prdata	out	64	Read data.
Status Monitor			
dbg_sm_signals	out	icache_dbg_sm_t	Tag hit vector of the lookup, miss state, and the valid and line address of the current line request, merged with the fields of both micro-caches.

Table 35. icache_err_log_info_t: ECC error information (59 bits)

Field	Width	Description
way	2	Way of the line with the error.
set	7	Set of the line with the error.
dbe_per_block	8	One bit for each 64-bit block of the line: a double-bit error in that block.
sbe_per_block	8	One bit for each 64-bit block of the line: a single-bit error in that block.
address	34	Line address of the request, physical address bits 39:6.

1.5.3. Cache Organization and Address Decode

Table 36 summarizes the organization of the L1 instruction cache.

Table 36. L1 instruction cache organization

Property	Value
Capacity	32 KB: 128 sets of four 64-byte lines
Associativity	Four-way set associative
Index	Physical address bits 12:6, 7 bits, select one of the 128 sets
Tag	Physical address bits 39:13, 27 bits, and one valid bit for each line
Line size	64 bytes, the size of the line request and of the line response
Replacement	LRU among the four ways of the set (Section 1.5.6.2)
Allocation	Every miss allocates a line, except in bypass mode. A line whose Shire Cache response is an error is not allocated.
Write policy	Read-only. Lines are written only by fills, and nothing is written back.
Coherence	None. A change to code in memory is seen only after a flush (Section 1.5.10).
Requests	One at a time, from either micro-cache

1.5.3.1. Configuration Parameters

Table 37 lists the defines that size the L1 instruction cache and its data RAM.

Table 37. L1 instruction cache configuration parameters

Define	Value	Description
ICACHE_SETS	128	Sets.
ICACHE_SET_ADDR_WIDTH	7	Set-index bits. log2(ICACHE_SETS) = log2(128) = 7
ICACHE_WAYS	4	Ways.
ICACHE_WAY_ADDR_WIDTH	2	Bits that number a way. log2(ICACHE_WAYS) = log2(4) = 2
ICACHE_BLOCK_BITS	512	Line size, 64 bytes. Shared with the micro-cache.
ICACHE_PA_TAG_SIZE	27	Tag bits. PA_SIZE - (ICACHE_SET_ADDR_WIDTH + ICACHE_BLOCK_ADDR_SIZE) = 40 - (7 + 6) = 27
ICACHE_LRU_SET_SIZE	16	LRU bits stored for one set. ICACHE_WAYS x ICACHE_WAYS = 16
ICACHE_MEM_WORDS	512	Words of one data RAM macro.
ICACHE_MEM_WIDTH	144	Bits of one word.
ICACHE_MEM_USED_BITS	128	Data bits of one word. The other 16 bits are check bits.
ICACHE_NR_MEMS	4	Data RAM macros. (ICACHE_WAYS x ICACHE_SETS x ICACHE_BLOCK_BITS) / (ICACHE_MEM_WORDS x ICACHE_MEM_USED_BITS) = 262,144 / 65,536 = 4
ICACHE_NR_MEMS_PER_ROW	4	Macros that one line spans. ICACHE_BLOCK_BITS / ICACHE_MEM_USED_BITS = 512 / 128 = 4
ICACHE_SRAM_ADD_WIDTH	9	Data RAM address bits. log2(ICACHE_MEM_WORDS) = log2(512) = 9, with one row of macros
ICACHE_SRAM_DATA_WIDTH	576	Data RAM response bits. ICACHE_NR_MEMS_PER_ROW x ICACHE_MEM_WIDTH = 4 x 144 = 576
ICACHE_ECC_BLOCKS	8	64-bit blocks of a line, each with its own 8-bit code. 512 / 64 = 8

1.5.3.2. Storage

Table 38 lists the storage of the L1 instruction cache. The tag array and the LRU array are register

files in icache_top, inside the Neighborhood; the data RAM is in the Shire Channel (Section 1.5.7).

Table 38. L1 instruction cache storage

Structure	Organization	Bits
Tag array	4 register files, one for each way, of 128 entries of a 27-bit tag; and 512 valid bits in flip-flops	14,336
LRU array	128 entries of 16 bits; and 128 flip-flops that record which sets were updated since reset	2,176
Data RAM	4 SRAM macros of 512 words of 144 bits: for each of the 512 lines, 512 data bits and 64 check bits	294,912

1.5.3.3. Address Decode

A line request carries the 40-bit physical address of the line. The L1 instruction cache divides it into three fields: bits 39:13 are the 27-bit tag, compared with the four ways of the set; bits 12:6 select one of the 128 sets; and bits 5:0 are not used, because the request is always for a whole line. A line is stored in the data RAM at the entry {set, way}, so the four ways of a set are in four consecutive words.

Figure: L1 instruction cache address decode.

1.5.4. Request Arbitration Between L0 Micro-Caches

The L1 instruction cache has one request port, and two micro-caches send line requests to it. Each micro-cache therefore has a holding register in icache_top. The miss handler of a micro-cache drives its line request until the holding register is free (f0_l1_miss_req_ready). The holding register then takes the address, and the miss handler moves to Fill_Wait (Section 1.4.8.1).

The two holding registers bid in a round-robin arbiter (arb_rr_data), which grants one of them when the L1 instruction cache can accept a request. The granted holding register is free again in the next cycle. When both holding registers bid, the grants alternate, so a micro-cache never waits for more than one request of the other.

The index of the granted micro-cache is registered with the request, and the line response goes back to that micro-cache only: f0_l1_miss_resp_valid and f0_l1_miss_resp_early_valid are driven to it alone, while the line and its error flags are driven to both micro-cache instances. A request that loses the arbitration waits for the complete service of the other one. If both micro-caches send a line request in the same cycle and both hit, one receives its line 13 cycles after its line request, and the other 26 cycles after.

1.5.5. Pipeline and Hit Timing

The L1 instruction cache serves a request in three stages, F0 to F2. These stages are its own and are numbered independently of the micro-cache stages. Table 39 lists them, together with the arbitration cycle before F0.

Table 39. L1 instruction cache pipeline stages

Stage	Operation
Arbitration	The round-robin arbiter grants a holding register. The tag array and the LRU array start to read the set of the granted address.
F0	The request and the index of its micro-cache are registered. The four tags of the set, their valid bits and the LRU state are available, and the LRU state selects the victim way.
F1	The tag is compared with the four ways. On a hit, the data array control requests the line at {set, hit way}; on a miss, the miss state machine takes the miss (Section 1.5.8). The LRU state records the way that hits or the victim.
F2	The data RAM request is registered and sent to the Shire Channel.

After F2 the request crosses into the Shire Channel, and the line returns from the data RAM six cycles later, after the access and both crossings. The data array control registers the line, checks and corrects it in the next cycle, and registers the corrected line with its error flags (Section 1.5.6.3). The line response leaves for the micro-cache one cycle later.

Table 40 lists the steps of a line request that hits. Cycle 0 is the cycle in which the miss handler of the micro-cache drives the line request. The line reaches the micro-cache 13 cycles later, and the L1 instruction cache accepts its next line request one cycle after that, so it serves at most one line request every 13 cycles. Seen from the core, a core request that misses in the micro-cache and hits in the L1 instruction cache has its line written into the micro-cache 16 cycles after the grant, and fill done follows three cycles later (Section 1.4.8.3).

Table 40. L1 hit, steps by cycle

Cycle	Block	Action
0	Micro-cache, holding register	The miss handler drives the line request, and the holding register takes the address.
1	Arbiter, tag and LRU arrays	The arbiter grants the request. The tag array and the LRU array start to read the set.
2	F0	The request is registered. The LRU state selects the victim way.
3	F1	One way hits. The data array control requests the line at {set, hit way}, and the LRU state records the way.
4	F2	The data RAM read request leaves for the Shire Channel.
10	Data RAM	The line and its check bits return.
11	Data array control	The eight blocks are checked and corrected.
12	Data array control	The corrected line and its error flags are registered. `f0_l1_miss_resp_early_valid` is driven to the micro-cache.
13	Line response	`f0_l1_miss_resp_valid` is driven, and the micro-cache writes the line.
14	Arbiter	The L1 instruction cache can accept the next line request.

Figure: L1 hit timing.

1.5.6. Tag, LRU and Data Arrays

The tag array and the LRU array work on the set of the request: the tag array tells which way holds the line, and the LRU array tells which way a miss replaces. The data array control connects the L1 instruction cache to its data RAM and checks every line that returns from it.

1.5.6.1. Tag Array

The tag array holds the tags of the 512 lines, in one register file for each way (`icache_tag_data_array`, 128 entries of 27 bits). Each register file has two read ports, one for the lookup and one for APB debug reads, and one write port. The valid bits, four for each set, are flip-flops beside the register files. Table 41 lists the ports of the array.

Table 41. L1 tag array ports

Port	Stage	Function
Read	Arbitration, F0	Reads the four tags and valid bits of the set; they are registered in F0.
Victim	F1	Clears the valid bit of the victim of a miss, outside bypass mode.
Write	Fill	Writes the tag of the new line into the victim way and sets its valid bit when the Shire Cache response arrives. Nothing is written for an error response or in bypass mode, and after a flush the valid bit stays clear (Section 1.5.10). A double-bit ECC error clears the valid bit of the way that hits through the same port. APB debug writes share the port, and the fill has priority.
Flush	–	`f0_flush_data` clears all valid bits (Section 1.5.10).
APB	–	Reads the tags and valid bits of two ways through the second read port, and writes them through the write port (Section 1.5.11).

Lookup. In F1 each of the four ways is compared with PA[39:13]. A way hits when it is valid, its tag matches, and no flush is in progress. A line is allocated only after a miss, so at most one way hits.

1.5.6.2. LRU Array

The LRU array (`icache_lru_array`) keeps the order of use of the four ways of each set. For each pair of ways it holds one bit that tells which of the two was used more recently, and the bits of a set are stored in a 16-bit entry. In F0 the entry of the set selects the victim, the way that is older than the other three. In F1 the entry is updated: the way that hits, or the victim of a miss, becomes the most recent one.

The LRU state is updated once for each request, and not again when the line of a miss arrives. The L1 instruction cache serves one request at a time, so no other access changes the order of the set while the line is outstanding. In bypass mode the LRU state is not updated. The victim does not depend on the valid bits, so an invalid way is not chosen first.

The LRU register file has no reset. One flip-flop for each set records whether the set has been updated since reset, and until then the entry reads as zero, which selects way 0 as the victim.

1.5.6.3. Data Array Control

The data array control (`icache_data_array`) turns a hit into a read request and the acceptance of a Shire Cache request into a write request. Both requests address the entry {set, way}: the way that hits for a read, and the victim for a write. The request is registered in F2 and held until the data RAM accepts it.

The data RAM returns a line as four 144-bit words, 576 bits. Each word holds two 64-bit blocks and their two 8-bit codes, so the line has eight blocks, each protected by its own code. The code corrects a single-bit error and detects a double-bit error in its block. The data array control registers the response, checks and corrects all eight blocks in the next cycle, and registers the corrected line together with a single-bit and a double-bit error flag for each block (Section 1.5.9).

1.5.7. External Data RAM

The data RAM holds the data of all 512 lines. It is in the high-voltage region of the Shire Channel, in `icache_data_ram_wrap`, and the Shire Channel holds one such wrapper for each of the four Neighborhoods of the Shire (`icache_mems`). The wrapper contains four SRAM macros (`icache_data_ram`) of 512 words of 144 bits. One line spans the four macros, one word in each, and the four macros are read and written together. The line {set, way} is at word set x 4 + way.

The L1 instruction cache reaches the wrapper through the voltage-domain crossing (Section 1.1.3). `f2_sram_req_*` carries the address and the write flag, and `f0_sram_resp_*` returns 576 bits. A read returns the stored line with its check bits. A write carries no data.

The data of a write comes from the Shire Cache response itself. That response passes the Shire Channel on its way to the Neighborhood, and the wrapper watches it: it takes the response addressed to the instruction cache of its Neighborhood, and generates the eight 8-bit codes of the line. A write request waits in the wrapper until that response arrives. The wrapper then writes the line and its codes into the four macros, and returns the same line and codes as its response, so the new line reaches the L1 instruction cache through the same path as a line read on a hit.

The wrapper serves a request in five stages of its own, F0 to F4. F0 accepts the request. F1 enables the macros, and holds a write until its line arrives. The macros need two cycles for an access, F1 and F2. F3 receives the output of the macros, and F4 registers it and returns the response.

The wrapper also gives the debugger and the memory BIST access to the macros (Section 1.5.11). Two DFT bits of the RAM configuration ESR, `esr_shire_cache_ram_cfg`, gate the chip enable and the write enable of the macros.

Figure: L1 data RAM organization.

1.5.8. Miss Path

A line request that misses in F1 is served from the Shire Cache. The miss state machine of the L1 instruction cache handles the miss in three states:

- **Ready:** no miss is outstanding. A miss in F1 moves the state machine to Request. The cache serves one request at a time, so the state machine is always in Ready when a request reaches F1.
- **Request:** drives the Shire Cache request, a read of the full line at its line-aligned physical address, until the memory request arbiter of the Neighborhood accepts it (Section 1.7.2). When the request is accepted, the data array control sends the write request for the victim entry to the data RAM, and the state machine moves to Fill_Wait.
- **Fill_Wait:** waits for the line. The state machine returns to Ready when the line response is sent to the micro-cache.

Figure: L1 miss state machine.

In F1 the miss also frees the victim: the LRU state chose the victim in F0, its valid bit is cleared, and the LRU state records it as the most recent way. The valid bit stays clear until the tag of the new line is written.

The Shire Cache response completes the fill in two places. On its way to the Neighborhood it passes the Shire Channel, where the data RAM, which holds the write request, writes the line into the victim entry and returns it (Section 1.5.7). In the Neighborhood the response reaches the L1 instruction cache through the fill FIFO, and the tag array writes the tag of the line (Section 1.5.6.1). The line response is sent to the micro-cache when both are present: the line from the data RAM, and the Shire Cache response, which carries the error status of the line. It leaves nine cycles after the Shire Cache response reaches the data RAM.

The time of a miss depends on the Shire Cache access, which is not fixed. Table 42 lists the steps of a line request that misses. Cycle *A* is the cycle in which the memory request arbiter accepts the Shire Cache request, cycle *S* the cycle in which the Shire Cache response reaches the data RAM, and cycle *F* the cycle in which the same response reaches the L1 instruction cache through the fill FIFO.

Table 42. L1 miss, steps by cycle

Cycle	Block	Action
0 to 2	Micro-cache, arbiter, F0	The line request is taken, granted and registered, and the LRU state selects the victim way.
3	F1, miss state machine	No way hits. The miss state machine moves to Request. The valid bit of the victim is cleared, and the LRU state records the victim.
4	Miss state machine	The Shire Cache request is driven to the memory request arbiter of the Neighborhood, and held until accepted.
A	Data array control	The write request for {set, victim way} is sent to the data RAM. The miss state machine moves to Fill_Wait.
S	Data RAM	The Shire Cache response arrives. The data RAM writes the line and its check bits into the victim entry and returns them.
F	Tag array	The tag of the line is written into the victim way, and its valid bit is set.
S + 9	Line response	`f0_l1_miss_resp_valid` is driven, the micro-cache writes the line, and the miss state machine returns to Ready.

1.5.9. Errors and ECC

A line can come with two kinds of error: an ECC error found in the data RAM, and an error response from the Shire Cache. The micro-cache stores both flags with the line and returns them with every hit on it (Section 1.4.6.2).

ECC errors. The data array control checks the eight blocks of every line that the data RAM returns. A single-bit error is corrected, so the micro-cache receives correct instructions; a double-bit error cannot be corrected. The error flags are used only for a line read on a hit; for the line of a fill, which the data RAM returns as it writes it, they are not used. An ECC error has three effects:

- **Report to the micro-cache:** `f0_l1_miss_resp_ecc_err` is set with the line if the class of the error is enabled in the error-log control ESR: bit 0 for single-bit errors, bit 1 for double-bit errors.
- **Log:** `esr_err_log_sbe` or `esr_err_log_dbe` pulses, and `esr_err_log_info` carries the way, the set, the error bits of each block and the line address to the Neighborhood ESR block, which logs the error (Section 1.9.4).
- **Invalidation:** on a double-bit error the valid bit of the way is cleared, so the next request for the line misses and fetches it again from the Shire Cache.

Shire Cache errors. If the Shire Cache returns an error response for the Shire Cache request, the tag is not written, so the line is not allocated. `f0_l1_miss_resp_l2_err` is set with the line response of that miss only.

1.5.10. Flush, Bypass Mode and Prefetch

The flush and the two special requests of the micro-cache also reach the L1 instruction cache:

- **Flush:** the flush of the instruction caches arrives on the same `f0_flush_data` as at the micro-caches (Section 1.4.9). In the flush cycle the valid bits of all 512 lines are cleared, and no lookup hits. A miss outstanding at the flush still writes its tag when the Shire Cache response arrives, but the line stays invalid, because the flush is remembered until the miss state machine returns to Ready. The LRU state does not change.
- **Bypass mode:** every lookup misses, and the L1 instruction cache writes no tag and does not update its LRU state, so it keeps no copy of the line. The line still returns to the micro-cache through the data RAM, as explained for bypass mode (Section 1.4.10.1).
- **Prefetch:** a line request of the code prefetcher is handled like any other line request, and its line is kept in the cache (Section 1.4.11.1).

1.5.11. Debug Access

The tag array and the data RAM of the L1 instruction cache are reached through two different APB paths. The tag array is read and written through the same 10-bit address as the micro-caches (Table 33). Bit 9 is 0 for the tag array, bits 7:1 select the set, and bit 0 selects a pair of ways: 0 for ways 0 and 1, 1 for ways 2 and 3. Bit 8 is not used.

- **Read:** the 64-bit read data returns both ways of the pair, the even way in bits 31:0 and the odd way in bits 63:32. Each half holds the tag in its bits 26:0 and the valid bit in its bit 27. The read uses the second read port of the tag array and does not wait for the lookup.

- **Write:** writes the tags and valid bits of both ways from the same bit positions. A write waits for a cycle in which no fill writes the tag array.

The data RAM is read and written over the APB of the Shire Channel, through the wrapper (Section 1.5.7). Three upper address bits select the Neighborhood, and the 13-bit address within the wrapper selects the word and the part of it:

- **Bits 12:4:** word of the four macros, which is the line {set, way}.
- **Bit 3:** 0 for a 64-bit data block, 1 for the check bits of the word, the four 16-bit check fields in one 64-bit value.
- **Bits 2:0:** data block of the line, 0 to 7.

A read returns the selected 64 bits. A write is a read-modify-write of the whole word: the word is read, the selected 64 bits are replaced, and the word is written back. A debug access waits for a cycle with no request from the L1 instruction cache.

`apb_pslverr` is never asserted by the tag array or by the wrapper.

1.6. Shared Page-Table Walker

1.6.1. Clients and Arbitration

1.6.2. Walk Sequence and Page Sizes

Figure: Page-table walk state machine.

1.6.3. PTE Cache and Invalidation

1.7. Memory Request and Response Path

1.7.1. Miss, Evict and Fill FIFOs

Figure: Neighborhood memory request and response path.

1.7.2. Request Arbitration

1.7.3. Response Routing and Backpressure

1.8. Tensor Support

1.8.1. Cooperative Tensor Load

1.8.2. Tensor Store Buffer

1.8.3. TBOX Router

1.9. Synchronization, Messaging, Registers and PMU

1.9.1. Fast Local Barrier

1.9.2. Local Message Network

1.9.3. IPI and FCC Paths

1.9.4. Neighborhood Registers and APB Access

1.9.5. Debug Channel

1.9.6. Performance Monitoring Unit